



Lecture NO.01

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk



Outline

1 Introduction

- Installation
- Starting MATLAB
- Use of the Command Window
- Online Help
- MATLAB Windows
- Ending a Session



Installation

If you intend to run MATLAB on a PC, particularly the Student Version, it is quite possible that you will have to install it yourself. The following points can be beneficial for installation of the MATLAB

- The task can be easily accomplished using the product CDs.
- Follow the installation instructions as you would with any new software installation.
- During the installation you may be asked which *toolboxes* you wish to install.
- We suggest that you install any that seem of interest to you or that you think you might use at some point in the future.
- But for the purpose of this course , you should be sure to include the *Symbolic Math Toolbox*.



Starting MATLAB

You start MATLAB as you would any other software application:

- On a PC you access it via the **Start** menu, in the **programs** folder under MATLAB R2010a or MATLAB 7.0.
- Alternatively you may have a desktop icon that enables you to start MATLAB with a simple-double click
- As you start of MATLAB, you will briefly see a window that displays the MATLAB logo alongwith some MATLAB product information, like version of MATLAB etc.
- Then a MATLAB desktop window will launch.
- That window will contain a title bar, a menu bar, a tool bar, and four and embedded windows, one of which is hidden.
- The largest and most important window is the **Command Window** on the left.



Starting MATLAB

- Later we will go into more detail on the use and manipulation of the other three windows: the *Workspace Browser*, the *Command History Window*, and the *Current Directory Browser*.
- Now, we focus on Command Window in order to get you started issuing MATLAB commands.
- At the top of Command Window, you may see some general information about MATLAB, the most important of all, you will see a command prompt (»).
- If the Command Window is active, its title bar will be dark, and the prompt will be followed by a cursor (a blinking vertical line). That is the place where you will enter your MATLAB Commands.

Starting MATLAB

- Figure 1 contains an example of a newly launched MATLAB Desktop.

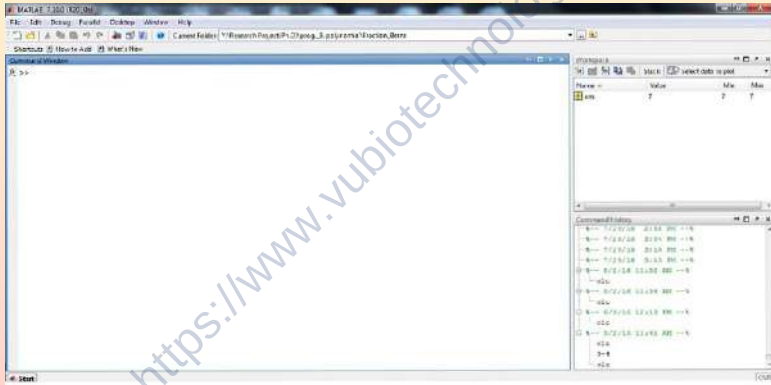


Figure: MATLAB Desktop



Use of the Command Window

To type the mathematics in a Command Window follow the following points:

- Click in the Command Window to make it active.
- When a Window becomes active, its titlebar darkens and a blinking cursor appear after the prompt.
- Now you in a position to enter simple commands in a Command Window.



Use of the Command Window

- Type $2 + 5$, then press ENTER key, the output will be 7.
- Type `factor(987654321)` and press ENTER key.
- Type `factor(9)`.
- Type `factor(10)`.
- Your MATLAB desktop looks like the Figures 2 and 3.



Use of Command Window

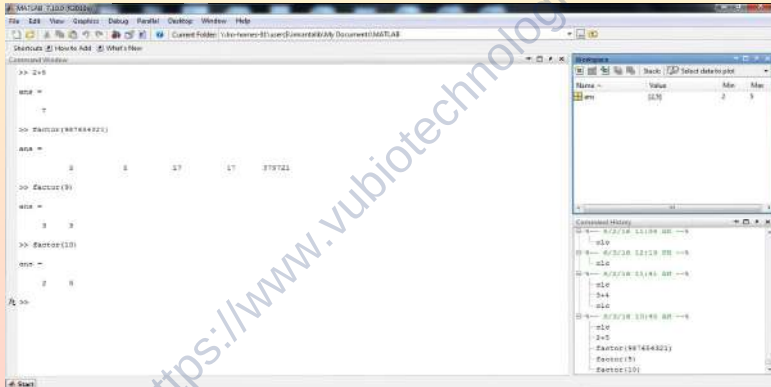


Figure: (2) MATLAB Desktop with several simple commands



Use of the Command Window

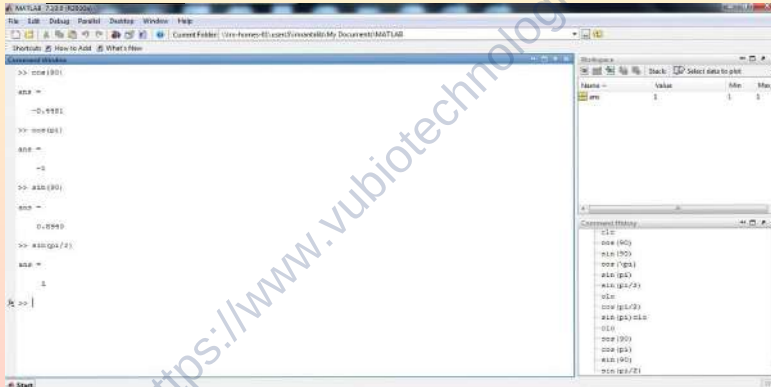


Figure: (3) MATLAB Desktop with several simple commands



Online Help

MATLAB has extensive online help that can be gotten in one of the several ways

- Typing **help** at the command prompt will reveal a long list of topics for which help is available.
- To see a general purpose MATLAB Commands, type **help general** will reveal a long list of general help commands.
- To learn about the solve commands, type **help solve**.
- Your MATLAB Desktop looks like the Figures 4, 5, and 6.



Online Help

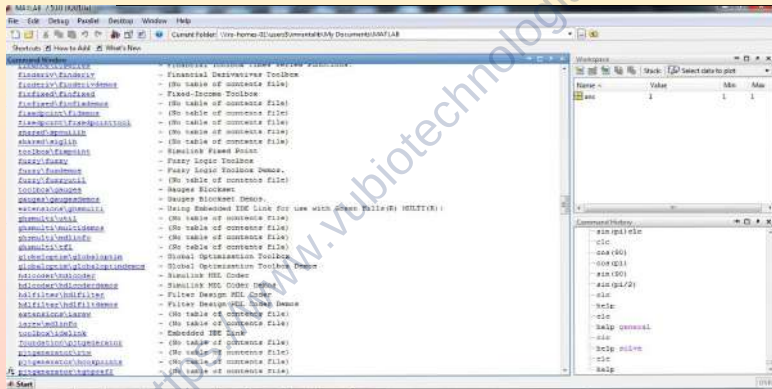


Figure: (4) MATLAB Desktop with help commands



Online Help

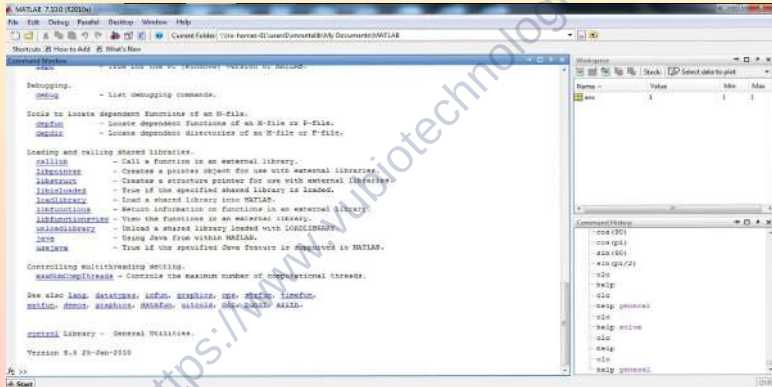


Figure: (5) MATLAB Desktop with help general commands



Online Help

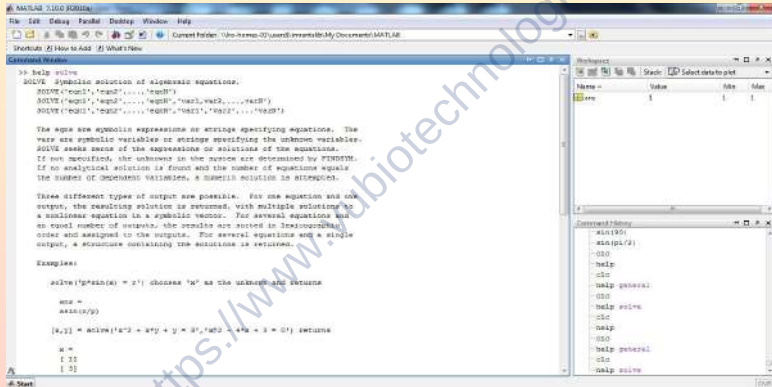


Figure: (6) MATLAB Desktop with help solve commands



Online Help

There is much more user friendly way to access the online help, namely via the MATLAB **Help Browser**. It can be activated in a several way. For example

- Type helpwin, helpdesk or doc in the Command Window.
- Your MATLAB Desktop looks like the Figures 7, 8, and 9.

Online Help

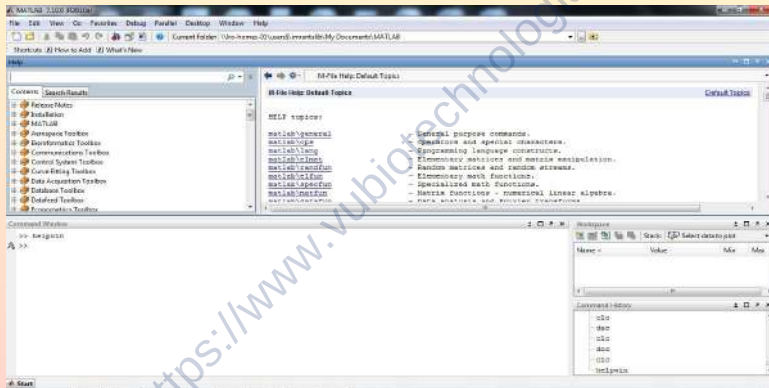


Figure: (7) MATLAB Desktop with helpwin command



Online Help

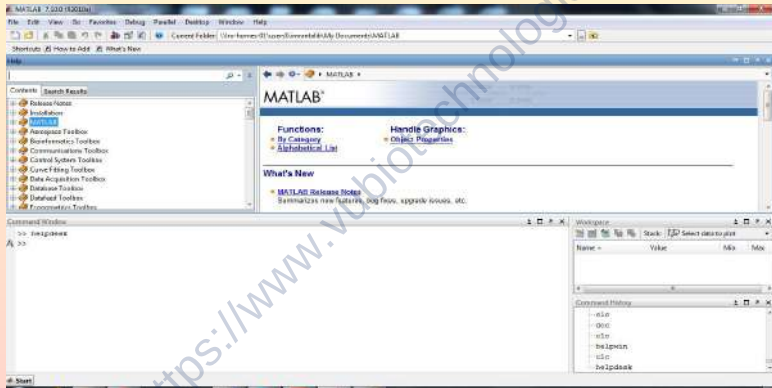


Figure: (8) MATLAB Desktop with helpdesk command



Online Help

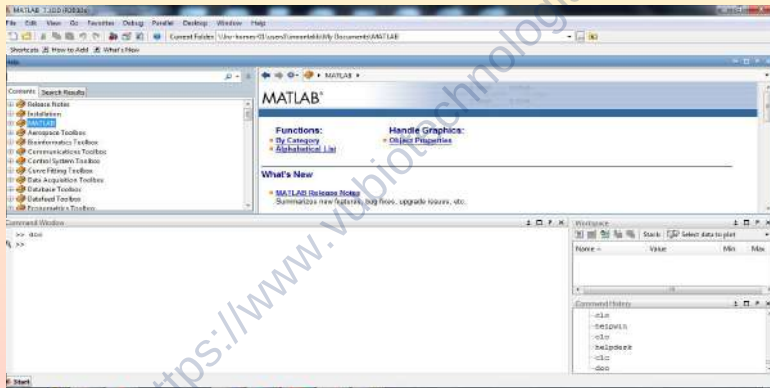


Figure: (9) MATLAB Desktop with doc command



MATLAB Windows

There are several windows you will encounter as you work with MATLAB, like

- Command Window
- Help Browser
- Command History Window
- Current Dictionary Browser
- Workspace Browser
- Editor (M-files)

Some of these windows launched separately, and some are embedded in the Desktop through the Desktop menu button. We will discuss later in detail the functionality of the Editor.



MATLAB Windows

The Matlab Desktop display after embedding the different windows inside the Desktop, see the following Figure

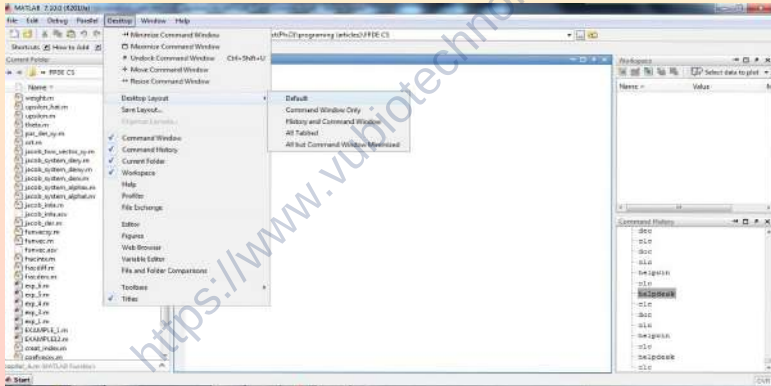


Figure: The Matlab Desktop display after embedding the different windows inside the Desktop

[visit our website to get more PDF](https://www.vubiotechnologist.com)

<https://www.vubiotechnologist.com>



Ending a Session

- The simplest way to conclude a MATLAB session is to type **quit** in the Command Window.
- Special symbol can also be used (like X in the upper right hand corner) that closes your window.
- **Exit MATLAB** option from the File menu of the Desktop can also be used to close the window.

Doing Mathematics with Matlab (I)

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk



Outline

- 1 Doing Mathematics with Matlab (I)
 - Arithmetic
 - Symbolic Computation
 - Substitution in Symbolic Expressions
 - Errors in input

- 2 Exercise



Outline

- 1 Doing Mathematics with Matlab (I)
 - Arithmetic
 - Symbolic Computation
 - Substitution in Symbolic Expressions
 - Errors in input
- 2 Exercise



Arithmetic

This lecture describes some of the basic MATLAB commands that will be used in this course. For example

- $+$ command is used for addition.
- $-$ command is used for subtraction.
- For division, we use $/$.
- $*$ command is used for multiplication.
- The MATLAB Desktop looks like the Figure 1 after performing these commands.

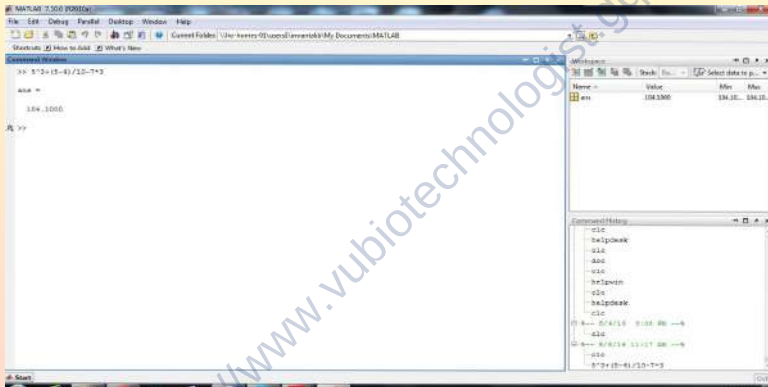


Figure: (1) The Matlab Desktop display after performing some basic arithmetic commands



Arithmetic

- Note that in the Figure 1, MATLAB prints the answer and assigns the value to a variable called **ans**.
- If the further calculation is needed then simply use the variable **ans**, rather than retype the answer.
- It is to be noted that MATLAB assigns a new value to **ans** with each calculation.
- For example the square root of the previous answer can be taken using the command **sqrt(ans)**. The output is displayed in the Figure 2

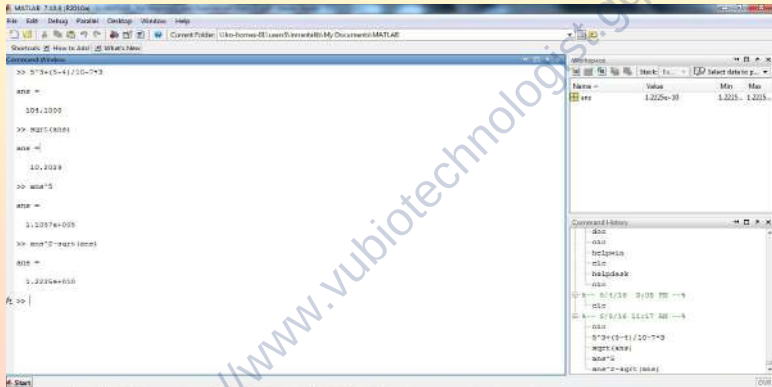


Figure: (2) The Matlab Desktop display after performing some basic arithmetic commands



Arithmetic

- On the other hand, sometimes while doing complex calculations, we prefer to assign the variables of our own choice to computed values.
- So, MATLAB gives us independence to assign variables of our own choice.
- For example, $\sin^2(1) + \cos^2(1)$ can be computed by assigning a variable to $\sin(1)$, and b variable to $\cos(1)$. The results are displayed in Figure 3.

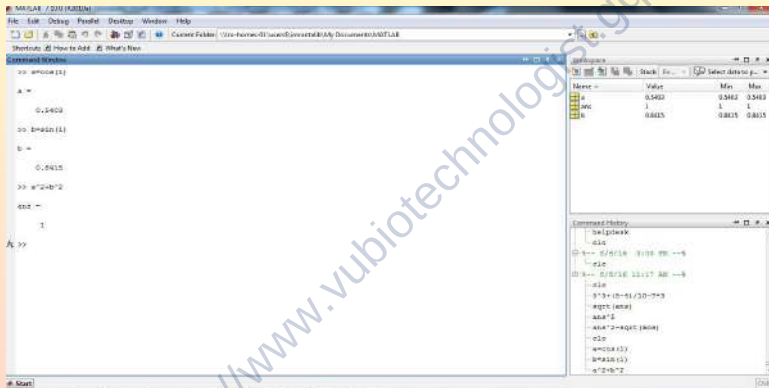


Figure: (3) The Matlab Desktop display after performing some basic arithmetic commands

Symbolic Computation

- To perform symbolic calculations, make sure that **Symbolic Toolbox** is installed on your system.
- For this type **help symbolic** in the Command Window and run this command by pressing enter key.
- If it is installed then the MATLAB desktop looks like the Figure 4, otherwise you have to install it like the installation of the other packages.

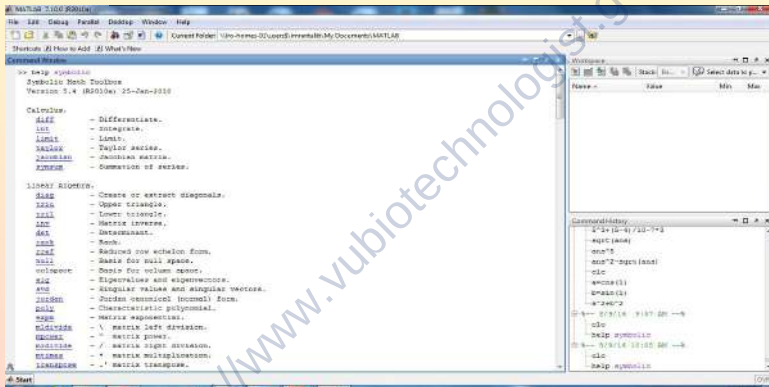


Figure: (4) The Matlab Desktop display with help symbolic command

Symbolic Computation

- To do symbolic computations, you must use syms to declare the variables you plan to use to be symbolic variables.
- Usually this command is used to solve algebraic expressions. For example
- To compute $(x - y) * (x - y)^2$, declare x and y to be symbolic.
- The MATLAB Desktop looks like Figure 5.

The screenshot shows the MATLAB Desktop environment. The Command Window on the left contains the following code:

```
>> syms x y
>> (x-y)*(x-y)^2

ans =

(x - y)^3

>> expand(ans)

ans =

x^3 - 3*x^2*y + 3*x*y^2 - y^3

>> factor(ans)

ans =

(x - y)^3

>> simplify((x^2-y^2)/(x*y))

ans =

y*(x^2 - y^2)/x

>> simplify((x^2-y^2)/(x*y))

ans =

x - y
```

The Workspace window on the right shows three variables: `ans`, `x`, and `y`, all of type `sym`. The Command History window at the bottom right shows the sequence of commands entered.

Figure: (5) The Matlab Desktop display after declaring x and y to be symbolic

Symbolic Computation

Remark

It is to be noted that MATLAB uses floating point arithmetic for its calculations. But exact arithmetics can also be done using the Symbolic Math Toolbox with symbolic expressions.

*For example, the floating point format of $\cos(\pi/2)$ in MATLAB is $6.1232e - 17$. However, it is known that $\cos(\pi/2)$ is really equal to zero. The inaccuracy is due to the fact that typing **pi** in MATLAB gives an approximation to π accurately to about 15 digits. But sym command gives accurate value of π . Look at the Figure 6*

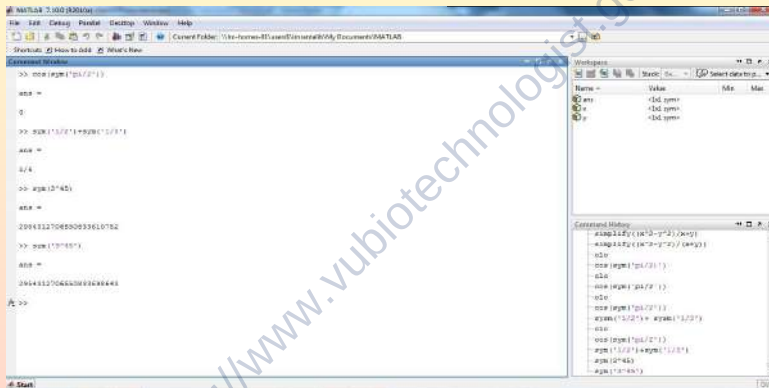


Figure: (6) The Matlab Desktop display with sym command

Substitution in Symbolic Expressions

- While working with symbolic expressions, some times it is necessary to substitute a numerical value, or even another symbolic expression, for one or more of the original variables in the expression.
- **subs** command is used for this purpose.
- For example, we want to substitute the numerical value 2 for the symbolic variable x in the symbolic expression $u = x + y^2$.
- For this we use the command **subs(u,x,2)**.
- Similarly, we can substitute the symbolic expression $y + x$ for the symbolic variable y in the symbolic expression $u = x + y^2$ as, **subs(u,y,y+x)**.
- The MATLAB Desktop looks like the Figure 7.

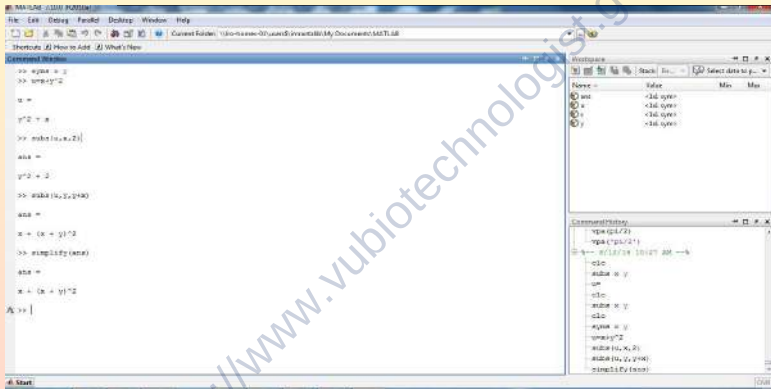


Figure: (7) The Matlab Desktop display with substitution command

Errors in input

- If you make an error while entering input in a Command Window, MATLAB will beep and print an error message.
- For example, note that what happens when you try to evaluate *sin10*:
- » *sin10* ??? Undefined function or variable '*sin10*'.
- Note that there you did not write the correct command. The correct command is *sin(10)*. Then Matlab gives you the correct output. See Figure 8.

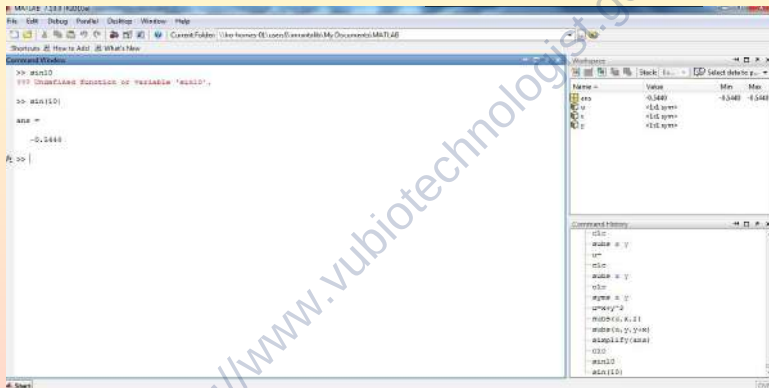


Figure: (8) The Matlab Desktop display while removing error



- ❶ Find the value of the following polynomials at $x = 2$ using **subs** command:
 - $p(x, y) = x^2 + 3y^2x + 10$
 - $u(x) = x^3 + 4x + 10$
- ❷ Find the value of the following trigonometric functions at $x = \frac{\pi}{2}$ using **sym** command:
 - $\cos(x), \sin(x), \tan(x)$.



1 Expand the following algebraic expressions using **expand** command:

- $(x + y)^{10}$
- $(x + y)^3 + (x + y)^2$

2 Simplify the following quantities:

- $2 + 5 - 4 + 10^{10} + 7^2$.
- $2/5 + 4/10 - 100^2$



Doing Mathematics with Matlab (II)

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk



Outline

1 Doing Mathematics with MATLAB II

- Variables and Assignments
- Vectors
- Built-in Functions
- User Defined Functions
- Solving Algebraic Equations

2 Exercise



Outline

1 Doing Mathematics with MATLAB II

- Variables and Assignments
- Vectors
- Built-in Functions
- User Defined Functions
- Solving Algebraic Equations

2 Exercise



Variables and Assignments

- In MATLAB, = sign is used to assign values to a variable.
- For example, to assign 7 value to a variable x, we proceed as
 - » x=7
 - x =
7
- From now to own-ward, whenever MATLAB sees the letter x, it will substitute the value 7.
- For example, if y has been defined as a symbolic variable, then
 - » x=7
 - x =
7
 - » syms y » $x^3 + 2 * x * y + 10 * x$
 - ans =
 $14 * y + 413$



Variables and Assignments

- To assign a new value to a variable x , type **clear x** command.
- For example, to assign 8 value, we have to proceed as
 - » syms y
 - » clear x » x=8
 - x =
 - 8
 - » $x^3 + 2 * x * y + 10 * x$
 - ans =
 - $16 * y + 592$
- The MATLAB Desktop looks like the Figure 1

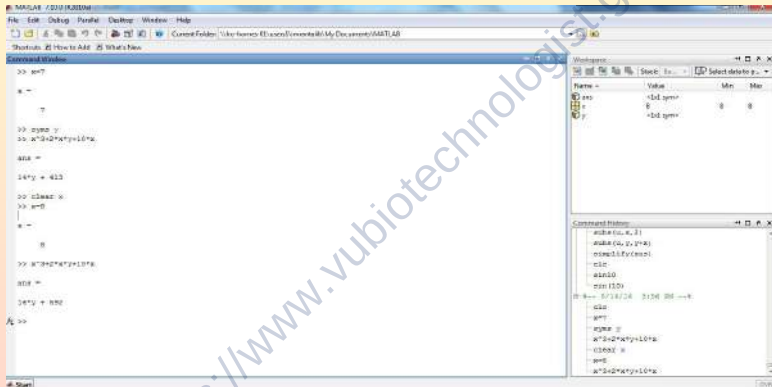


Figure: The Matlab Desktop display with Variable assign command



Vectors

A vector is an ordered list of numbers. A vector of any length can be entered in MATLAB as

- By typing a list of numbers, separated by commas or spaces, inside square brackets. For example
- A vector X with entries 2, 4 and 5 can be generated as
 - » $X=[2,4,5]$
 - $X =$
 - 2 4 5
- On the same fashion a vector Y with entries 2, 7, and 8 can be generated as
 - » $Y=[2,7,8]$
 - $Y =$
 - 2 7 8

Vectors

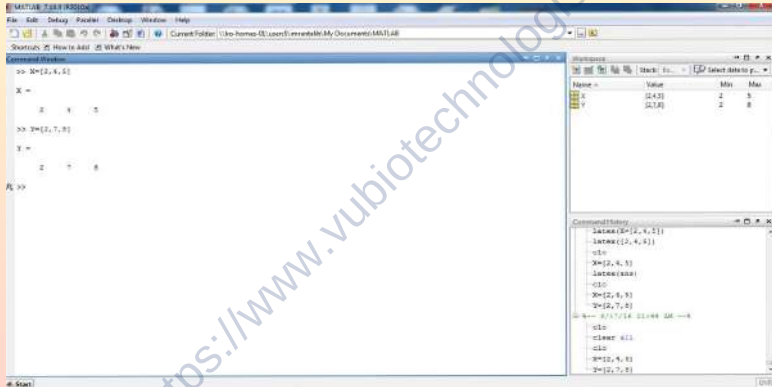


Figure: The Matlab Desktop display with Vector command



Vectors

- Vector can also be created without typing each number. For example
- The vector X with entries 2, 3, 4 and 5 can also be created as
 - » $X=2:5$
 $X =$
2 3 4 5
- The notation $X=2:5$ represents a vector of numbers running from 2 to 5 with increment 1, that increment can also be specified as the second of three arguments as
 - » $Y=1:2:12$
 $Y =$
1 3 5 7 9 11



Vectors

- To change a vector Y from a row vector to a column vector, simply put a prime (') after Y , as
- » Y'
ans =
1
3
5
7
9
11
- You can also extract the entries of vector Y . For example, the entry 5 can be extracted as
- » $Y(3)$
ans =
5

Vectors

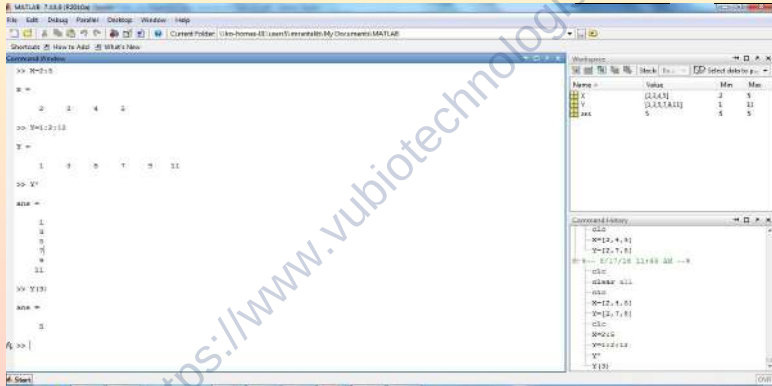


Figure: The Matlab Desktop display with increment command



Vectors

- Mathematical operations can also be performed on vectors. For example, to square the elements of the vector Y , use the following command
- $Y.^2$
ans =
1 9 25 49 81 121
- Similarly you can perform other mathematical operations, see the Figure 4

Vectors

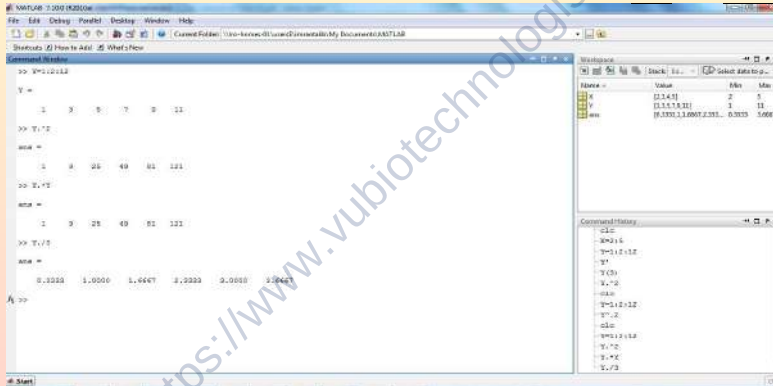


Figure: The Matlab Desktop display with operations on vectors



Built-in Functions

- MATLAB has all the usual **elementary functions** built in. For example,
- **exp(x)** indicates the exponential function of x .
- **log(x)** represents the natural logarithm function of the argument x .
- **sqrt** is used to take the square root of an entity.



User Defined Functions

- We can also define our own functions in MATLAB using the command **inline** or using the operator **@**.
- We prefer the latter command. For example, the function x^3 can be defined as
- $\gg f = @(x)x^3$
f =
@(x)x³



User Defined Functions

- Once the function is defined, it can be evaluated at any real number. For example
- To calculate the value of x^3 at $x = 10$, we will proceed as
- ```
» f = @(x)x^3
f =
@(x)x^3
» f(10)
ans =
1000
```
- See the Figure 5

# User Defined Functions

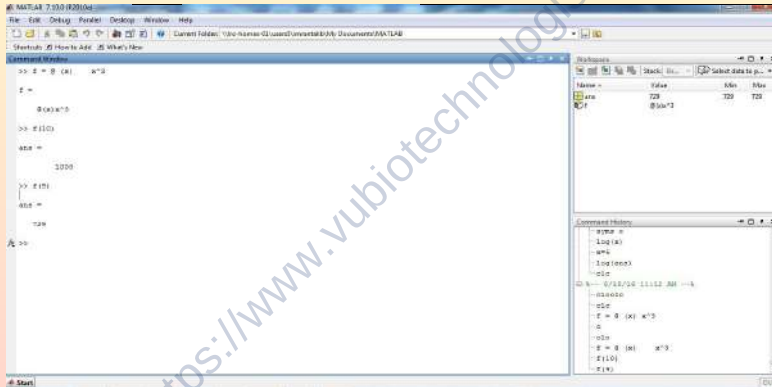


Figure: The Matlab Desktop display with user defined function command



## User Defined Functions

- This command can also be used to define functions of several variables. For this we proceed as
- ```
» h = @(x,y)x^2 + y^2
```



```
h =
```



```
@(x,y)x^2 + y^2
```
- $h(x, y) = x^2 + y^2$ can also be evaluated at different values of x and y as,
- ```
» h(1,2)
```

```
ans =
```

```
5
```
- See the Figure 6



# User Defined Functions

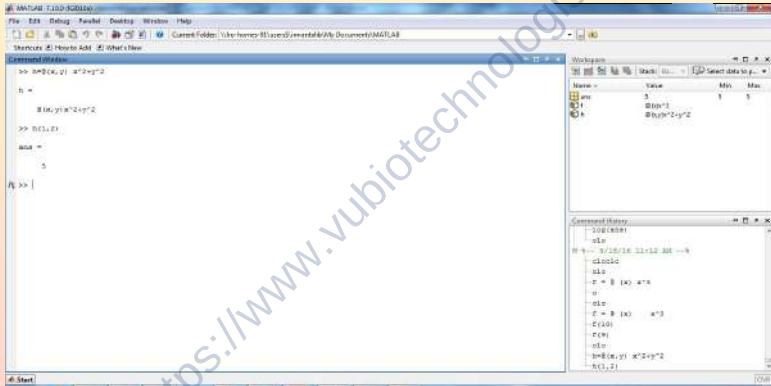


Figure: The Matlab Desktop display with user defined function command



# Solving Algebraic Equations

Now, we learn how to solve the Algebraic equations in MATLAB. The command **solve** or **fzero** is used for this purpose. For example,

- The MATLAB solution of the equation  $x^2 - 5x + 6 = 0$  can be find as
- » `solve('x^2 - 5 * x + 6 = 0')`.
- The MATLAB output is
- `ans =`  
2  
3
- Note that here the equation to be solved is specified as a string, that is, it is surrounded by a single quotes.
- See the Command Window 7

# Solving Algebraic Equations

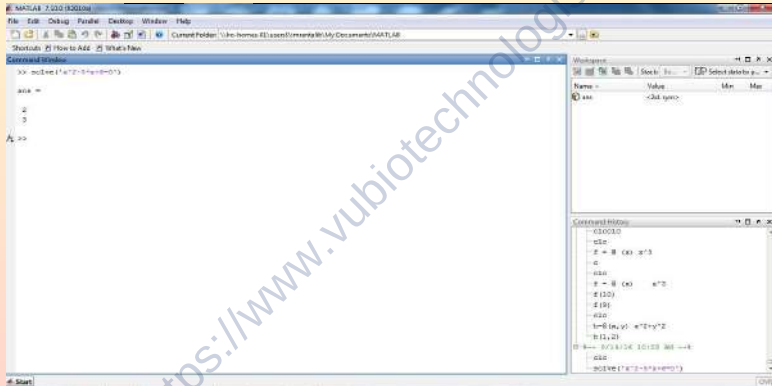


Figure: The Matlab Desktop display with Algebraic equations solver



# Solving Algebraic Equations

Note that, the input to **solve** can also be a symbolic expression. For instance

- The MATLAB syntax for solving  $x^2 - 3x = -7$  is:

- `syms x;`  
`solve(x2 - 3 * x + 7)`

- The MATLAB generated output is

- `ans =`  
 $3/2 - (19^{(1/2)} * i)/2$   
 $3/2 + (19^{(1/2)} * i)/2$





# Solving Algebraic Equations

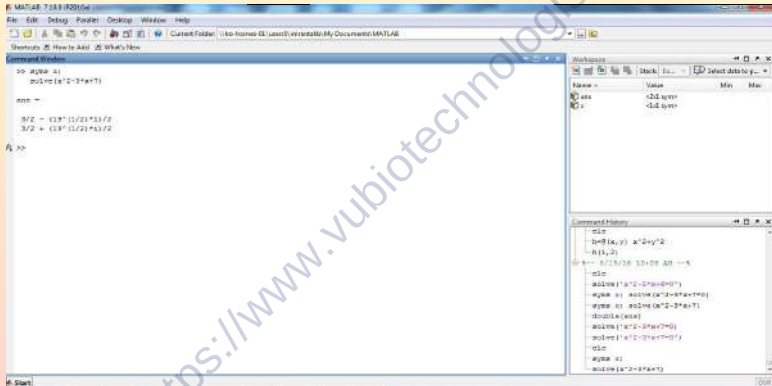


Figure: The Matlab Desktop display with Algebraic equations symbolic command



# Solving Algebraic Equations

- The **solve** command can also be used to solve higher degree polynomial equations, as well as many other types of equations.
- It can also be used to solve equation having more than one variables. For example to find the solution of  $x^2 - y = 2$  and  $y - 2x = 5$ , the following syntax is used:
  - » `[x,y] = solve('x^2 - y = 2','y - 2 * x = 5')`
- The MATLAB generated output is:
  - $x =$   
 $2 * 2^{(1/2)} + 1$   
 $1 - 2 * 2^{(1/2)}$

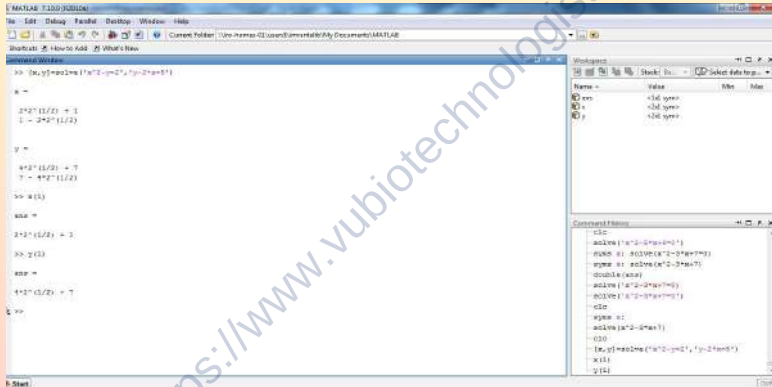


# Solving Algebraic Equations

- Note that, MATLAB reports the solution of the above system of equations by giving two  $x$  values and two  $y$  values.
- Thus the first solution consists of first value of  $x$  together with the first value of  $y$ .
- Similar case exists for the second solution.
- The first solution can also be extracted by typing  $x(1)$  and  $y(1)$  in the Command Window.
- See the Command Window 9



# Solving Algebraic Equations



**Figure:** The Matlab Desktop display with the solution of the system of algebraic equations



# Solving Algebraic Equations

- Sometimes an equation have more than one solutions and you may not get what you expected. For example the MATLAB syntax:
- » `solve('exp(-x) = sin(x)')`, gives
- the complex number answer. But we are interested to see its another numerical solution near to 0.5. For this use the following syntax:
- » `h = @(x)exp(-x) - sin(x);`  
» `fzero(h,0.5)`
- The required output will be
- `ans =`  
0.5885



# Solving Algebraic Equations

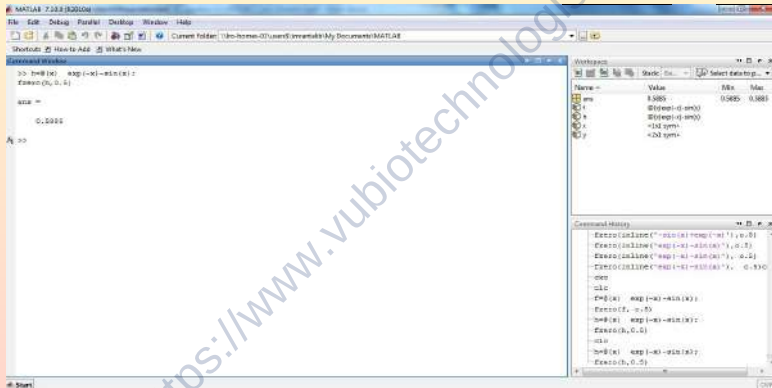


Figure: The Matlab Desktop display with fzero command



## Exercise

Solve the following equations using MATLAB **solve** command

- $x^2 + 2x - 4 = 0$ .
- $x^3 + 5x^2 + 4x + 3 = 0$

Solve the following equations for  $x$  using **solve** command

- $x + \log(y) = 3$ .
- $x + 10y = 6$ .

Find the solution of the following equation using **fzero** command near to  $x = 3$

- $\exp(-x) = \sin(x)$ .

Solve the following system of equations using **solve** command

- $x + y^2 = 2, y - 3x = 7$ .
- $x + y^2 = 2, 2y^2 - 3x = 7$ .

# Graphics Techniques in MATLAB

Department of Mathematics and Statistics  
Virtual University of Pakistan, Lahore, Pakistan.

*imrantalib@vu.edu.pk*



# Outline

## 1 Graphics Techniques in MATLAB

- ezplot Command
- ezplot command with symbolic expression
- ezplot command on anonymous function
- Modification of Graphs
- Example
- Plot command
- Graphs of Multiple Curves
- Parametric Plots
- Contour Plot
- Exercise

## ezplot Command

In this lecture, we learn the use of the basic plotting commands in MATLAB.

- **ezplot** command is used to plot the graph of a function of one variable.
- It expects a string, a symbolic expression or a function representing the function to be plotted.
- For example, the MATLAB command to draw the graph of the function  $x^3$  on the interval  $[-2, 2]$ , using the string form of **ezplot** is:
- » `ezplot('x^3',[-2,2])`.
- Note that the result will display in a new window labeled "Figure1". Look at the Figure 1



## ezplot Command

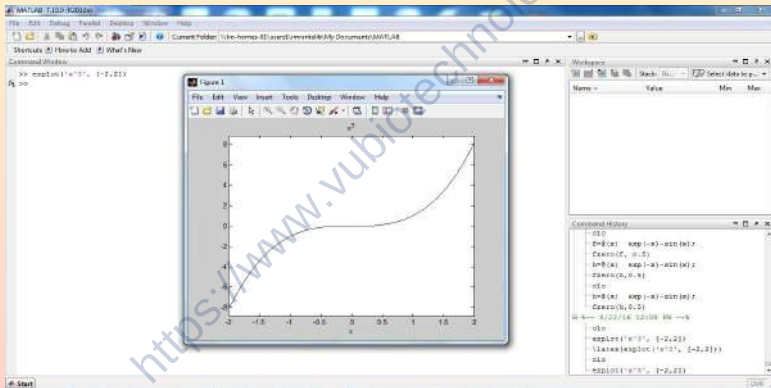
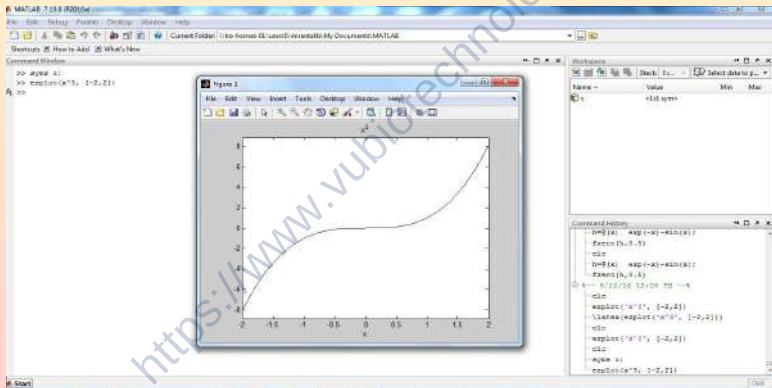


Figure: String form plot with ezplot command



- **ezplot** command can also accept a symbolic expression. For example
- The graph of  $x^3$  can also be produced using the following input:
  - » syms x;
  - » ezplot( $x^3$ , [-2,2])
- See the Figure 2

## ezplot command with symbolic expression



**Figure:** Symbolic expression form plot with ezplot command



- The graph of function  $x^3$  can also be plotted by using an anonymous function as the argument of **ezplot**. For this case the input will be:
- » `ezplot(@(x)x.^3, [-2,2])`.
- See the Figure 3

# ezplot command on anonymous function

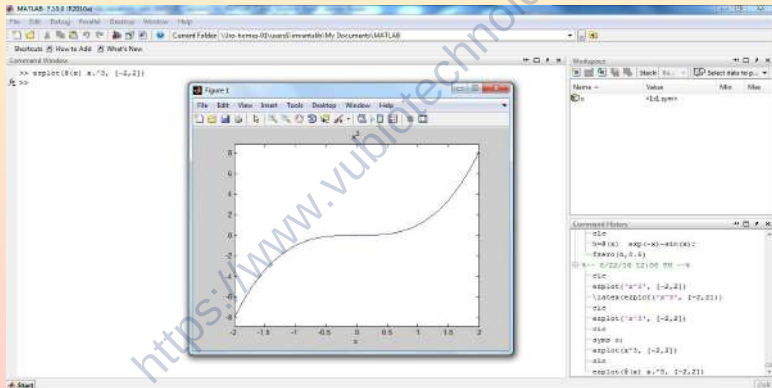


Figure: Anonymous function form plot with ezplot command



- Graphs can be modified in a number of ways. For example
- The title of the graph can be inserted.
- Different types of labels can be given to both axis.
- The region of the Graph can be seen in the form of grid.
- To accomplish this task, simply type the following commands in the Command Window
- ```
syms xy;  
y=ezplot(x3, [-2,2])  
xlabel('x');  
ylabel('y');  
title('The graph of  $y = x^3$ ');  
grid;
```

 see the output 3



Modification of Graphs

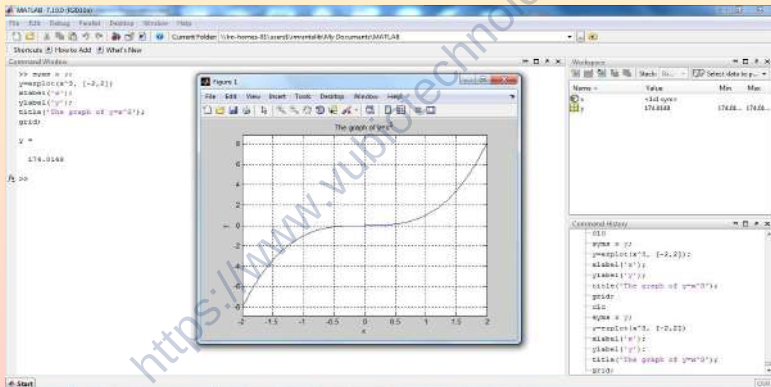


Figure: Modification of the Graph $y = x^3$

Example:

- Plot the graph of the function $y = x^2$ on $[-3, 3]$ using ezplot command considering x and y to be symbolic variables.
- Label the horizontal axis as "VU" and vertical axis as "IT".
- Keep the title of the graph as "Parabola"

Solution:

- The following syntax can be used to attain the required graph:
- » `syms x, y;`
`y=ezplot(x2, [-3,3])`
`xlabel('VU');`
`ylabel('IT');`
`title('Parabola');`
`grid;`
- Also see the Graph 5



ezplot Command with symbolic expression
 ezplot command on anonymous function
 Modification of Graphs
Example
 Plot command
 Graphs of Multiple Curves
 Parametric Plots
 Contour Plot
 Exercise

Graphical view of the Example

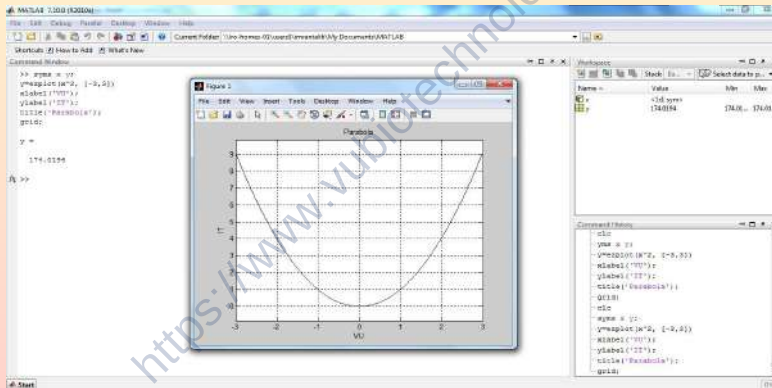


Figure: Modification of the Graph $y = x^3$



plot Command

- This command works on the vectors of numerical data.
- The syntax is **plot(X,Y)**.
- X and Y are the vectors of same length.
- This command considers the vectors X and Y to be lists of x and y coordinates of successive point on a graph, and joins the points with line
- Under **Plot** command the vectors $X = [3 \ 5 \ 7]$ and $Y = [9 \ 11 \ 13]$ are connected by line segments as (3,9) to (5,11) to (7,13).
- See their graphical view in Figure 7.



Graphical view of plot command

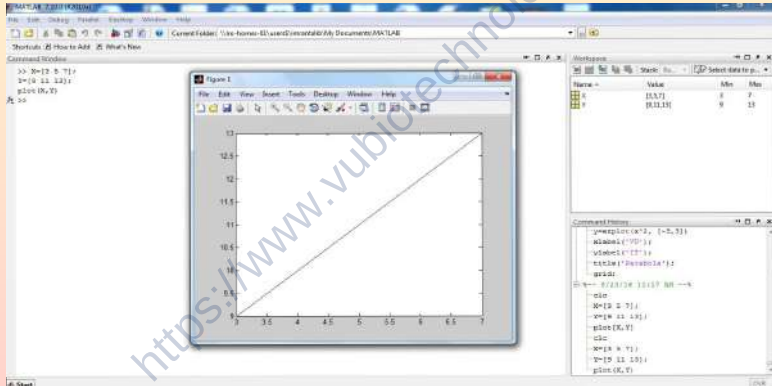


Figure: Plot command view of vectors of same length

Example:

Example:

- Using the plot command, plot the graph of $x^2 + x + 1$ on the interval $[-2, 2]$ by taking increment of 0.1 of the domain set.
- Label the horizontal axis with x and vertical axis with y .
- Finally, gives the title "Parabola" to the graph of $x^2 + x + 1$.

Solution:

- Firstly, we make a list X of x values.
- For this we need the enough values of x to get surety about the smoothness of the resulting graph. That is why we are going to subdivide the
- Thus a recipe for graphing the parabola is:



Example:

Solution:

- » `X=-2:0.1:2;`
`plot(X, X.^2 + X + 1)`
`xlabel('x');`
`ylabel('y');`
`title('Parabola');`
`grid;`
- The results appear in Figure 7



Graphical view of Example

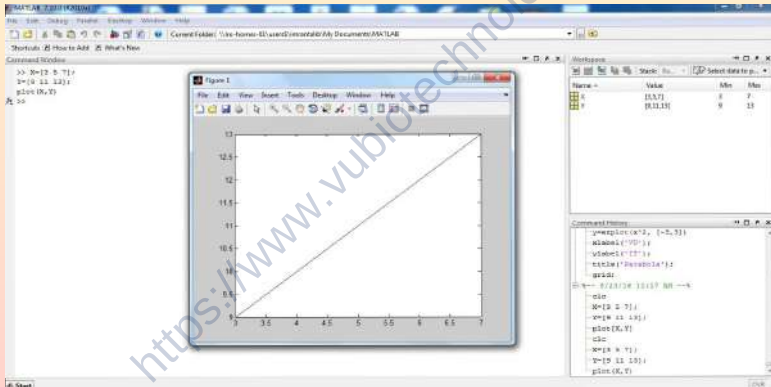


Figure: Plot command view of the Parabola

Graphs of Multiple Curves

- Each time we execute a plotting command, MATLAB erases the old plot and draws a new one.
- But if we want to see the two or more plots, then use command **hold on**.
- This command instructs MATLAB to retain the old graphics and draw any new graphics on top of the old.
- This command remains in effect until we type **hold off**.

Graphs of Multiple Curves

Example:

- Plot the graphs of e^{-x} and $\sin(x)$ on the interval $[0, 8]$ using the **ezplot**, **hold on**, and **hold off** commands.
- Label the horizontal axis with x and vertical axis with y .
- Finally, gives the title "Multiple Graphs".

Graphs of Multiple Curves

Solution:

- The recipe for graphing the multiple curves is:
- » syms x;
ezplot($\exp(-x)$),[0,8])
hold on
ezplot($\sin(x)$),[0,8])
hold off
title('Multiple Graphs')
xlabel('x');
ylabel('y');
grid;
- The results appear in Figure 8

Graphical view of Example

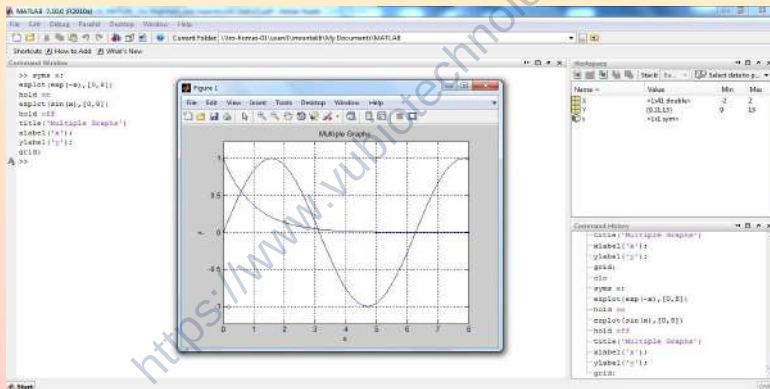


Figure: Ezplot command view of multiple curves

Graphs of Multiple Curves

Example:

- Plot the graphs of e^{-x} and $\sin(x)$ on the interval $[0, 8]$ using the **plot**, **hold on**, and **hold off** commands.
- Label the horizontal axis with x and vertical axis with y .
- Finally, gives the title "Multiple Graphs".

Graphs of Multiple Curves

Solution:

- The recipe for graphing the multiple curves is:
- » `X=0:0.1:8;`
`plot(X, exp(-X))`
`hold on;`
`plot(X, sin(X))`
`hold off;`
`title('Multiple Graphs')`
`xlabel('x');`
`ylabel('y');`
`grid;`



Graphical view of Example

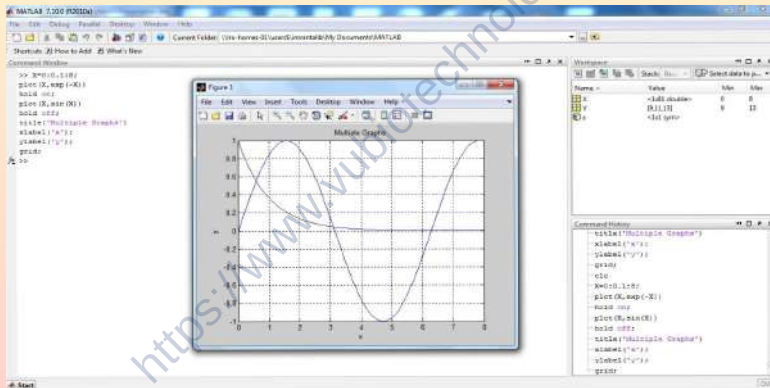


Figure: plot command view of multiple curves

Parametric Plots

- We observe the flexibility of the **plot** command by parametric plot of the circle centered at $(0,0)$, and radius 1.
- The parametric representation of the circle can be expressed with the aid of equations $x = \cos(2\pi t)$ and $y = \sin(2\pi t)$, t varies from 0 to
- The recipe for graphing the parametric plots is:
 - » `t=0:0.1:1;`
 - » `plot(cos(2*pi*t),sin(2*pi*t))`
 - » `xlabel('x');`
 - » `ylabel('y');`
 - » `title('Parametric plot of circle');`
 - » `grid;`
 - » `axis square;`

Parametric plot of the circle

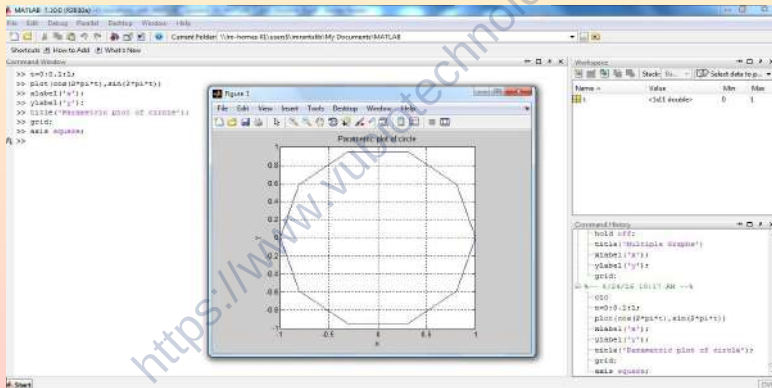


Figure: Parametric plot of the circle using plot command

Parametric Plots

Example:

- Plot the parametric graph of the circle with center at $(0,0)$, radius 1, and with increment of 0.01 in the domain set.
- Label the horizontal axis with x and vertical axis with y .
- Make the shape of the graph square.
- Keeps the title of the graph "Parametric plot of the circle".

Parametric Plots

Solution:

- The recipe for graphing the parametric plots is:
 - » `t=0:0.01:1;`
 - » `plot(cos(2*pi*t),sin(2*pi*t))`
 - » `xlabel('x');`
 - » `ylabel('y');`
 - » `title('Parametric plot of circle');`
 - » `grid;`
 - » `axis square;`
- The results are presented in Figure 10



Parametric plot of the circle

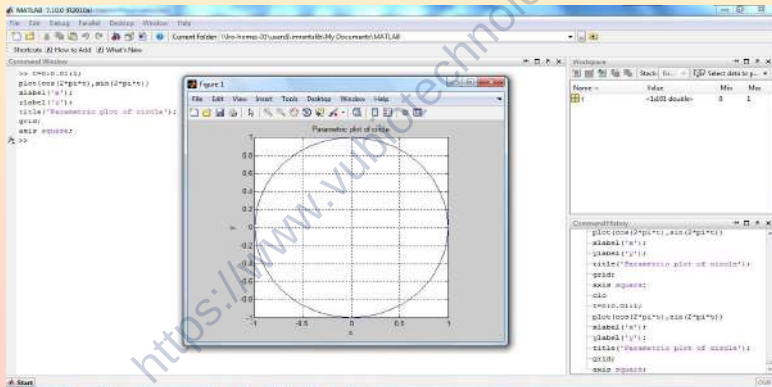


Figure: Parametric plot of the circle using plot command

Contour Plot

- A contour plot of an expression in two variables is a plot of the level curves of the expression.
- For example, the level curves of the expression $x^2 + y^2$ are the circles.
- **meshgrid** and **contour** are the two commands used for the contour plotting.
- **meshgrid** command is used to produce a grid of points in a specified rectangular region, with a specified spacing.
- **contour** command is used to produce a contour plots in the specified regions.

Contour Plots

Example:

- Plot the contour plots of the expression $x^2 + y^2$ using **meshgrid** and **contour** commands.
- Label the horizontal axis with x and vertical axis with y .
- Make the shape of the graph square.
- Keeps the title of the graph "Contour plotting of squares".

Contour Plots

Solution:

- The recipe for graphing the contour plots is:
- » `[X,Y] = meshgrid(-2:0.1:2,-2:0.1:2);`
» `contour(X, Y, X.^2 + Y.^2)`
» axis square
» `xlabel('x');`
» `ylabel('y');`
» `grid;`
» `title('Contour plotting of squares');`

Contour Plot

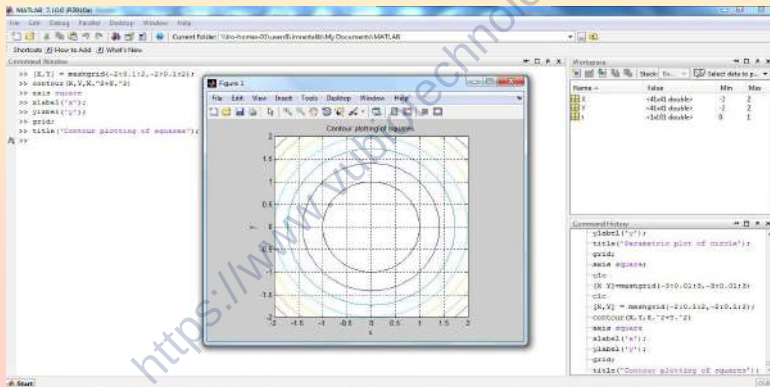


Figure: Contour plotting

Exercise

- Plot the graph of the function $x^3 + x^2 + 4$ on the interval $[-2, 2]$, using **ezplot** command.

Label the horizontal axis with x and vertical axis with y .

Make the shape of the graph square.

Keeps the title of the graph "The Graph of $x^3 + x^2 + 4$ ".

Keeps the layout of the graph as grid.

Exercise

- Plot the graph of the function $\sin(x) + \cos(x)$ on the interval $[0, 1]$, using **plot** command.

Label the horizontal axis with x and vertical axis with y .

Make the shape of the graph square.

Keeps the title of the graph "The Graph of $\sin(x) + \cos(x)$ ".

Keeps the layout of the graph as grid.



Exercise

- Plot the multiple curves of the functions $\sin(x) + \cos(x)$ and $e^{-x} + x^2$ on the interval $[0, 4]$, using **ezplot** command.

Label the horizontal axis with x and vertical axis with y .

Make the shape of the graph square.

Keeps the title of the graph "The Graph of Multiple curves using ezplot command".

Keeps the layout of the graph as grid.

Exercise

- Plot the multiple curves of the functions $\sin(x) + \cos(x)$ and $e^{-x} + x^2$ on the interval $[0, 4]$, using **plot** command and without the use of **hold on** and **hold off** commands.

Label the horizontal axis with x and vertical axis with y .

Make the shape of the graph square.

Keeps the title of the graph "The Graph of Multiple curves using plot command".

Keeps the layout of the graph as grid.



Exercise

- Plot the contour plots of the circles $x^2 + y^2$ of radius 1, 2, $\sqrt{2}$, $\sqrt{3}$.
Label the horizontal axis with x and vertical axis with y .
Make the shape of the graph square.
Keeps the title of the graph "Contour plots of circles".
Keeps the layout of the graph as grid.



Calculus With MATLAB

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk



Outline

- 1 Doing Calculus with MATLAB
 - Differentiation
 - Differentiation with respect to a particular variable
- 2 Higher Order Derivatives
 - Higher Order Derivatives of Univariate Expression
 - Higher Order Derivatives of Multi-variate Expression
- 3 Integration
 - Indefinite integral
 - Definite Integral
 - Improper Integral
- 4 Exercises



Outline

1 Doing Calculus with MATLAB

- Differentiation
- Differentiation with respect to a particular variable

2 Higher Order Derivatives

- Higher Order Derivatives of Univariate Expression
- Higher Order Derivatives of Multi-variate Expression

3 Integration

- Indefinite integral
- Definite Integral
- Improper Integral

4 Exercises



Outline

1 Doing Calculus with MATLAB

- Differentiation
- Differentiation with respect to a particular variable

2 Higher Order Derivatives

- Higher Order Derivatives of Univariate Expression
- Higher Order Derivatives of Multi-variate Expression

3 Integration

- Indefinite integral
- Definite Integral
- Improper Integral

4 Exercises



Outline

1 Doing Calculus with MATLAB

- Differentiation
- Differentiation with respect to a particular variable

2 Higher Order Derivatives

- Higher Order Derivatives of Univariate Expression
- Higher Order Derivatives of Multi-variate Expression

3 Integration

- Indefinite integral
- Definite Integral
- Improper Integral

4 Exercises

Differentiation

- To differentiate a symbolic expression, MATLAB uses **diff** command.
- The recipe for differentiation of a symbolic expression is:
 - » `syms x`
 - » `diff(sin(x2), x)`
 - `ans =`
 - `2 * x * cos(x2)`

Differentiation

Example:

- Differentiate the following functions using **diff** command.
- Moreover plot the multiple graphs of the given functions and their derivatives on the interval $[0, 5]$.
- $f(x) = x^2 + 2x + 10$.
- $g(x) = \sin(x)$.

Differentiation

Solution: The recipe of the solution for the function $f(x)$ is as under:

```

>> syms x
diff(x2 + 2 * x + 10, x)
ans =
2 * x + 2
ezplot(x2 + 2 * x + 10, [0,5]);
hold on
ezplot(ans, [0,5]);
hold off
title('Multiple plot of x2 + 2 * x + 10 and its derivative 2 * x + 2');
xlabel('x');
ylabel('y');
grid;

```

Differentiation

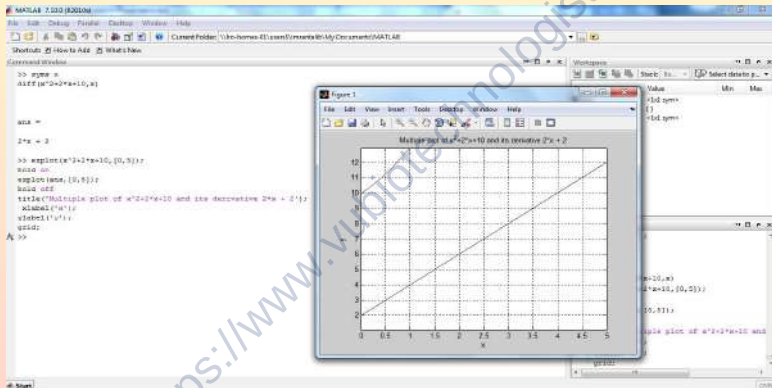


Figure: Multiple plot of the function $f(x)$ and its derivative.

Differentiation

Solution: The recipe of the solution for the function $g(x)$ is as under:

```

>> syms x
diff(sin(x), x)
ans =
2 * x + 2
ezplot(sin(x),[0,5]);
hold on
ezplot(ans,[0,5]);
hold off
title('Multiple plot of sin(x) and its derivative cos(x)');
xlabel('x');
ylabel('y');
grid;

```

Differentiation

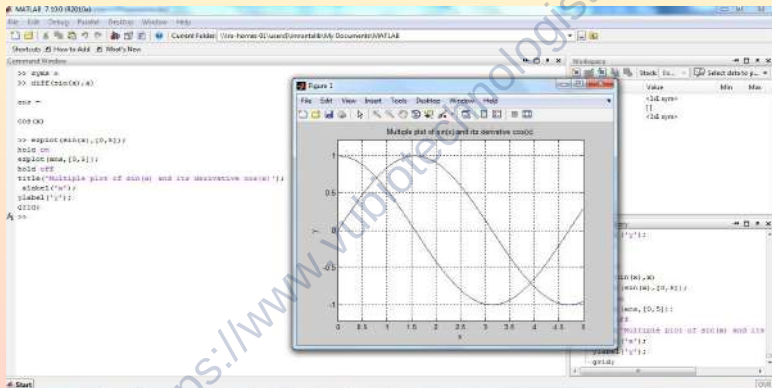


Figure: Multiple plot of the function $g(x)$ and its derivative.

Differentiation with respect to a particular variable

- Sometimes there are two or more variables are involved in a function.
- Then it is necessary to specify the differentiation variable.
- For example the MATLAB syntax to find the derivative of the function $f(x, t) = \sin(x * t^2)$ with respect to x and t is as follows

• `syms x t`

`diff(sin(x * t^2), x);`

`diff(sin(x * t^2), t);`

- The following outputs are generated:

• `» syms x t`

`» diff(sin(x * t^2), x)`

`ans =`

`t^2 * cos(t^2 * x)`

`» diff(sin(x * t^2), t)`

`ans =`

`2 * t * x * cos(t^2 * x)`

Differentiation with respect to a particular variable

Example:

- Find the derivative of the following function with respect to x , and t variables using **diff** command. Moreover plot the multiple graph of the function and their derivatives by defining X and Y using **meshgrid** command, Then define Z and create a surface plot:
- $f(x, t) = xe^{-x^2-t^2}$, $x, t \in [-2, 2]$.

Differentiation with respect to a particular variable

Solution:

- The recipe for the solution is as under:
- » `[X,T]=meshgrid(-2:0.2:2,-2:0.2:2);`
`Z1=X.*exp(-X.^2-T.^2);`
`surface(X,T,Z1)`
`hold on`
`view(3)`
`Z2=1./exp(T.^2+X.^2)-(2*X.^2)./exp(T.^2+X.^2);`
`surface(X,T,Z2)`
`hold off`
`view(3)`
`xlabel('x');`
`ylabel('y');`
`title('Surface plot')`

Differentiation with respect to a particular variable

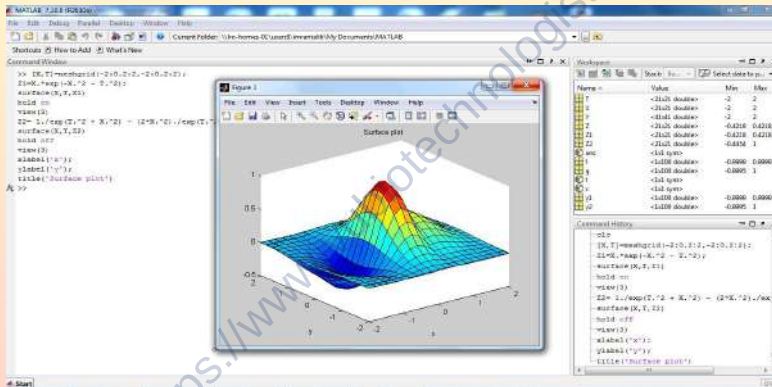


Figure: Surface plot of the function $f(x, t) = xe^{-x^2 - t^2}$, $x, t \in [-2, 2]$.

Higher Order Derivatives of Univariate Expression

- MATLAB also facilitate to find the higher order derivatives of univariate and multi-variate expressions.
- The recipe to find the higher order derivatives of univariate expression is as under:
- » `syms x`
 » `diff(x10 + 2 * (x2) + cos(x),order of derivative)`

Higher Order Derivatives of Univariate Expression

Example:

- Find the first-order, second-order, and fourth-order derivatives of the function $f(x) = x^{10} + 2 * (x^2) + \cos(x)$ using **diff** command.
- Plot the multiple graphs of the outputs using **ezplot** command.
- Label the horizontal axis with x and vertical axis with y .
- Keeps the title "Multiple plot of the function and its derivatives"

Higher Order Derivatives of Univariate Expression

Solution:

- The recipe for the solution is as under:

- $\gg \text{diff}(x^{10} + 2 * (x^2) + \cos(x), 1)$

ans =

$$4 * x - \sin(x) + 10 * x^9$$

- $\gg \text{diff}(x^{10} + 2 * (x^2) + \cos(x), 2)$

ans =

$$90 * x^8 - \cos(x) + 4$$

- $\gg \text{diff}(x^{10} + 2 * (x^2) + \cos(x), 4)$

ans =

$$\cos(x) + 5040 * x^6$$

Higher Order Derivatives of Univariate Expression

Solution:

- The recipe for graphical view is as under:
- » `ezplot(x10 + 2 * (x2) + cos(x),[-pi,pi])`
hold on
`ezplot(4 * x - sin(x) + 10 * x9,[-2*pi,2*pi])`
hold on
`ezplot(90 * x8 - cos(x) + 4,[-3*pi,3*pi])`
hold on
`ezplot(cos(x) + 5040 * x6,[-2*pi,2*pi])`
hold off
grid
`xlabel('x')`
`ylabel('y')`
`title('Multiple plot of the function and its derivatives')`

Higher Order Derivatives of Univariate Expression

The screenshot shows the MATLAB 7.0.0 R2010a interface. The Command Window contains the following code:

```
>> diff(x^10+2*(x^2)+cos(x),1)
ans =
10*x^9 - sin(x) + 3.0e+00
>> diff(x^10+2*(x^2)+cos(x),2)
ans =
90*x^8 - cos(x) + 4
>> diff(x^10+2*(x^2)+cos(x),3)
ans =
720*x^7 + sin(x)
>> diff(x^10+2*(x^2)+cos(x),4)
ans =
-5040*x^6 + cos(x) + 4
```

The Workspace window shows the following variables:

Name	Value	Min	Max
ans	-4.1e+00	-4.1e+00	4.1e+00
ans	4.1e+00	4.1e+00	4.1e+00
ans	4.1e+00	4.1e+00	4.1e+00
ans	4.1e+00	4.1e+00	4.1e+00

The Command History window shows the following commands:

```
hold on
plot(x^10+2*(x^2)+cos(x),-4,4,'-b',2*pi,0*pi)
hold on
plot(cos(x)+5040*x^6,-2*pi,2*pi)
hold off
grid
xlabel('x')
ylabel('y')
title('Multiple plot of the function y')
axis
diff(x^10+2*(x^2)+cos(x),1)
diff(x^10+2*(x^2)+cos(x),2)
diff(x^10+2*(x^2)+cos(x),3)
diff(x^10+2*(x^2)+cos(x),4)
```

Figure: Higher order derivatives of univariate expressions.

Higher Order Derivatives of Univariate Expression

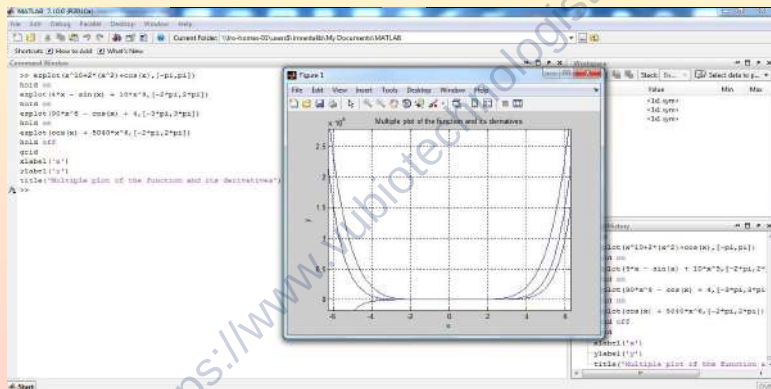


Figure: Multiple plot of the function and its derivatives.

Higher Order Derivatives of Multi-variate Expression

- MATLAB also facilitate to find the higher order derivatives of multi-variate expressions
- The recipe to find the higher order derivatives is as under:
- » `syms x y`
 » `diff(2 * x * y + (x * y)3, y, order of derivative)`

Higher Order Derivatives of Multi-variate Expression

Example:

- Find the first-order, second-order, and fourth-order derivatives of the function $f(x) = 2 * x * y + (x * y)^3$ with respect to variable y using **diff** command.

Higher Order Derivatives of Multi-variate Expression

Solution:

- The recipe for the solution is as under:

```
» syms x y
» diff(2 * x * y + (x * y)^3,y,1)
ans =
3 * x^3 * y^2 + 2 * x
» diff(2 * x * y + (x * y)^3,y,2)
ans =
6 * x^3 * y
» diff(2 * x * y + (x * y)^3,y,4)
ans =
0
```

Higher Order Derivatives of Multi-variate Expression

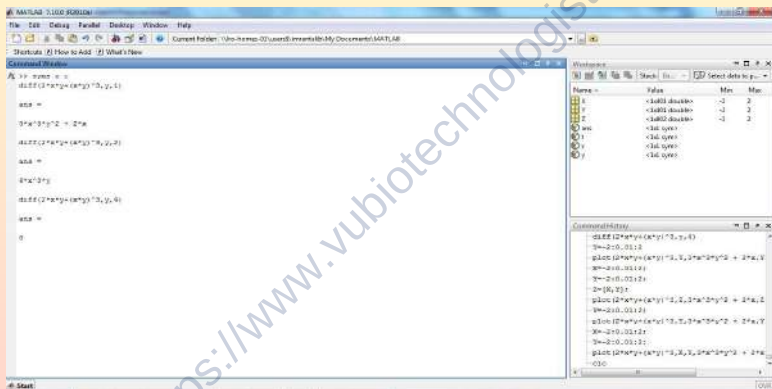


Figure: Multiple plot of the function and its derivatives.

Indefinite integral

- Matlab also used to evaluate the indefinite and definite integrals of the functions.
- **int** command is used for this purpose.
- For example, the integral of the function x^3 can be evaluated using the following syntax:
 - `» syms x;`
`» int( $x^3$ ,x)`
- The output is
- $x^4/4$.

Indefinite integral of Multi-variate Expression

- To find the indefinite integral of multi-variable expression, it is necessary to mention the integrable variable.
- For example, the integral of the function $x/(1 + y^2)$ with respect to variables x and y can be evaluated using the following MATLAB syntax:
- » `syms x y`
 » `int(x/(1 + y^2),x)`
`ans =`
 $x^2/(2 * (y^2 + 1))$
 » `int(x/(1 + y^2),y)`
`ans =`
 $x * \text{atan}(y)$

Indefinite Integral

Example:

- Find the integral of the function $f(x) = x^2 + 10x + 4$ using **int** command.
- Plot the multiple graphs of the function and its output using **plot** command.
- Label the horizontal axis with x and vertical axis with y .
- Keeps the title "Graph of function and its integral"



Indefinite Integral

Solution:

```

» syms x
int(x2 + 10 * x + 4)
ans =
(x * (x2 + 15 * x + 12))/3
» X=-2:0.01:2;
plot(X.2 + 10. * X + 4, X, (X. * (X.2 + 15. * X + 12))./3,X)
» grid;
» xlabel('x');
» ylabel('y');
» title('Graph of function and its integral')

```

Indefinite Integral

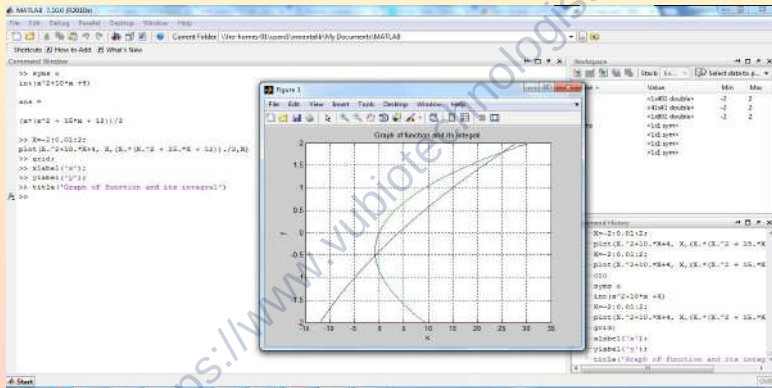


Figure: Multiple plot of the function and its integral.



Definite Integral of Univariate Expression

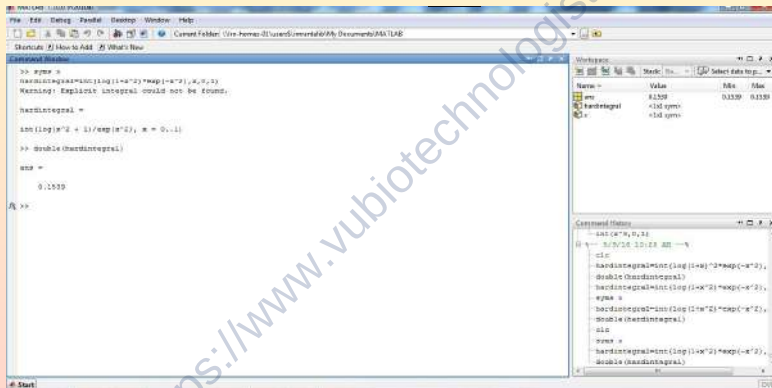
- Definite integral can also be evaluated using **int** command.
- For example, the MATLAB syntax to evaluate the definite integral of the function x^3 from 0 to 1 is as under:
 - » syms x
 - » int(x^3 ,0,1)
 - ans =
 - 1/4



Definite Integral of Univariate Expression

- Some integrals can not be evaluated symbolically in MATLAB using **int** command.
- Such type of integrals can be computed numerically using **double** command.
- For example the integral of the function $\log(1 + x^2) * e^{(-x^2)}$ can not be computed symbolically. it is computed using the following MATLAB
 - » syms x
 - » hardintegral=int(log(1 + x²) * e^(-x²),x,0,1)
 - » double(hardintegral).
- The out put is:
ans =
0.1539

Definite Integral



The image shows a MATLAB Command Window and Workspace. The Command Window contains the following code and output:

```
>> sym s
hardintegral=int(log(1+a^2)*exp(-a^2),a,0,1)
Warning: Explicit integral could not be found.

hardintegral =

int(log(a^2 + 1)/exp(a^2), a = 0..1)

>> double(hardintegral)

ans =

0.1239
```

The Workspace window shows the following variables:

Name	Value	Size	Class
ans	0.1239	0.1239	0.1239
hardintegral	<1d sym>	<1d sym>	<1d sym>

The Command History window shows the following commands:

```
>> s=sym('a');
>> hardintegral=int(log(1+a^2)*exp(-a^2),a,0,1);
>> double(hardintegral);
>> ans=double(hardintegral);
```

Figure: Evaluation of the definite integral with double command.



Improper Integral

- MATLAB also have ability to evaluate the improper integrals using **int** command.
- For example the integral of the function $\sin(x)/x$ from 0 to infinity can be computed using the following MATLAB syntax:
 - » syms x
 - » int($\sin(x)/x$, x, 0, inf)
- The output is:
ans =
pi/2



Exercises

Problem:

- Find the first-order, second-order, and fourth-order derivatives of the function $f(x) = x^{10} + 2 * (x^2) + \sin(x)$ using **diff** command.
- Plot the multiple graphs of the outputs using **ezplot** command on the interval $[-\pi, \pi]$.
- Label the horizontal axis with x and vertical axis with y .
- Keeps the title "Multiple plot of the function and its derivatives"



Exercises

Problem:

- Find the first-order, fourth-order, and fifth-order derivatives of the function $f(x) = \ln x + \sin(x)$ using **diff** command.
- Plot the multiple graphs of the outputs using **plot** command on the interval $[0, 1]$ and without using **hold on** and **hold off**
- Label the horizontal axis with x and vertical axis with y .
- Keeps the title "Multiple plot of the function and its derivatives"



Exercises

Problem:

- Find the first-order, fourth-order, and fifth-order derivatives of the function $f(x, y) = \ln x * y + \sin(x^2)$ using **diff** command.
- Plot the multiple graphs of the outputs using **plot** command on the interval $[0, 1]$ and without using **hold on** and **hold off** commands
- Label the horizontal axis with x and vertical axis with y .
- Keeps the title "Multiple plot of the function and its derivatives"



Use Of the M-Files

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk



Outline

1 M-Files

- Attributes of the M-Files
- Script M-files
- File saving procedure
- Execution or run of the Script file
- Display of the commands with results in Command Window
- Adding Comments
- Structuring Script M-File
- Function M-Files
- Loops

2 Exercise

Outline

1 M-Files

- Attributes of the M-Files
- Script M-files
- File saving procedure
- Execution or run of the Script file
- Display of the commands with results in Command Window
- Adding Comments
- Structuring Script M-File
- Function M-Files
- Loops

2 Exercise



- Up to now, we are working in **Command Window**. But for the complicated problems, **Command Window** is insufficient to produce the results.
- To handle the coding of the complicated problems, a much better approach is to create an **M-File**.
- MATLAB uses two different kinds of **M-Files**. One is script **M-File** and the other is function **M-File**.
- We learn the use of both types of **M-Files**.



- **M-Files** are ordinary text files containing **MATLAB** commands.
- We can create and modify the **MATLAB** commands using **MATLAB's** built in **M-File** editor, known as **Editor/Debugger**.
- This **Editor** automatically checks for certain **MATLAB** syntax requirements, and highlights different parts of the file in special colours.
- This **Editor** can be opened with the "**New M-File**" or "**Open**" icons on the tool bar.
- It can also be opened by typing **edit** at the command prompt.
- In the next slide, the **Editor** is opened writing **edit** in the command prompt.

Editor Window along with Command Window

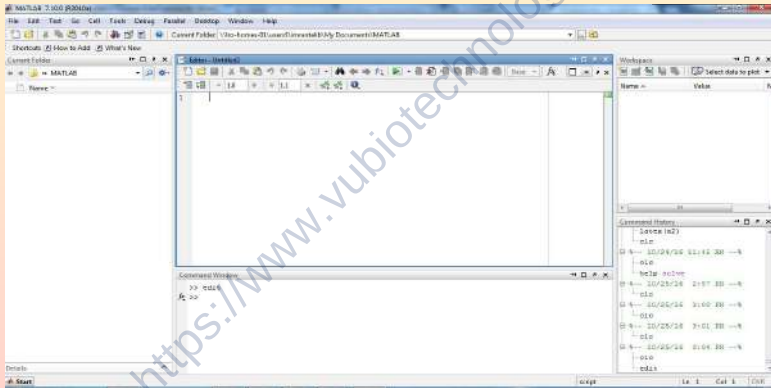


Figure: Display of the MATLAB desktop with Editor Window



- Now, we are going to learn how to construct a script M-File to solve different kind of problems.
- For this we use the **Editor/Debugger** to create the script M-File.
- For more clarity, consider an example

Example:

- Calculate $\frac{\sin(x)}{x}$ at $x = 0.1, 0.01, 0.001, 0.0001, 0.00001$ using **Editor/Debugger**. Moreover display the outputs up to 15 digits using **format long** command.



Solution:

- Use **Editor/Debugger** to create the file containing the following lines:
- $x = [0.1, 0.01, 0.001, 0.0001, 0.00001];$
 $y = \sin(x)./x.$
- To see the output it is necessary to save and execute the above code.
- The execution and saving procedure is explained in the next slide.



- Now save the file with the name **scriptfile1.m** in your working directory, or in some directory on your path, using the **Save AS**, which is a menu item in the **File** menu of the **editor**.
- Note that you can also save the file with the name what ever you like, but the **".m"** suffix is mandatory to write.
- Note that **editir** add this suffix automatically.



- The Script file can be executed or run by writing **scriptfile1** in the **Command Window**.
- Note that while the execution of the Script file, there is no need to write **scriptfile1.m** in the Command Window. Just write only **scriptfile1**.
- The required out put will be displayed in the **Command Window**.
- Remember that the **Command Window** does not give us liberty of editing.
- But **Editor Window** has advantage of editing.
- From now to onward, we will write our all codes in **Editor Window** instead of **Command Window**.



- Note that the out put will be displayed in the command window not in the **Editor/Debugger** window.
- The desire out put is as follows:
- $y =$
0.9983 1.0000 1.0000 1.0000 1.0000
- Above discussion is summarized in the next slide.

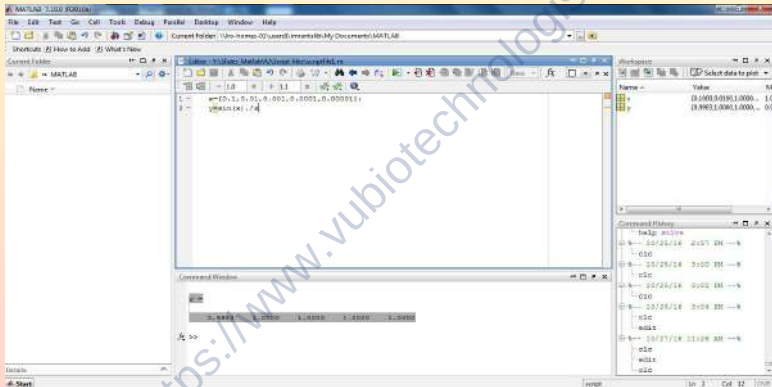


Figure: Computational values of $\frac{\sin(x)}{x}$

Echoing Commands

- Note that in the previous discussion we observe that the commands in the **Script M-File** were not automatically be displayed in the **Command Window**. Only results are to be displayed.
- But **echo Command** displays the Commands along with the results.
- echo** command works as;
- echo on
format long
 $x = [0.1, 0.01, 0.001, 0.0001, 0.00001];$
 $y = \sin(x)./x.$
echo off
- For more clarity see the next Figure.

- Attributes of the M-Files
- Script M-files
- File saving procedure
- Execution or run of the Script file
- Display of the commands with results in Command Window**
- Adding Comments
- Structuring Script M-File
- Function M-Files
- Loops

Figure: Display of the Commands along with the results in the Command Window using echo command.



- It is very much important to include comments in the **Script M-File** to explain what is being done in the calculation.
- Any line in a **Script M-File** that begins with a **percent sign** is treated as a comments and is not executed by MATLAB.
- For more clarity see the solution of the above example with comments.



Solution with adding comments

- echo on
- format long
- $x = [0.1, 0.01, 0.001, 0.0001, 0.00001];$
- $y = \sin(x) ./ x$
echo off
- For more clarity see the next slide

M-Files
Exercise

Attributes of the M-Files
 Script M-files
 File saving procedure
 Execution or run of the Script file
 Display of the commands with results in Command Window
Adding Comments
 Structuring Script M-File
 Function M-Files
 Loops

The screenshot shows the MATLAB environment. The script file contains the following code:

```

1 - echo on
2 - % Solution of the scriptFile
3 - % Format long & turns on 15 digits display
4 - s=[0.1,0.01,0.001,0.0001,0.00001]; % choose domain values
5 - y=sin(x)./x & display the output
6 - echo off
  
```

The Command Window shows the execution results:

```

>> scriptFile
% Solution of the scriptFile:
Format long & turns on 15 digits display
s=[0.1,0.01,0.001,0.0001,0.00001]; % choose domain values
y=sin(x)./x & display the output

y =

    Columns 1 through 4
    0.999999999999999    0.999999999999999    0.999999999999999    0.999999999999999

    Column 5
    0.999999999999999
  
```

Figure: Computational of the values of $y = \sin(x)/x$ at given domain values with adding comments.



- Make sure that your current MATLAB session is not affecting by other variables that had been defined in the last session. That is make sure that your present **Script M-File** is self contained, unaffected by other variables, and uncorrupted by left over graphics.
- For this ensuring, **clear all** command should be used. This command ensure that previous definitions of variables do not affect the results.
- Then in general your **Script M-File** should usually start with the **clear all** command.
- You should also either include the **close all** command at the beginning of an **M-File** that creates graphics.
- This command will automatically will close all **graphics Windows** and start with a clean slate.
- For more clarity observe the new version of the above example with **clear all** and **close all** commands in the next slide.



The screenshot displays the MATLAB R2010b environment. The main window shows the script file `scriptfile1.m` with the following content:

```

1 % Remove all variables definitions
2 clear all
3 % Remove all graphics windows
4 close all
5 % Display the command lines in the Command Window
6 echo on
7 % Solution of the scriptfile
8 format long % Turn on 15 digits display
9 a=[0.1,0.01,0.001,0.0001,0.00001]; % domain domain values
10 y=a.*a % display the output
11 echo off

```

The Command Window shows the execution of the script:

```

>> scriptfile1
% Solution of the scriptfile
format long % Turn on 15 digits display
a=[0.1,0.01,0.001,0.0001,0.00001]; % domain domain values
y=a.*a % display the output

? =

Columns 1 through 5
    0.000000000000002    0.000000000000000    0.000000000000000    0.000000000000000    0.000000000000000

Column 5
    0.000000000000000

```

The Workspace window shows the variables `a` and `y` with their values. The Command History window shows the commands executed.

Figure: Structuring of the solution file with clear all and close all command.



- Sometimes, it is necessary to repeat a process several times for different input values of a parameter.
- For instance, we can provide different inputs to a built-in function in order to find an output that meets a given criterion.
- We have already defined our own functions using **@** syntax or the **inline command**.
- In many situations, however, it is more convenient to define a function using an **M-File** instead of an anonymous or inline function.
- **M-File** allows us to extend the functionality of MATLAB by defining our own functions.

Solution of the above example using **sinepower** function

- The solution of the previous example using **M-File** is as under:
- function `y = sinepower(c)`
format long
`b=1:c;`
$$y = (\sin(10.^{(-b)})./10.^{(-b)})';$$
- Now save this **M-File** with **sinepower** name.
- Then execute this file by typing `sinepower(c)` in the **Command Window**. But, make sure that the numerical value has been assigned to **c**. Otherwise, it gives execution error.
- For more clarity see the next slide.



M-Files Exercise

Attributes of the M-Files
Script M-files
File saving procedure
Execution or run of the Script file
Display of the commands with results in Command Window
Adding Comments
Structuring Script M-File
Functioning M-Files
Loops

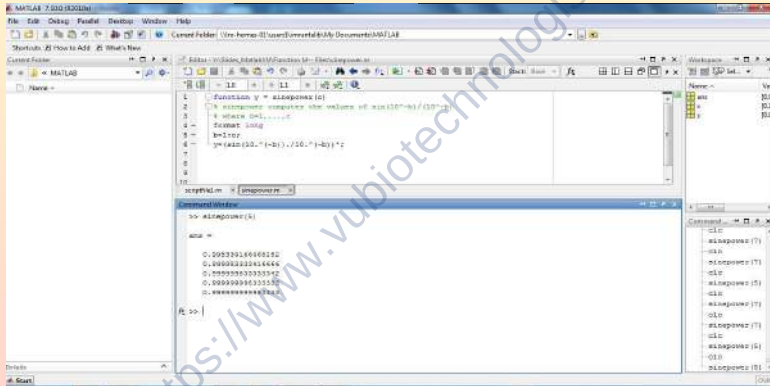


Figure: Solution of the previous example using Function M-File.



- A loop specifies that a command or group of commands should be repeated several times.
- The easiest way to create a loop is to use a **for** statement.
- For more clarity see the upcoming example:

For Loop

Example:

- Compute the values of $\sin(x)$ at $x = 0, 1$, and 2 using **for** loop.

Solution:

- The recipe of the solution is as under
- format long
for i= 0:2
y = $\sin(i)$
end
y
- For more clarity see the next slide.



M-Files Exercise

Attributes of the M-Files
Script M-files
File saving procedure
Execution or run of the Script file
Display of the commands with results in Command Window
Adding Comments
Structuring Script M-File
Function M-Files
Loops

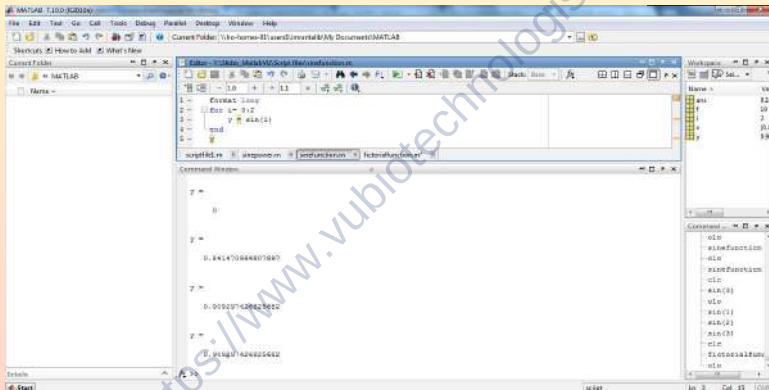


Figure: Computation of the values of sine function using for loop.

Discussion

- The loop begins with the **for** statement and ends with end **statement**.
- The command between those statements is executed a total of three times, once for each value of i from 0 to 2.
- The **Editor** automatically colors the commands **for** and **end** in blue.



Solve the following algebraic equations using **Script M-File**.

- $x^2 + 2x - 4 = 0$.
- $x^3 + 5x^2 + 4x + 3 = 0$

Solve the following equations for x using **Script M-File**.

- $x + \log(y) = 3$.
- $x + 10y = 6$.

Solve the following system of equations using **Script M-File**.

- $x + y^2 = 2, y - 3x = 7$.
- $x + y^2 = 2, 2y^2 - 3x = 7$.

Exercises

Problem:

- Find the first-order, second-order, and fourth-order derivatives of the function $f(x) = x^{10} + 2 * (x^2) + \sin(x)$ using **diff** command.
- Plot the multiple graphs of the outputs using **ezplot** command on the interval $[-\pi, \pi]$.
- Label the horizontal axis with x and vertical axis with y .
- Keeps the title "Multiple plot of the function and its derivatives"
- All the coding should be written using **Script M-File**.

Exercises

Problem:

- Find the first-order, fourth-order, and fifth-order derivatives of the function $f(x) = \ln x + \sin(x)$ using **diff** command.
- Plot the multiple graphs of the outputs using **plot** command on the interval $[0, 1]$ and without using **hold on** and **hold off** commands
- Label the horizontal axis with x and vertical axis with y .
- Keeps the title "Multiple plot of the function and its derivatives"
- All the coding should be written using **Script M-File**.

Exercises

Problem:

- Find the first-order, fourth-order, and fifth-order derivatives of the function $f(x, y) = \ln x * y + \sin(x^2)$ using **diff** command.
- Plot the multiple graphs of the outputs using **plot** command on the interval $[0, 1]$ and without using **hold on** and **hold off** commands
- Label the horizontal axis with x and vertical axis with y .
- Keeps the title "Multiple plot of the function and its derivatives"
- All the coding should be written using **Script M-File**.



Solving Ordinary Differential Equations (ODEs) in MATLAB

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk

Outline

- 1 Solving Ordinary Differential Equations (ODEs) in MATLAB
 - Explicit solution of the first order equations
 - Graphical Views of the First-order Equations
 - Graphical view of the family of solutions
- 2 Second-order Ordinary Differential Equations in MATLAB
 - Explicit Solution of Second-order ODEs
 - Examples
- 3 Solution Of System Of Differential Equations
 - Example
- 4 Exercise

Outline

- 1 Solving Ordinary Differential Equations (ODEs) in MATLAB
 - Explicit solution of the first order equations
 - Graphical Views of the First-order Equations
 - Graphical view of the family of solutions
- 2 Second-order Ordinary Differential Equations in MATLAB
 - Explicit Solution of Second-order ODEs
 - Examples
- 3 Solution Of System Of Differential Equations
 - Example
- 4 Exercise

Outline

- 1 Solving Ordinary Differential Equations (ODEs) in MATLAB
 - Explicit solution of the first order equations
 - Graphical Views of the First-order Equations
 - Graphical view of the family of solutions
- 2 Second-order Ordinary Differential Equations in MATLAB
 - Explicit Solution of Second-order ODEs
 - Examples
- 3 Solution Of System Of Differential Equations
 - Example
- 4 Exercise

Outline

- 1 Solving Ordinary Differential Equations (ODEs) in MATLAB
 - Explicit solution of the first order equations
 - Graphical Views of the First-order Equations
 - Graphical view of the family of solutions
- 2 Second-order Ordinary Differential Equations in MATLAB
 - Explicit Solution of Second-order ODEs
 - Examples
- 3 Solution Of System Of Differential Equations
 - Example
- 4 Exercise

First-order equations

- MATLAB is primarily a numerical package, it can certainly solve differential equations symbolically.
- For this purpose MATLAB uses **dsolve** command.
- For example, the input and output for solving $\frac{dy}{dx} = x * y$ in MATLAB is given below:
- » `dsolve('Dy = x * y','x')`
ans =
 $C2 * \exp(x^2/2)$
- Note that MATLAB uses capital letter **D** to indicate the derivative and requires that the entire equation appears in single quote.

First order equations

- Alternatively, if we are interested in using the $\frac{dy}{dx} = x * y$ a number of times, then define it as a variable, say, eqn1
- The input and output for this purpose is given below:
- » eqn1 = 'Dy = x * y'
eqn1 =
Dy = x * y
» y=dsolve(eqn1,'x')
y =
C2 * exp(x²/2)

First-order equations

- MATLAB also has ability to find the particular solution of the differential equations corresponding to certain initial and boundary conditions.
- For example, the particular solution of the first-order problem $\frac{dy}{dx} = x * y$ subject to initial condition $y(1) = 1$ in MATLAB can be found as
- » `eqn1 = 'Dy = x * y'`
`eqn1 =`
`Dy = x * y`
» `y=dsolve(eqn1,'y(1)=1','x')`
`y =`
`exp(x^2/2)/exp(1)^(1/2)`

First-order equations

- Suppose, we want to observe the behaviour of the solution of the differential equation $\frac{dy}{dx} = x * y$.
- For this purpose, the **ezplot** command can be used to see the graphical view of the solution subject to certain initial condition.
- The input and output command for seeing the graphical view is as under:
- ```
» eqn1 = 'Dy = x * y'
```

```
eqn1 =
```

```
Dy=x * y
```

```
» y=dsolve(eqn1,'y(1)=1','x')
```

```
y =
```

```
exp(x^2/2)/exp(1)^(1/2)
```

```
» ezplot(y,[0,2]).
```

# First-order equations

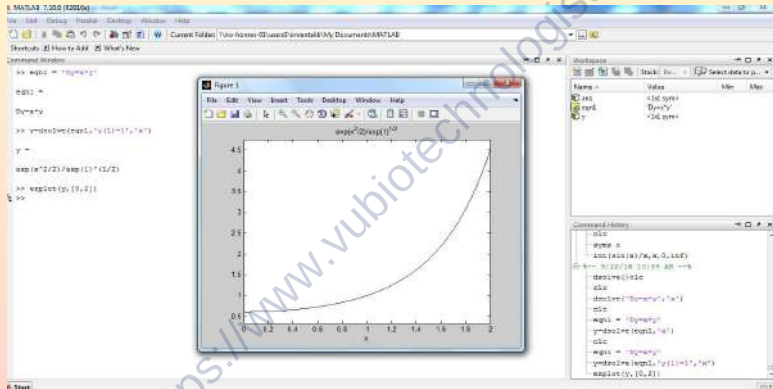


Figure: Graphical view of the solution of initial value problem on the interval  $[0,2]$ .

# First-order equations

## Example:

- Find the solution of the following initial value problem
- $\frac{dy}{dx} = x^2 + y, y(1) = 3.$
- Moreover, plot the graph of the solution using **ezplot** command on the interval  $[0,2]$ .

# First-order equations

## Solution:

- The recipe for the solution is as under:

- » `eqn1 = 'Dy = x^2 + y'`

`eqn1 =`

`Dy = x^2 + y`

- » `y=dsolve(eqn1,'y(1)=3','x')`

`y =`

`(8 * exp(x))/exp(1) - 2 * x - x^2 - 2`

- » `ezplot(y,[0,3])`

- » `xlabel('x')`

- » `ylabel('y')`

- » `grid`

- » `title('Graphical view of the initial')`

# First order equations

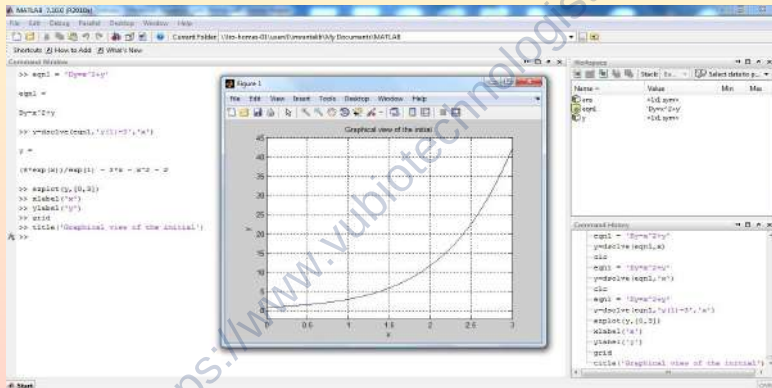


Figure: Graphical view of the solution of initial value problem on the interval  $[0, 3]$ .

## First-order equations

- Sometimes, we often want to see the family of solutions of the differential equations obtained by varying the initial conditions.
- Here is a natural way to do this in MATLAB.
- Begin by solving the given differential equation with a generic initial value.
- For example,
  - »  $\text{eqn1} = 'Dy = x^2 + y'$
  - $\text{eqn1} =$
  - $Dy = x^2 + y$
  - »  $y = \text{dsolve}(\text{eqn1}, 'y(0)=c', 'x')$
  - $y =$
  - $\exp(x) * (c + 2) - x^2 - 2 * x - 2$

## First-order equations

- The solution formula  $\exp(x) * (c + 2) - x^2 - 2 * x - 2$  is in terms of the initial value  $c$ .
- Now let we want to plot the solution curves with initial values  $y(0) = -2, -1, 0, 2, -2$ , on the interval  $[0, 2]$ .
- one approach is to execute the command using for loop as explained in the next slide:

## First-order equations

```
» eqn1 = 'Dy = x^2 + y'
eqn1 =
Dy = x^2 + y
» y=dsolve(eqn1,'y(0)=c','x')
y =
exp(x) * (c + 2) - x^2 - 2 * x - 2
» figure, hold on;
» syms x;
» for cval = -2:2
ezplot(subs(y,'c',cval),[0,2])
end
» axis tight
» xlabel('x')
» ylabel('y')
» title('Plot of family curves of $Dy = x^2 + y$ at different initial conditions.
')
```



# First order equations

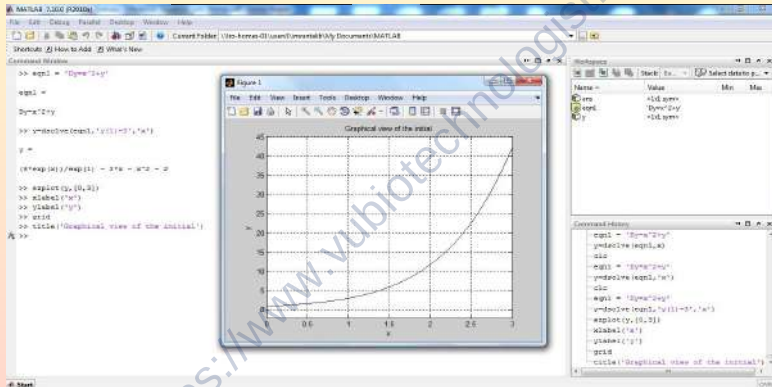


Figure: Plot of the family of curves of  $Dy = x^2 + y$  at different initial conditions on the interval  $[0,2]$  .

# First order equations

## Example:

- Plot the family of curves of the following differential equation on the interval  $[0,3]$ , with initial values  $y(0) = -3 : 3$ ,
- $\frac{dy}{dx} = x * y + 2 * x^2$ .

# First-order equations

## Solution:

- The recipe for the solution is as under:
- » `eqn1 = 'Dy = x * y + 2 * x^2'` `eqn1 =`  
 $Dy = x * y + 2 * x^2$
- » `y=dsolve(eqn1,'y(0)=c','x')`
- `y =`  

$$c * \exp(x^2/2) - \exp(x^2/2) * ((2 * x) / \exp(x^2/2) - (pi^{1/2} * x^3) / (2 * (x^2/2)^{3/2})) + (pi^{1/2} * x^3 * \operatorname{erfc}((x^2/2)^{1/2})) / (2 * (x^2/2)^{3/2}))$$
- » `figure; hold on`
- » `for cval = -3:3`
- `ezplot(subs(y,'c',cval),[0,3])`
- `end`
- » `axis tight`
- » `xlabel('x')`
- » `ylabel('y')`
- `title('Solutions of Dy = x * y + 2 * x^2 with initial conditions`  
`y(0)=-3,-2,...,3').`

# First-order equations

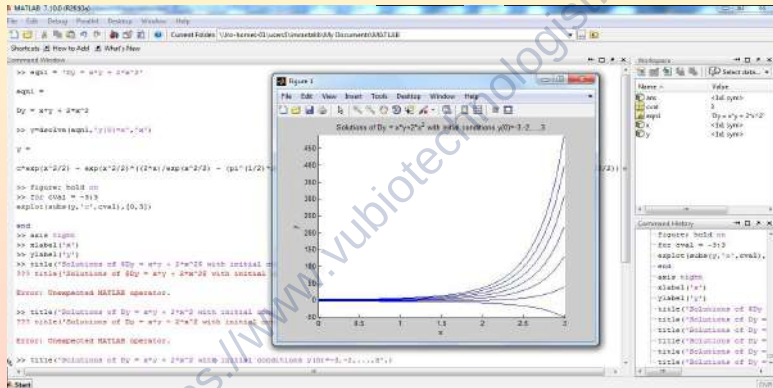


Figure: Plot of the family of curves of  $Dy = x^2 + y$  at different initial conditions on the interval  $[0, 2]$ .

## Second-order ODEs

- **dsolve** command can also be used to find the solution of certain second-order ODEs.
- For example, the MATLAB syntax for solving the  $y''(x) + 8y'(x) + 2y(x) = \cos(x)$  is as under:
- » `eqn='D2y+8*Dy+2*y=cos(x)';`  
  » `dsolve(eqn,'x')`



## Example

- Solve the following second-order differential equation
- $y''(x) + y'(x) - 6y = 0$
- subject to the following initial conditions
- $y(0)=1, y'(0)=0$ .
- Moreover plot the solution curve on the interval  $[0, 2]$ .

## Solution

- The recipe for the solution is as under:
- »  $eqn = 'D^2y + Dy - 6 * y = 0';$   
initialcondns= 'y(0)=1,Dy(0)=0'; dsolve(eqn,initialcondns,'x')  
ans =  
 $(3 * \exp(2 * x))/5 + 2/(5 * \exp(3 * x))$   
»  $ezplot((3*\exp(2*x))/5 + 2/(5*\exp(3*x)),[0,2])$   
»  $xlabel('x');$   
»  $ylabel('y');$   
»  $grid$   
»  $title('The graph of (3 * \exp(2 * x))/5 + 2/(5 * \exp(3 * x))')$  .

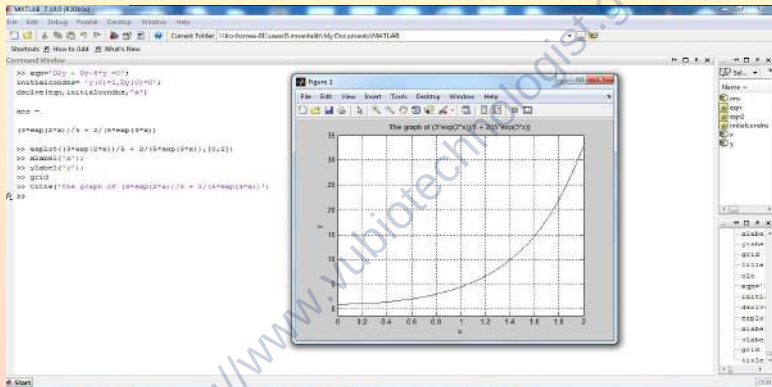


Figure: Graphical view of the solution curve of  $y''(x) + y'(x) - 6y = 0$ .





## Example

- Solve the following second-order differential equation
- $y''(x) + y'(x) - 6y = 0$
- subject to the following initial conditions
- $y(0)=1, y'(1)=0$ .
- Moreover plot the solution curve on the interval  $[0, 3]$ .

## Solution

- The recipe for the solution is as under:
- »  $eqn = 'D^2y + Dy - 6 * y = 0';$   
initialcondns= 'y(0)=1,Dy(1)=0'; dsolve(eqn,initialcondns,'x')  
y =  
 $(2 * \exp(2))/( \exp(3 * x) * (2 * \exp(2) + 3/\exp(3))) + (3 * \exp(2 * x))/( \exp(3) * (2 * \exp(2) + 3/\exp(3)))$   
» ezplot(y,[0,2])  
» xlabel('x');  
» ylabel('y');  
» grid  
» title('The graph of the solution curve y')

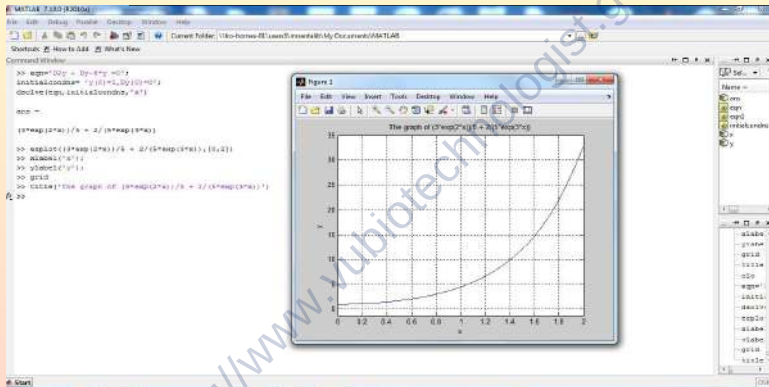


Figure: Graphical view of the solution curve of  $y''(x) + y'(x) - 6y = 0$ .



- **dsolve** command can also be used to find the solution of system of differential equations.
- For example, we want to find the solution of the following system of differential equations
- $x'(t) = x(t) + 2y(t) - z(t),$
- $y'(t) = x(t) + z(t),$
- $z'(t) = 4x(t) - 4y(t) + 5z(t).$
- The MATLAB syntax with **dsolve** command is mentioned in the next slide:



- »  $eqn1 = ' Dx = x + 2 * y - z';$
- »  $eqn2 = ' Dy = x + z';$
- »  $eqn3 = ' Dz = 4 * x - 4 * y + 5 * z';$
- »  $[x,y,z]=dsolve(eqn1,eqn2,eqn3).$

- The desire output is as under:

- $x =$   
 $-(C16 * \exp(t))/2 - (C17 * \exp(2 * t))/2 - (C18 * \exp(3 * t))/4,$   
 $y =$   
 $(C16 * \exp(t))/2 + (C17 * \exp(2 * t))/4 + (C18 * \exp(3 * t))/4 ,$   
 $z =$   
 $C16 * \exp(t) + C17 * \exp(2 * t) + C18 * \exp(3 * t) .$



- Find the solution of the following system of differential equations
- $x'(t) = x(t) + 2y(t) - z(t),$   
 $y'(t) = x(t) + z(t),$   
 $z'(t) = 4x(t) - 4y(t) + 5z(t).$  corresponding to the following conditions  
 $x(0)=1,$   
 $y(0)=2,$   
 $z(0)=3.$
- Moreover plot the solution curves using the plot command.



## Solution:

- The recipe for the solution is as under:
- »  $eqn1 = ' Dx = x + 2 * y - z';$   
 $eqn2 = ' Dy = x + z';$   
 $eqn3 = ' Dz = 4 * x - 4 * y + 5 * z';$   
 $initialcondns = 'x(0) = 1, y(0) = 2, z(0) = 3';$   
 $[x, y, z] = dsolve(eqn1, eqn2, eqn3, initialcondns)$   
 $x =$   
 $6 * exp(2 * t) - (5 * exp(3 * t))/2 - (5 * exp(t))/2$   
 $y =$   
 $(5 * exp(3 * t))/2 - 3 * exp(2 * t) + (5 * exp(t))/2$   
 $z =$   
 $10 * exp(3 * t) - 12 * exp(2 * t) + 5 * exp(t)$



## Solution:

- » `t=linespace(0,.5,25);`
- » `t=linspace(0,.5,25);`
- » `xx=eval(vectorize(x));`
- » `yy=eval(vectorize(y));`
- » `zz=eval(vectorize(z));`
- » `plot(t,xx,t,yy,t,zz)`
- » `xlabel('x');`
- » `ylabel('y');`
- » `grid;`
- » `title('Graph of the solution curves of system of differential equations')`



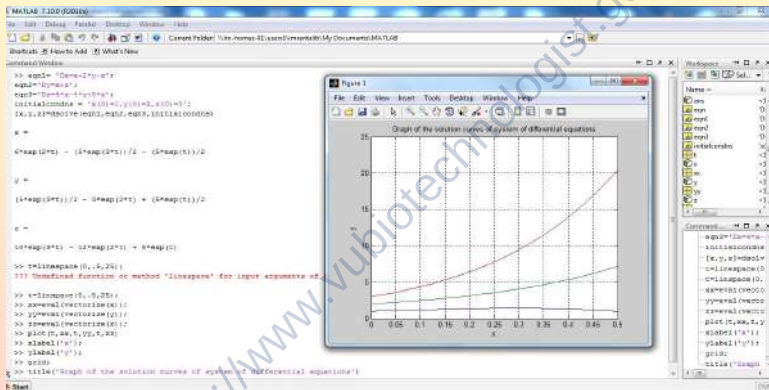


Figure: Graphical view of the solution curve of the system of differential equations.



- Find the solution of the following system of differential equations
- $x''(t) = x'(t) + 2y'(t) - z'(t),$   
 $y''(t) = x'(t) + z'(t),$   
 $z''(t) = 4x'(t) - 4y'(t) + 5z'(t).$   
corresponding to the following conditions  $x(0)=1, x'(0)=2$   
 $y(0)=1, y'(0)=4$   
 $z(0)=3, z'(0)=4.$
- Moreover plot the solution curves using the plot command.



## Solution:

- The recipe for the solution is as under:
- »  $eqn1 = 'D2x = Dx + 2 * Dy - Dz';$   
 $eqn2 = 'D2y = Dx + Dz';$   
 $eqn3 = 'D2z = 4 * Dx - 4 * Dy + 5 * Dz';$   
 »  $initialconditns = 'x(0) = 1, Dx(0) = 2, y(0) = 1, Dy(0) = 4, z(0) = 3, Dz(0) = 4';$   
 »  $[x, y, z] = dsolve(eqn1, eqn2, eqn3, initialconditns)$   
 $x =$   
 $7/3 - (exp(t) * (16 * exp(2 * t) - 72 * exp(t) + 72))/12$   
 $y =$   
 $(exp(t) * (32 * exp(2 * t) - 72 * exp(t) + 144))/24 - 10/3$   
 $z =$   
 $(exp(t) * (32 * exp(2 * t) - 72 * exp(t) + 72))/6 - 7/3$



## Solution:

```
» t=linespace(0,1,20);
» xx=eval(vectorize(x));
» yy=eval(vectorize(y));
» zz=eval(vectorize(z));
» plot(t,xx,t,yy,t,zz)
» xlabel('x');
» ylabel('y');
» grid;
» title('Graph of the solution curves of system of differential equations')
```

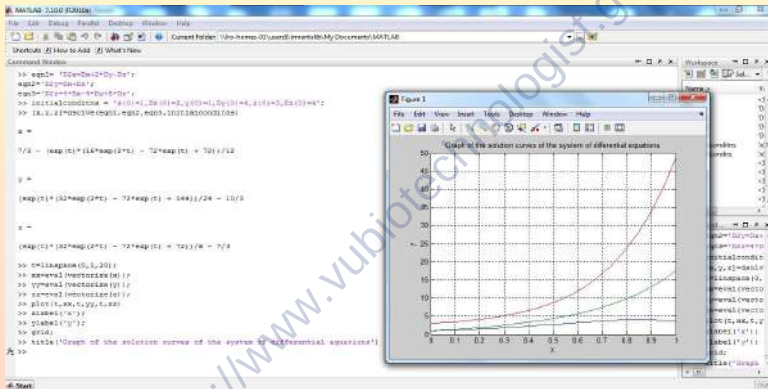


Figure: Graphical view of the solution curve of the system of differential equations.



- Find the general solution of the following differential equation

$$y'(x) = 2xy + 2x^3.$$

Also plot the family of solution curves on the interval  $[-4, 4]$  of the above differential equation obtained using the following initial conditions,  $y(0) = -3, -2, 0, 1, 2, 3$



- Find the general solution of the following boundary value problem  $y''(x) + 8y'(x) + 2y(x) = \cos(x)$ .  
Then plot the solution curve on the interval  $[0, 4]$  corresponding to following boundary conditions  $y(0)=4, y(1)=5$ .



- Find the solution of the following system of differential equations

- $x''(t) = 2x'(t) + 2y'(t) - z'(t),$

$$y''(t) = 3x'(t) - 3z'(t),$$

$$z''(t) = 5x'(t) - 4y'(t) + 5z'(t),$$

corresponding to the following conditions

$$x(0)=2, x'(0)=1$$

$$y(0)=3, y'(0)=4$$

$$z(0)=3, z'(0)=5.$$

- Moreover plot the solution curves using the plot command.





- Find the solution of the following system of differential equations
- $x'(t) = 2x(t) + 2y(t) - 3z(t),$   
 $y'(t) = 2x(t) + 3z(t),$   
 $z'(t) = x(t) - y(t) + z(t),$   
corresponding to the following conditions  
 $x(0)=2,$   
 $y(0)=1,$   
 $z(0)=4.$
- Moreover plot the solution curves using the plot command.



# MATLAB coding of some numerical methods

Department of Mathematics and Statistics  
Virtual University of Pakistan, Lahore, Pakistan.

[imrantalib@vu.edu.pk](mailto:imrantalib@vu.edu.pk)



# Outline

- 1 Euler's Method
  - Application of the Method
- 2 Improved Euler Method
  - Application of the Method
- 3 Runge-Kutta Method
  - Application of the Method
- 4 Higher Order Taylor's Method
  - Taylor's method of second order
  - Application of the Method
- 5 Exercises



# Outline

- 1 Euler's Method
  - Application of the Method
- 2 Improved Euler Method
  - Application of the Method
- 3 Runge-Kutta Method
  - Application of the Method
- 4 Higher Order Taylor's Method
  - Taylor's method of second order
  - Application of the Method
- 5 Exercises



# Outline

- 1 Euler's Method
  - Application of the Method
- 2 Improved Euler Method
  - Application of the Method
- 3 Runge–Kutta Method
  - Application of the Method
- 4 Higher Order Taylor's Method
  - Taylor's method of second order
  - Application of the Method
- 5 Exercises



# Outline

- 1 Euler's Method
  - Application of the Method
- 2 Improved Euler Method
  - Application of the Method
- 3 Runge–Kutta Method
  - Application of the Method
- 4 Higher Order Taylor's Method
  - Taylor's method of second order
  - Application of the Method
- 5 Exercises



# Outline

- 1 Euler's Method
  - Application of the Method
- 2 Improved Euler Method
  - Application of the Method
- 3 Runge–Kutta Method
  - Application of the Method
- 4 Higher Order Taylor's Method
  - Taylor's method of second order
  - Application of the Method
- 5 Exercises



- Consider the general first order differential equation

$$\frac{dy}{dx} = f(x, y); \quad y(x_0) = y_0. \quad (1.1)$$

- Suppose we want to find its numerical solution on the interval of  $x$  - values  $[x_0, x_n]$ .
- Our goal will be to approximate the value of the solution  $y(x)$  at each of the  $x$  values in a partition  $P = [x_0, x_1, \dots, x_n]$ .
- To achieve this goal, the following formula is used:

$$y_{n+1} \approx y_n + f(x_n, y_n) \Delta x. \quad (1.2)$$

- Where  $\Delta x$  represents the length of each subinterval of equal length.





- Find the numerical solution of the following differential equation on the interval  $[0, 1]$  using Euler's Formula for  $n = 10$

$$\frac{dy}{dx} = \sin(xy); \quad y(0) = \pi. \quad (1.3)$$



## Solution

- The recipe of the solution is as under:
- function [xvalues,yvalues]= euler(f,x0,y0,xn,n)

format long

n=10;

x0=0;

xn=1;

y0=pi;

x(1)=x0;

y(1)=y0;

xn=1;

dx=(xn-x0)/n;

for k=1:n

x(k+1)=x(k)+dx;

y(k+1)=y(k)+sin(x(k)\*y(k))\*dx;

end

xvalues=x';

yvalues=y'

plot(xvalues,yvalues)

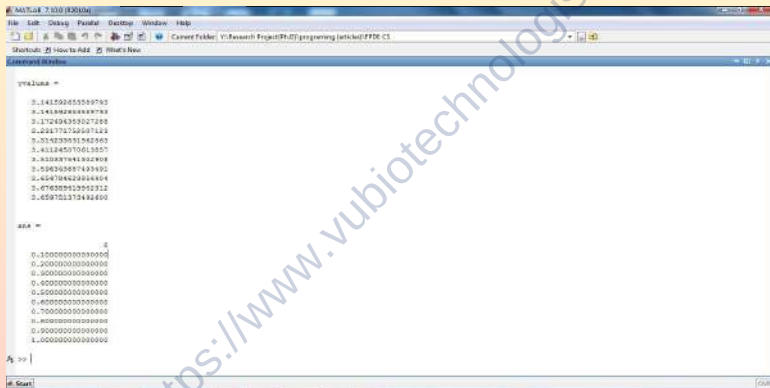


- The coding of Euler's formula in Editor (Function M-File)

```
1 function [xvalues,yvalues]=euler(5,X0,y0,Nb)
2 %Solves function M--File that solve the ODE
3 %Format long
4 %x0:
5 %y0:
6 %x0:
7 %y0:
8 %Number of iterations
9 %x1=x0;
10 %y1=y0;
11 %Left end point of the interval
12 %x0;
13 %Right end point of the interval
14 %h=(X0-X1)/N;
15 %Length of a equally spaced subintervals
16 for k=1:N
17 x(k)=x0+h*(k-1);
18 y(k)=y0+h*(k-1)*sin(x(k)*y(k)); %Euler's formula
19 end
20 xvalues=x;
21 yvalues=y;
22 plot(xvalues,yvalues)
```

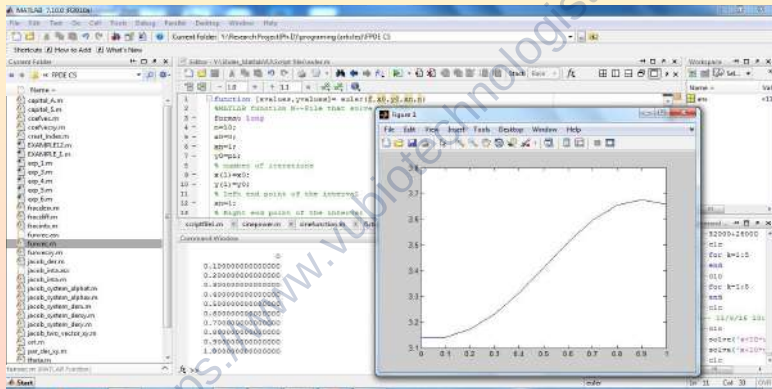
Figure: Coding of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$  using Euler's Formula

- Output of the above coding shown in **Command Window**.



**Figure:** Numerical solution (output) of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$  using Euler's Formula

- Graphical view of the results of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$ .



**Figure:** Graphical view along with the command window and function M-File of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$ .



- The general form of improved Euler's method is as under:

$$y_{k+1} = y_k + \frac{\Delta x}{2} (y'_k + f(x_{k+1}, y_k + \Delta x y'_k)), \quad i = 0, 1, 2, \dots, n-1. \quad (2.1)$$

- I will not go into the detail study of this method.
- The detail is mentioned in **MTH603**.



- Find the numerical solution of the following differential equation on the interval  $[0, 1]$  using improved Euler's Formula for  $n = 10$

$$\frac{dy}{dx} = \sin(xy); \quad y(0) = \pi. \quad (2.2)$$



## VuB Solution

- The recipe of the solution is as under:
- `function [xvalues,yvalues] = impeuler(f,x0,x1,xn,n)`  
format long  
`f = @(x,y)sin(x.*y);`  
`n=10;`  
`x0=0;`  
`y0=pi;`  
`xn=1;`  
`x(1)=x0;`  
`y(1)=y0;`  
`dx=(xn-x0)/n;`  
for `k=1:n` `x(k+1)=x(k)+dx;`  
`y(k+1)= y(k)+dx/2*(f(x(k),y(k))+f(x(k),y(k)+dx*y(k)));`  
end  
`xvalues=x'`  
`yvalues=y'`  
`plot(xvalues,yvalues)`





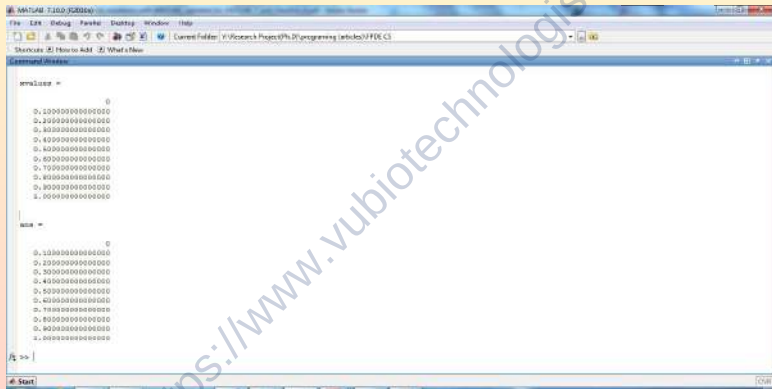
- The coding of Improved Euler's formula in Editor (Function M-File)

```
1 function [xvalues,yvalues] = improved_euler(f,x0,x1,h)
2
3 % Solve the IVP y' = f(x,y), y(x0) = y0, using the Improved Euler method.
4 % The solution is returned in the vector yvalues. The x values are in xvalues.
5 % The function f is a string or a function handle.
6 % The initial condition y0 is a scalar.
7 % The interval [x0,x1] is a vector of two scalars.
8 % The step size h is a scalar.
9
10 % Initialize the solution vector.
11 n = (x1-x0)/h + 1;
12 xvalues = zeros(1,n);
13 yvalues = zeros(n,1);
14
15 % Set the initial condition.
16 xvalues(1) = x0;
17 yvalues(1) = y0;
18
19 % Compute the solution using the Improved Euler method.
20 for k = 2:n
21 % Compute the slope at the current point.
22 k1 = f(xvalues(k-1),yvalues(k-1));
23
24 % Compute the slope at the midpoint.
25 xmid = xvalues(k-1) + h/2;
26 ymid = yvalues(k-1) + h*k1/2;
27 k2 = f(xmid,ymid);
28
29 % Compute the new value of y.
30 yvalues(k) = yvalues(k-1) + h*(k1+k2)/2;
31
32 % Compute the new value of x.
33 xvalues(k) = xvalues(k-1) + h;
34 end
35
36 % Plot the solution.
37 plot(xvalues,yvalues,'b');
38 hold on;
39
40 % Compute the exact solution.
41 % The exact solution is y = pi/sqrt(x).
42 % Compute the error.
43 error = abs(yvalues - pi/sqrt(xvalues));
44
45 % Display the error.
46 disp('Error:');
47 disp(error);
```

Figure: Coding of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$  using Improved Euler's Formula



- Output of the above coding shown in **Command Window**.



**Figure:** Numerical solution (output) of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$  using Improved Euler's Formula



- Graphical view of the results of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$ .

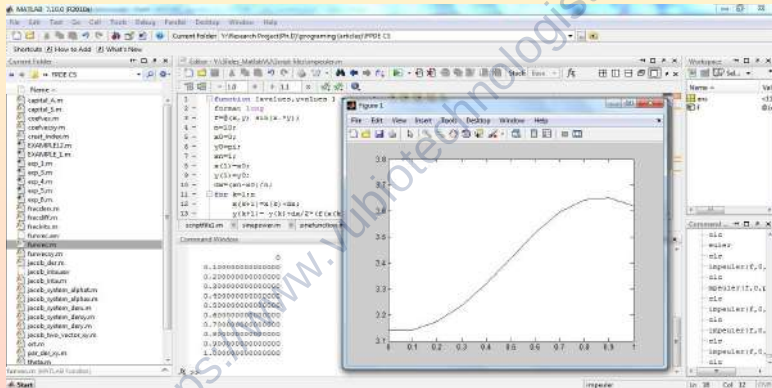


Figure: Graphical view along with the command window and function M-File of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$ .



- The recursive relation of fourth-order Runge-Kutta Method is as under:

$$y_{k+1} = y_k + \frac{h}{6} (K_1 + 2K_2 + 2K_3 + K_4),$$

where

$$\begin{aligned} K_1 &= f(x_k, y_k), \\ K_2 &= f\left(x_k + \frac{\Delta x}{2}, y_k + \frac{\Delta x}{2} K_1\right), \\ K_3 &= f\left(x_k + \frac{\Delta x}{2}, y_k + \frac{\Delta x}{2} K_2\right), \\ K_4 &= f(x_{k+1}, y_k + \Delta x K_3). \end{aligned} \tag{3.1}$$



### Example:

- Find the numerical solution of the following differential equation on the interval  $[0, 1]$  using Runge-Kutta method of fourth order for  $n = 10$

$$\frac{dy}{dx} = \sin(xy); \quad y(0) = \pi. \quad (3.2)$$



## Solution:

- The coding of the solution is as under:
- `function [xvalues, yvalues] = RungeKutta(x0,xn,f,n,y0)`  
format long  
 $f = @(x,y)\sin(x.*y);$   
`n=10;`  
`x0=0;`  
`xn=1;`  
`y0=pi;`  
`x(1)=x0;`  
`y(1)=y0;`  
`dx=(xn-x0)/n;`  
`for k=1:n`  
`x(k+1)=x(k)+dx;`  
`k1=f(x(k),y(k));`



## Solution Continuing

### Solution:

```
k2=f(x(k)+dx/2,y(k)+dx/2*(k1));
k3=f(x(k)+dx/2,y(k)+dx/2*(k2));
k4=f(x(k+1),y(k)+dx*(k3));
y(k+1)=y(k)+dx/6*(k1+2*k2+2*k3+k4);
end
xvalues =x';
yvalues=y'
plot(xvalues,yvalues)
title('Runge Kutta method od order 4');
xlabel('x');
ylabel('y');
```



- The coding of Runge-Kutta method in Editor (Function M-File)

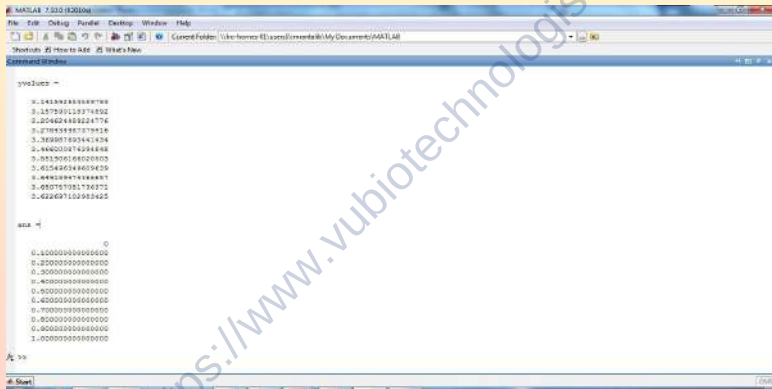
```
1 function [xvalues, yvalues] = RungeKutta(x0, x1, f, y0)
2
3 % Runge-Kutta
4 f=@(x,y) sin(x*y);
5
6 % Initial conditions
7 x0=0;
8 y0=pi;
9
10 % Step size
11 h=0.1;
12
13 % Number of steps
14 n=(x1-x0)/h;
15
16 % Initialize arrays
17 xvalues=[];
18 yvalues=[];
19
20 % Loop
21 for k=1:n
22 % Calculate k1
23 k1=f(x0,y0);
24 % Calculate k2
25 k2=f(x0+h/2,y0+k1*h/2);
26 % Calculate k3
27 k3=f(x0+h,y0+k1*h);
28 % Calculate k4
29 k4=f(x0+h,y0+k1*h);
30
31 % Update x and y
32 x0=x0+h;
33 y0=y0+h*(k1+k2+k3+k4)/4;
34
35 % Store values
36 xvalues=[xvalues; x0];
37 yvalues=[yvalues; y0];
38
39 % Plot
40 plot(xvalues, yvalues);
41
42 % Title
43 title('Runge-Kutta method solution');
44
45 % Axis labels
46 xlabel('x');
47 ylabel('y');
48
49 end
```

Figure: Coding of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$  using Runge-Kutta Formula





- Output of the above coding shown in **Command Window**.



**Figure:** Numerical solution (output) of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$  using Runge-Kutta Formula



- Graphical view of the results of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$ .

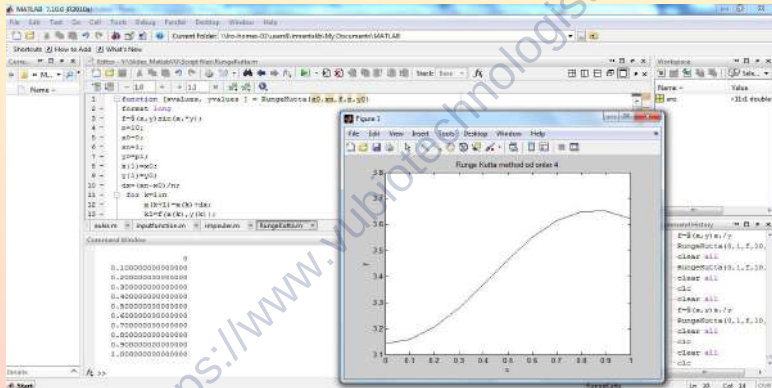


Figure: Graphical view along with the command window and function M-File of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$ .



- The basic idea of Euler's method can be improved in a straight forward manner by using higher order Taylor's polynomials.
- Remember that while deriving Euler's method, we approximate  $y(x)$  with a first order polynomial.
- More generally, if we use a Taylor polynomial of order  $n$ , we obtain the Taylor method of order  $n$ .
- For example, if we use a Taylor polynomial of order 2, we get a Taylor method of order 2.
- Similarly, the idea can be extended to higher order Taylor methods by increasing the order of Taylor polynomial.



- I will not go into the detail study of the second order Taylor's method.
- Simply, I will write down the iterative relation of the method, which is as under:

$$y_{k+1} \approx y_k + f(x_k, y_k) \Delta x + \left( \frac{\partial f}{\partial x}(x_k, y_k) + \frac{\partial f}{\partial y}(x_k, y_k) f(x_k, y_k) \right) \frac{(\Delta x)^2}{2} \quad (4.1)$$



- Find the numerical solution of the following differential equation on the interval  $[0, 1]$  using Taylor's method of the second order for  $n = 10$

$$\frac{dy}{dx} = \sin(xy); \quad y(0) = \pi. \quad (4.2)$$



## Solution:

- The MATLAB coding of the Method is given as under:
- `function [xvalues,yvalues] = Taylororder2(x0,xn,f,n,y0)`  
`format long`  
`n=10;`  
`f = @(x,y)sin(x.*y);`  
`x0=0;`  
`xn=1;`  
`y0=pi;`  
`x(1)=x0;`  
`y(1)=y0;`



## Solution:

```
• dx=(xn-x0)/(n);
 for k=1:n
 x(k+1)=x(k)+dx;
 y(k+1) = y(k) + f(x(k), y(k)) * dx + (y(k) * cos(x(k) * y(k)) + x(k) *
 cos(x(k) * y(k)) * f(x(k), y(k))) * (dx)^2/2;
 end
 xvalues=x';
 yvalues=y'
 plot(xvalues,yvalues)
 xlabel('x');
 ylabel('y'); title('Taylor's method of second Order.')
```



- The MATLAB coding of Taylor's method of second Order in Editor (Function M-File)

```

1 function [xvalues, yvalues] = TaylorOrder2(x0, x1, f, h, y0)
2
3 format long
4 n=(x1-x0)/h;
5 t=(x0, y0) sin(x.*y) /
6 m=h/2;
7 am=h;
8 y0=pi;
9 x(1)=x0;
10 y(1)=y0;
11 dx=(x1-x0)/n;
12 for k=1:n
13 g(k+1)=m(h)+dx;
14 y(k+1)=y(k)+F(x(k), y(k))*dx + 1/2*(F(x(k))*cos(x(k))*y(k)-m(k)*cos(x(k))*g(k))*F(x(k), y(k))*dx^2/2;
15
16 end
17 xvalues=x;
18 yvalues=y;
19 plot(xvalues, yvalues)
20 xlabel('x')
21 ylabel('y')
22 title('Taylor method of second Order.')
23
24
25
26
27

```

Figure: Coding of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$  using Taylor's method of second Order.





- Output of the above coding shown in **Command Window**.

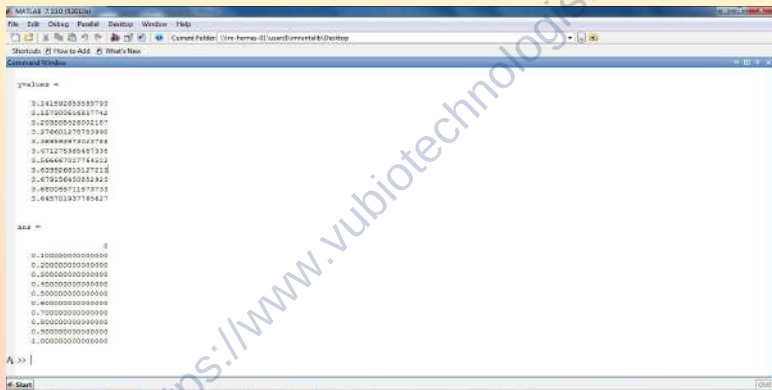


Figure: Numerical solution (output) of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$  using Taylor's method of second Order.



- Graphical view of the results of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$ .

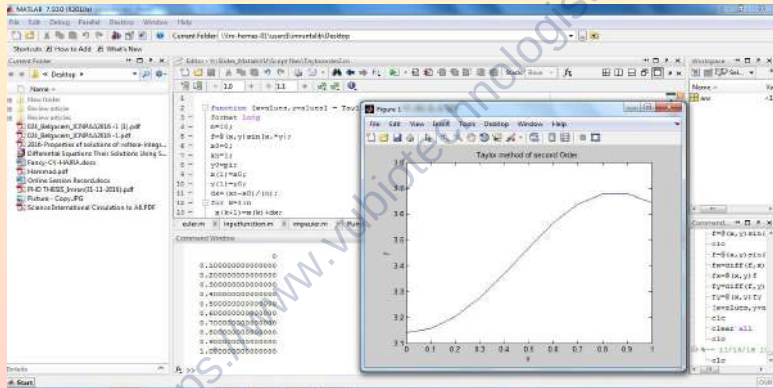


Figure: Graphical view along with the command window and function M-File of  $\frac{dy}{dx} = \sin(xy)$ ;  $y(0) = \pi$ .



- Find the numerical solution of the following differential equation on the interval  $[0, 1]$  using Euler's method for  $n = 20$

$$\frac{dy}{dx} = \frac{y - x}{y + x}; \quad y(0) = 0.5. \quad (5.1)$$



- Find the numerical solution of the following differential equation on the interval  $[0, 1]$  using Improved Euler's method for  $n = 20$

$$\frac{dy}{dx} = \frac{y - x}{y + x}; \quad y(0) = 0.5. \quad (5.2)$$



- Find the numerical solution of the following differential equation on the interval  $[0, 1]$  using Runge-Kutta method of fourth order for  $n = 20$

$$\frac{dy}{dx} = \frac{y - x}{y + x}; \quad y(0) = 0.5. \quad (5.3)$$



- Find the numerical solution of the following differential equation on the interval  $[0, 1]$  using Taylor's method of second order for  $n = 20$

$$\frac{dy}{dx} = \frac{y - x}{y + x}; \quad y(0) = 0.5. \quad (5.4)$$



# Numerical Solutions of ODEs in MATLAB

Department of Mathematics and Statistics  
Virtual University of Pakistan, Lahore, Pakistan.

*imrantalib@vu.edu.pk*



# Outline

## 1 First-Order initial value problems

- Examples
- Family of approximate solutions
- Extraction of the numerical values at some particular domain values
- Comparison between exact and approximate solutions
- Limitations of the ode45 command
- ode45 command when initial condition point lies between the interval





# First-Order initial value problems (IVPs)

- Sometimes it is not possible to find the exact solution of certain differential equations, then in this case we are forced to compute their numerical solutions.
- To find the numerical solutions of the first-order ordinary differential equations, we use **ode45** command.
- This command attempts to produce an approximate solution with error less than or equal to  $10^{-3}$ .
- This command produces the plot of the approximate solution by default.
- To make the picture more clear, consider the following example:



### Example1:

- Find the numerical plot of the following initial value problem on  $[0, 2]$ :

$$y'(x) = y - x^2 + 1, \quad y(0) = 0.5$$

### Solution:

- Define the right hand side as an anonymous function, like that
- $f = @(x,y)y - x.^2 + 1.$
- The dot in front of the power sign indicates that the function must be able to operate on vectors.
- The recipe of the solution is given in the next slide.



- »  $f = @(x,y)y - x.^2 + 1$   
f =  
 $@(x,y)y - x.^2 + 1$   
» `ode45(f,[0,2],0.5)`  
» `xlabel('x');`  
» `ylabel('y');`  
» `title('Graphical view of the numerical solution of  $y'(x) = y - x^2 + 1$ ')`
- The graphical view of the approximate solution is given in the next slide

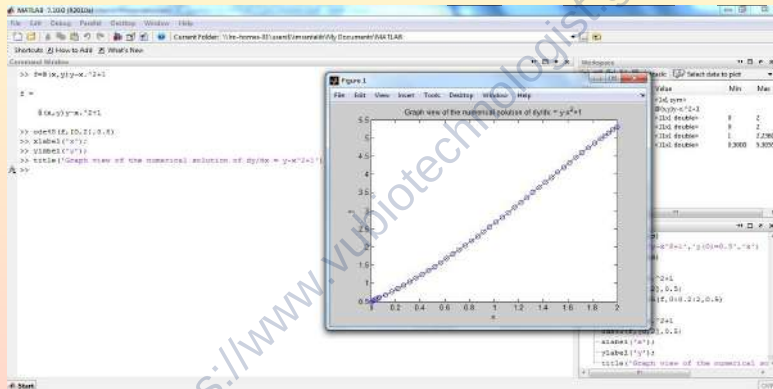


Figure: Graphical view of the numerical solution of  $y'(x) = y - x^2 + 1$



- Using **ode45** command, we can also plot the family of solutions of the above initial value problem.
- For this, we have to define the initial condition  $y(0)$  at different values. For example
- $y(0) = 0.5 : 0.2 : 3$ .
- That is we have to change the third argument (0.5) of the command **ode45(f,[0,2],0.5)** with 0.5:0.2:3.
- Study the following example:



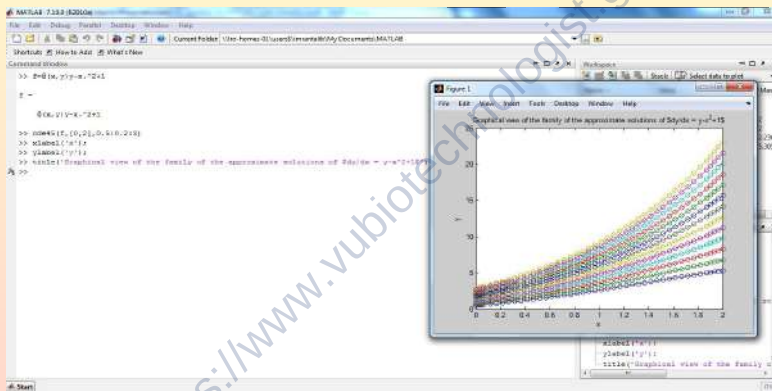
## Example2:

- Plot the family of the approximate solutions of the following IVPs

$$y'(x) = y - x^2 + 1, \quad y(0) = 0.5 : 0.2 : 3$$

## Solution:

- The recipe of the solution is as under:
- »  $f = @(x,y)y - x.^2 + 1$   
 $f =$   
 $@(x,y)y - x.^2 + 1$   
» `ode45(f,[0,2],0.5:0.2:3)`  
» `xlabel('x');`  
» `ylabel('y');`  
» `title('Graphical view of the family of the approximate solutions of  $y'(x) = y - x^2 + 1$ ')`
- See also the following figure.



**Figure:** Graphical view of the numerical solution of  $y'(x) = y - x^2 + 1$



## Extraction of the numerical values at some particular domain values

- Up to now, we see the results of the approximate solutions in the form of graphs.
- But using **ode45** command, we can also extract the numerical values at some specific domain values.
- For this, we have to make some changes in the second argument of the command **ode45(f,[0,2],0.5)**.
- Study the next example.



**Example3:**

- Compute the numerical values of the approximate solution of the following IVP at  $x = 0:0.2:2$

$$y'(x) = y - x^2 + 1, \quad y(0) = 0.5$$

**Solution:**

- The recipe of the solution is as under:

- $\gg f = @(x,y)y - x.^2 + 1$

$f =$

$@(x,y)y - x.^2 + 1$

- $\gg [x,ya]=ode45(f,0:0.2:2,0.5)$

$x =$

0

0.200000000000000000

0.400000000000000000

0.600000000000000000

0.800000000000000000

1.000000000000000000

1.200000000000000000



## Solution:

- $x = 1.6000000000000000$

$$1.8000000000000000$$

$$2.0000000000000000$$

- $y_a =$

$$0.5000000000000000$$

$$0.829298806102213$$

$$1.214087852350423$$

$$1.648940818452941$$

$$2.127229773242783$$

$$2.640859343211129$$

$$3.179941816703143$$

$$3.732400315281681$$

$$4.283484106130412$$

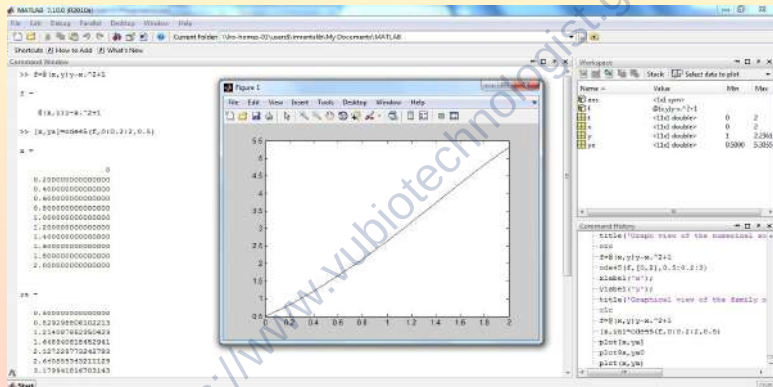
$$4.815176603278762$$

$$5.305472351203669$$



## Solution:

- Note that, MATLAB stores the domain values in vector  $x$ .
- The corresponding approximate values are stored in vector  $ya$ .
- The graphical view corresponding to these specific domain values is given in the next slide.



**Figure:** Graphical view of the numerical solution of  $y'(x) = y - x^2 + 1$  at some specific domain values



- We can also compare the exact and approximate solutions to observe the accuracy of the **ode45** command.
- For this purpose, we have to find the exact solution of the under-discussion differential problem.
- The methodology is discussed in Example 4.

#### Example4:

- Find the exact solution of the following IVP then compute its approximate solution at  $x = 0:0.2:2$  using **ode45** command. Finally, makes the table of the values of  $t$ ,  $ya$ , and  $y$ (exact solution).

$$y'(x) = y - x^2 + 1, \quad y(0) = 0.5$$



## Solution:

- The recipe of the solution is as under:
- »  $\text{eqn} = 'Dy = y - x^2 + 1';$   
»  $\text{initialcondn} = 'y(0)=0.5';$   
»  $y = \text{dsolve}(\text{eqn}, \text{initialcondn}, 'x')$   
 $y =$   
 $2 * x - \exp(x)/2 + x^2 + 1$   
»  $f = @(x,y)y - x.^2 + 1;$   
»  $[x, ya] = \text{ode45}(f, 0:0.2:2, 0.5);$   
»  $[x, ya, 2.*x - \exp(x)./2 + x.^2 + 1]$
- The output is displayed in the next slide



## First-Order initial value problems

### Examples

Family of approximate solutions

Extraction of the numerical values at some particular domain values

Comparison between exact and approximate solutions

Limitations of the ode45 command

ode45 command when initial condition point lies between the interval

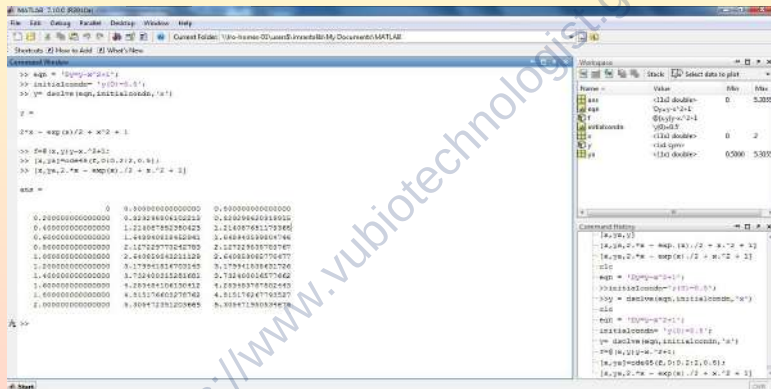


Figure: Comparison between exact and approximate solutions of  $y'(x) = y - x^2 + 1$  at some specific domain values



## Solution:

- Note that, MATLAB stores the domain values in vector  $x$ .
- The corresponding approximate values are stored in vector  $ya$ .
- And the corresponding values of the exact solution are stored in  $y$  (Third Column).
- It is to be noted that the approximate solution is correct to at least 4 digits in all cases





## Limitations of the ode45 command

- **ode45** command will only compute a solution on an interval with the initial condition is given at one of the end points.
- We have discussed the case when initial condition is given at the left end point in the above discussed examples.
- But when the initial condition is given at the right end point of the interval, then enter the values in decreasing order.
- For example, to plot the approximate solution of  $y'(x) = y - x^2 + 1$ ,  $y(0) = 0.5$  on the interval  $[-1, 0]$ , type the following syntax in the command
- `ode45(f,[0,-1],0.5).`



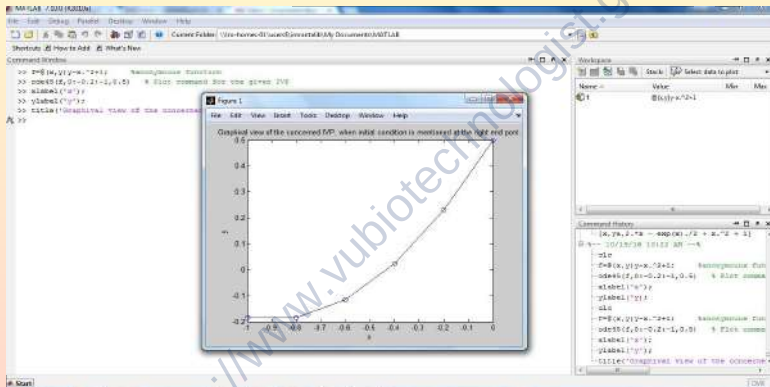
### Example5:

- Plot the approximate solutions of the following IVP at some specific domain elements,  $x = -0.2, -0.4, -0.6, -0.8, -1, 0$  in the interval  $[-1, 0]$  using **ode45** command.

$$y'(x) = y - x^2 + 1, \quad y(0) = 0.5$$

### Solution:

- The recipe of the solution is as under:
- »  $f = @(x,y)y - x.^2 + 1;$   
» `ode45(f,0:-0.2:-1,0.5)`  
» `xlabel('x');`  
» `ylabel('y');`  
» `title('Graphical view of the concerned IVP, when initial condition is mentioned at the right end pont.')`
- The graphical view of the approximate solution is given in the next slide.



**Figure:** Graphical view of the concerned IVP, when initial condition is mentioned at the right end point.



- Sometimes we are interested in finding the approximate solution of the IVP, when initial condition is defined at the end points of the interval but the end points of the interval are other than the initial point. Then in such situation, we subdivide the interval
- For example, if the initial condition is defined at 0, and the domain of interest is  $[-1, 2]$ , then the possible subdivision of the domain set is  $[-1, 2] = [-1, 0] \cup [0, 2]$ .
- Then we compute the approximate solution separately on each subinterval.
- The procedure is explained in the coming slide.



### Example6:

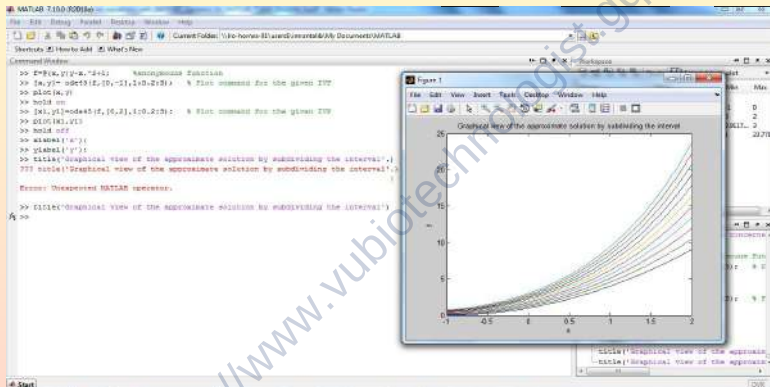
- Plot the family of the approximate solutions of the following IVP on the interval  $[-1, 2]$ . using **ode45** command.

$$y'(x) = y - x^2 + 1, \quad y(0) = 1 : 0.2 : 3$$



## Solution:

- The recipe of the solution is as under:
- »  $f = @(x,y)y - x.^2 + 1;$
- » `[x,y]=ode45(f,[0,-1],1:0.2:3);`
- » `plot(x,y)`
- » `hold on`
- » `[x1,y1]=ode45(f,[0,2],1:0.2:3);`
- » `plot(x1,y1)`
- » `hold off`
- » `xlabel('x');`
- » `ylabel('y');`
- » `title('Graphical view of the approximate solution by subdividing the interval')`
- The graphical view is shown in the following slide



**Figure:** Graphical view of the family of approximate solutions by subdividing the interval.

$$\Gamma(\tau + 1 + i)$$



# Matrix Algebra and MATLAB

Department of Mathematics and Statistics  
Virtual University of Pakistan, Lahore, Pakistan.

[imrantalib@vu.edu.pk](mailto:imrantalib@vu.edu.pk)





# Outline

- 1 Matrices in MATLAB
  - Creating Vector and Matrix Variables
- 2 Matrices
  - Some useful operations on matrices
- 3 Matrix Algebra
  - Matrices Multiplication
  - Determinant of a matrix
  - Inverse of a matrix
- 4 Exercise



# Outline

- 1 Matrices in MATLAB
  - Creating Vector and Matrix Variables
- 2 Matrices
  - Some useful operations on matrices
- 3 Matrix Algebra
  - Matrices Multiplication
  - Determinant of a matrix
  - Inverse of a matrix
- 4 Exercise



# Outline

- 1 Matrices in MATLAB
  - Creating Vector and Matrix Variables
- 2 Matrices
  - Some useful operations on matrices
- 3 Matrix Algebra
  - Matrices Multiplication
  - Determinant of a matrix
  - Inverse of a matrix
- 4 Exercise



# Outline

- 1 Matrices in MATLAB
  - Creating Vector and Matrix Variables
- 2 Matrices
  - Some useful operations on matrices
- 3 Matrix Algebra
  - Matrices Multiplication
  - Determinant of a matrix
  - Inverse of a matrix
- 4 Exercise



Matrices can be introduced into MATLAB in several ways:

- Entered by an explicit list of elements.
- Generated by so called built-in statements and functions.
- Loaded by external data.
- Created by M-Files.

Generating matrices as a list of its elements. The following basis conventions have to be taken into account:

- The elements of a row are separated by commas or spaces.
- A semi-colon indicates the end of each row.
- The entire list of elements has to be surrounded by brackets [ ].

# Row Vector

- Row vector variables can be created in several ways.
- The simplest method is to put the values that you want in the variable in square brackets, separated by either spaces or commas.
- For instance, a row vector with four elements in MATLAB can be generated as:
- `rowvec=[2,4,5,6]`
- The required output is as under
- `rowvec =`  
`2 4 5 6`

## Row vector using Colon Operator

- Row vector can also be generated using Colon operator(;)
- The entries in the generated Row vector iterates from the starting to the ending value with a default step of one.
- In this case the square brackets are not necessary.
- For example, a row vector having 2, 3, 4, 5, 6 entries can be generated as
  - » colvec=2:6
- The necessary output is
- colvec =  
2 3 4 5 6



## Colon Operator with Step value

- Row vector with Colon operator can also be generated by defining step value for each entry of the matrix.
- For example, the row vector with entries 2, 4, 6, 8, 10 can be generated as
- » `stepvec = 2:2:10`
- The desired output is
- `stepvec =`  
2 4 6 8 10

- Row vector is generated in different ways

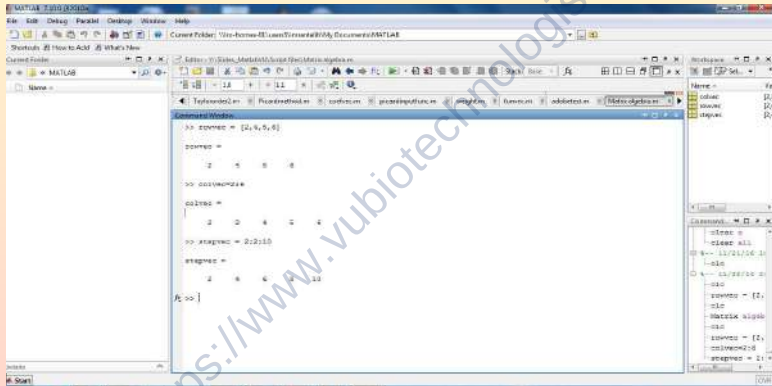


Figure: Display of the Row vector in different ways

# Column Vector

- There are two basic methods for creating Column vector:
- Either by putting the values in square brackets, separated by semicolons.
- or by creating a row vector and then transposing it.
- The transpose operator in MATLAB is the apostrophe.

## Column Vector with Square bracket and semicolons

- For example, the column vector with entries 2, 5, 10 can be generated as
- » `colvec=[2;5;10]`
- The required output is  
colvec =  
2  
5  
10

# Column Vector with apostrophe

- Alternatively, the column vector with entries 2, 5, 10 can also be generated as
- » Rowvec=[2,5,10]  
 Rowvec =  
 2 5 10  
 » colvec=Rowvec'  
 colvec =  
 2  
 5  
 10

- Column vector is generated in different ways

```

MATLAB 7.3.0 (R2006b)

>> colvec=[2;5;10]

colvec =

 2
 5
 10

>> Rowvec=[2,5,10]

Rowvec =

 2 5 10

>> colvec=Rowvec'

colvec =

 2
 5
 10

fx >>

```

Figure: Display of the Column vector in different ways

- Matrix variables can be created by putting the values in square brackets.
- The values within the rows are separated by either spaces or commas.
- Each individual row is separated by semicolons.
- The must always be the same number of values in every row.
- The colon operator can be used to iterate within the rows.



- For example, the matrix

$$A = \begin{bmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{bmatrix}$$

in MATLAB is generated as

- » `A=[5/6,1/6,0;5/6,0,1/6;0,5/6,1/6]`

A =

0.8333 0.1667 0

0.8333 0 0.1667

0 0.8333 0.1667

or





- $A = \text{sym}([5/6, 1/6, 0; 5/6, 0, 1/6; 0, 5/6, 1/6])$

$$A = \begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$$

- Generation of the matrix in MATLAB

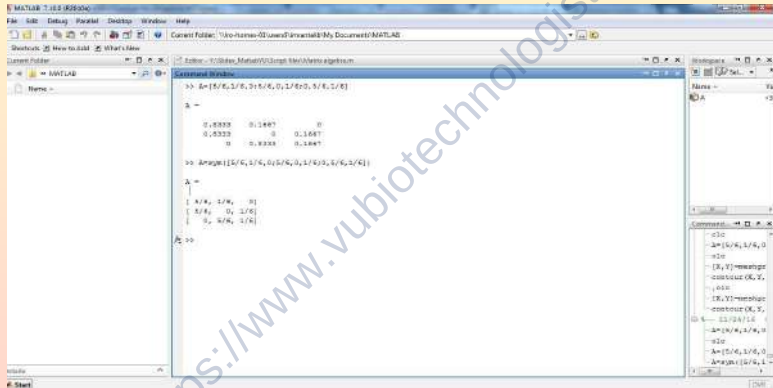


Figure: Graphical view of the matrices

# Sum of column vectors of a matrix

- The **sum** command computes a **Row vector** containing the sum of the **columns** of a matrix.
- Note that MATLAB prefers to work with the columns of a matrix.
- For example, the sum of the columns of the matrix  $A$  is computed as:
- 
- $A = \text{sym}([5/6, 1/6, 0; 5/6, 0, 1/6; 0, 5/6, 1/6])$

$$A = \begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$$

$$\gg B = \text{sum}(A)$$

$$B =$$

$$\left( \frac{5}{3} \quad 1 \quad \frac{1}{3} \right)$$

## Sum of row vectors of a matrix

- To compute the sum of Row vectors of a matrix, we should take the transpose, then compute the sum and finally transpose the result.
- For example, the sum of the **Row** vectors of the matrix  $A$  is calculated in the next slide. Please see it.

- Sum of the entries of the column vectors of A

The image shows the MATLAB 7.10.0 R2010a interface. The Command Window displays the following code and output:

```
>> A=sym([5/6,5/6,0;5/6,0,1/6;0,5/6,1/6])
A =
[5/6, 5/6, 0]
[5/6, 0, 1/6]
[0, 5/6, 1/6]

>> B=sum(A)
B =
[5/3, 1, 5/3]

A >>
```

The Workspace window shows the variables A and B. The Command History window shows the commands entered: `ans =`, `A =`, `B = sum(A)`, `B =`, and `A = sym([5/6, 5/6, 0; 5/6, 0, 1/6; 0, 5/6, 1/6])`.

Figure: Display of the sum command

# Sum of row vectors of a matrix

```

» A =
 5/6 1/6 0
 5/6 0 1/6
 5/6 5/6 1/6
 0 0 0

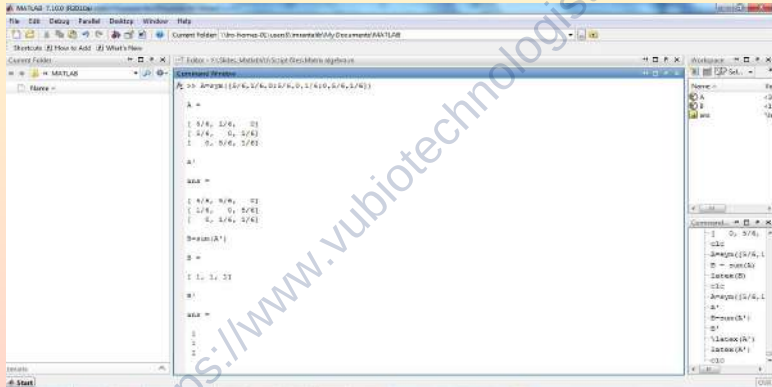
» A' =
 5/6 5/6 0
 1/6 0 5/6
 0 1/6 1/6

» B = sum(A')
B =
 1 1 1

» B'
ans =
 1
 1
 1.

```

- Sum of the entries of the row vectors of  $A$



The image shows a MATLAB Command Window with the following code and output:

```

>> A=eye(3)/6; B=[0, 1/6, 0; 1/6/6, 0, 1/6; 0, 1/6, 1/6];
A =
 1/6 1/6 0
 1/6 0 1/6
 0 1/6 1/6

B =
 0 1/6 0
 1/6 0 1/6
 0 1/6 1/6

S=sum(A')
S =
 1 1 1

sum(S)
ans =
 3

```

The Command Window also shows the workspace variables: A, B, S, and ans.

Figure: Display of the sum command

# Column vector containing the Diagonal elements of a matrix

- The **diag** command produces a column vector containing the elements of the main diagonal of a matrix.

- For example,

- »  $A = \begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$

- The MATLAB displays the following result:

- »  $B = \text{diag}(A)$

$B =$

$\frac{5}{6}$

$0$

$\frac{1}{6}$



## Sum of individual row and column

- The collective use of **sum** command and **colon (:)** operator is used to compute the sum of any individual column and row of a matrix.
- The colon operator refers to all elements in a row or column of a matrix.
- The key word **end** refers to last column or row.
- For example, the sum of last column of the matrix *A* in MATLAB is calculated as

$$\gg A = \begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ \frac{5}{6} & \frac{5}{6} & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$$

- The MATLAB displays the following result:

$$\gg B = \text{sum}(A(:, \text{end}))$$

$$B =$$

$$1/3$$

- Sum of the entries of the last and the second column of A

The image shows a MATLAB Command Window with the following code and output:

```

>> A=ones(3,3)/8; B=[5,0,1/6; 0,1/6,0; 1/6,0,1/6];
A =

 1/8 1/8 1/8
 1/8 0 1/6
 1/8 1/6 1/6

>> C=sum(A(:,end))

C =

 1/2

>> C=sum(A(:,2))

C =

 1

A/;>

```

The output shows that the sum of the last column of A is 1/2, and the sum of the second column of A is 1.

Figure: Display of the sum command with colon operator

## Matrices multiplication using for loop

- The matrix multiplication can be defined using MATLAB's **for** loop, colon notation, and vector scalar product.
- For example, the MATLAB syntax for the multiplication of two matrices  $A$  and  $B$  using for loop is as under:

```
clear all;
close all;
clc;
A=[1,2,3;4,5,6];
B=[1,4;6,7;0,1];
for i=1:2
 for j=1:2
 C(i,j)=A(i,:)*B(:,j);
 end
end
C
```

- Multiplication of the matrices  $A$  and  $B$

The screenshot shows the MATLAB 7.10.0 R2010a interface. The Command Window displays the following code and output:

```

1 = clear all;
2 = close all;
3 = clc;
4 = A=[1,2,3;4,5,6];
5 = B=[1,4;6,7;0,1];
6 = %size A=1x2
7 = %size B=2x2
8 = C=A*B;
9 = %size C=1x2
10 = end
11 =

```

The Command Window output shows:

```

C =
 13 28
 24 50

```

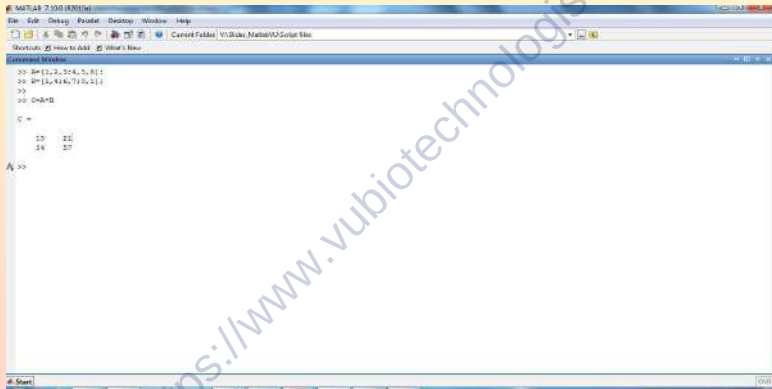
The Workspace window shows the variables A, B, and C. The Command History window shows the commands entered.

Figure: Matrices multiplication using for loop

## Matrices multiplication using asterisk operator

- Single asterisk operator (\*) can also be used for the matrices multiplication.
- For example, above defined matrices,  $A$  and  $B$  can also be multiplied in MATLAB as
  - »  $A=[1,2,3;4,5,6];$
  - »  $B=[1,4;6,7;0,1];$
  - »
  - »  $C=A*B.$

- Multiplication of the matrices  $A$  and  $B$ .



The image shows a screenshot of the MATLAB 7.10.0 (R2010a) interface. The Command Window displays the following code and output:

```
>> A=[1,2,3;4,5,6];
>> B=[1,2,3;4,5,6];
>>
>> C=A*B
```

The output shows the resulting matrix C:

```
C =
13 14 15
34 35 36
```

Figure: Matrices multiplication using asterisk operator



- The determinant of a square matrix in MATLAB can be determined using **det** command.
- For example, the determinant of the following matrix is determined as
- »  $A = \begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$
- » `A=sym([5/6,1/6,0;1/6,0,5/6;0,5/6,1])`
- » `det(A)`
- The desired MATLAB output is  
ans =  
-131/216

- The inverse of a square matrix in MATLAB is determined using the **inv** command.
- For example, the inverse of the matrix  $A$  is computed as
- » `inv(A)`
- The required output is as under:

$$\text{ans} = \begin{pmatrix} \frac{150}{131} & \frac{36}{131} & -\frac{30}{131} \\ \frac{36}{131} & -\frac{180}{131} & \frac{150}{131} \\ -\frac{30}{131} & \frac{150}{131} & \frac{6}{131} \end{pmatrix}$$



- Determinant and inverse of the matrix  $A$ .

Figure 1: MATLAB R2010b command window showing the calculation of the inverse of a 3x3 matrix A. The matrix A is defined as  $A = \begin{bmatrix} 5/6 & 1/6 & 0 \\ 1/6 & 0 & 1/6 \\ 0 & 5/6 & 1 \end{bmatrix}$ . The inverse  $A^{-1}$  is calculated using the 'inv' function and displayed as a matrix with fractions. The result is then converted to LaTeX format using the 'latex' function, showing the LaTeX code for the inverse matrix.

Figure: det and inv command in MATLAB



Compute  $AB$  using for loop in MATLAB, if

- $A = \begin{pmatrix} 5 & 1 & 0 \\ 6 & 6 & 5 \\ 6 & 0 & 6 \\ 0 & 5 & 1 \end{pmatrix}$
- $B = \begin{pmatrix} 5 & 1 & 0 \\ 2 & 2 & 5 \\ 1 & 0 & 3 \\ 3 & 5 & 1 \\ 0 & 5 & 1 \end{pmatrix}$



Compute the sum of the second column of the following matrix using **sum** command.

- $C = \begin{pmatrix} 1 & \frac{1}{2} & 0 \\ \frac{1}{3} & \frac{1}{6} & 2 \\ 0 & 0 & 1 \end{pmatrix}$

- Compute the sum of the second row of the matrix  $C$  using **sum** command.



# Linear Algebra and MATLAB

Department of Mathematics and Statistics  
Virtual University of Pakistan, Lahore, Pakistan.

*imrantalib@vu.edu.pk*



# Outline

- 1 Eigen Values and Eigen Vectors
- 2 Reduced row echelon form
- 3 Solution of system of linear equations
  - Solution of equations when coefficient matrix have inverse
- 4 Elementary row operations
- 5 Spanning set
- 6 Exercise



# Outline

- 1 Eigen Values and Eigen Vectors
- 2 Reduced row echelon form
- 3 Solution of system of linear equations
  - Solution of equations when coefficient matrix have inverse
- 4 Elementary row operations
- 5 Spanning set
- 6 Exercise



# Outline

- 1 Eigen Values and Eigen Vectors
- 2 Reduced row echelon form
- 3 Solution of system of linear equations
  - Solution of equations when coefficient matrix have inverse
- 4 Elementary row operations
- 5 Spanning set
- 6 Exercise



# Outline

- 1 Eigen Values and Eigen Vectors
- 2 Reduced row echelon form
- 3 Solution of system of linear equations
  - Solution of equations when coefficient matrix have inverse
- 4 Elementary row operations
- 5 Spanning set
- 6 Exercise





# Outline

- 1 Eigen Values and Eigen Vectors
- 2 Reduced row echelon form
- 3 Solution of system of linear equations
  - Solution of equations when coefficient matrix have inverse
- 4 Elementary row operations
- 5 Spanning set
- 6 Exercise



# Outline

- 1 Eigen Values and Eigen Vectors
- 2 Reduced row echelon form
- 3 Solution of system of linear equations
  - Solution of equations when coefficient matrix have inverse
- 4 Elementary row operations
- 5 Spanning set
- 6 Exercise



- The **eig** command is used to determine the Eigen value of a square matrix.
- This command with slightly modification also used to determine the associated Eigen vector.
- Consider the following example:

### Example:

- Compute the eigen values and eigen vectors for the following matrix:

$$B = \begin{pmatrix} 0 & 1 & 3 \\ 2 & 4 & 1 \\ 6 & 1 & 8 \end{pmatrix}$$



- The recipe of the solution is as under:
- $B = [0, 1, 3; 2, 4, 1; 6, 1, 8];$   
 $\text{eig}(B)$   
 $[P, D] = \text{eig}(B)$

### Note:

- The columns of the matrix  $P$  denotes the eigen vectors of the matrix  $B$ .
- The matrix  $D$  denotes the diagonal matrix having eigen values on the main diagonal.



- Computation of the eigen values and eigen vectors.

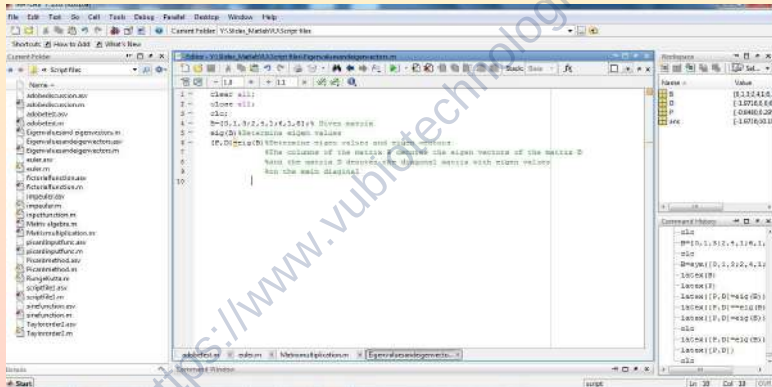


Figure: Editor file having code of eigen values and eigen vectors



- Results of the eigen values and eigen vectors.

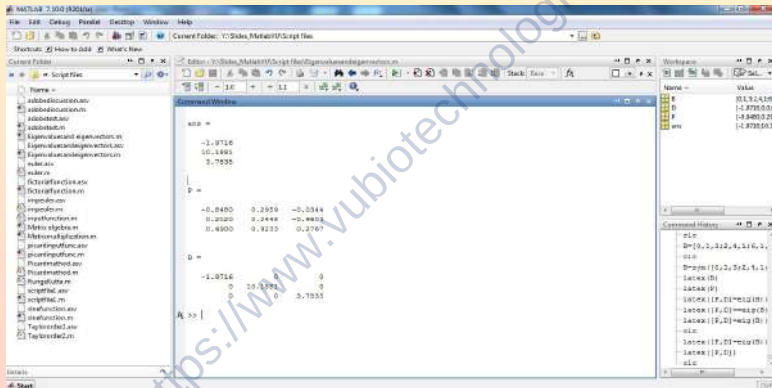


Figure: Command window having outputs of the eigen values and eigen vectors



- MATLAB also gives the reduced echelon form of a matrix.
- For this **rref** command is used.
- Consider the following example:

### Example:

- Find the reduced echelon form of the following matrix:

$$B = \begin{pmatrix} 0 & 1 & 3 \\ 2 & 4 & 1 \\ 6 & 1 & 8 \end{pmatrix}$$



- Reduced echelon form.

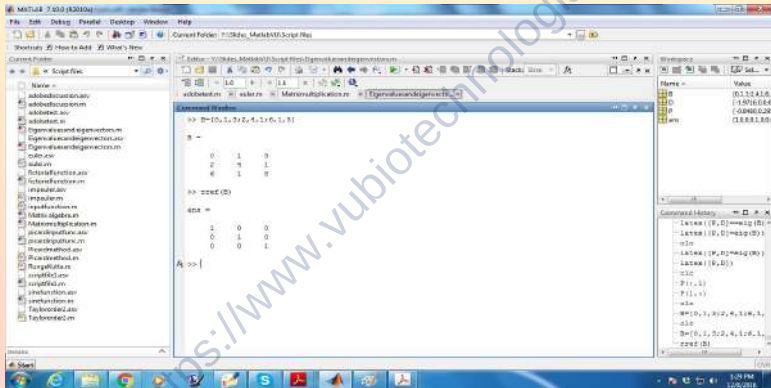


Figure: output of `rref` command





- While dealing with system of linear equations two situation often occurs:
- Whether the coefficient matrix have inverse or have not?
- First we discuss the case when the coefficient matrix have inverse



### Example:

- Find the solution of the following system of linear equations:

$$\begin{aligned}x_1 + 3x_2 &= -2 \\ 2x_1 + 4x_2 &= 1.\end{aligned}\tag{3.1}$$

### Solution:

- The coefficient matrix is as under:

$$A = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

- The column matrix is given below:

$$B = \begin{pmatrix} -2 \\ 1 \end{pmatrix}$$



- The MATLAB code for the solution of the system is as under:
- $A = \text{sym}([1,3;2,4]);$   
 $B = \text{sym}([-2;1]);$   
 $C = \text{inv}(A);$   
 $X = C*B$
- For more clarity see the next slide.

- Solution of linear equations in MATLAB when coefficient matrix have inverse.

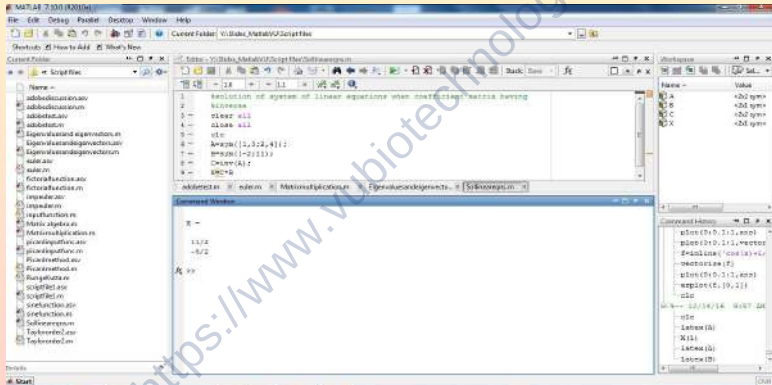


Figure: Display of the MATLAB Editor with code of the solution of linear equations



### Example:

- Solve the following system of linear equations using **solve** command:

$$\begin{aligned}3x + 2y + z &= 1, \\ y &= 2, \\ x + 2y &= 3.\end{aligned}\tag{3.2}$$



## Solution:

- The MATLAB code for the solution is:
- ```
eqn1='3*x+2*y+z=1';  
eqn2='y=2';  
eqn3='x+2*y=3';  
sol=solve(eqn1,eqn2,eqn3);  
X=double([sol.x,sol.y,sol.z])'
```



• Solution of linear equations

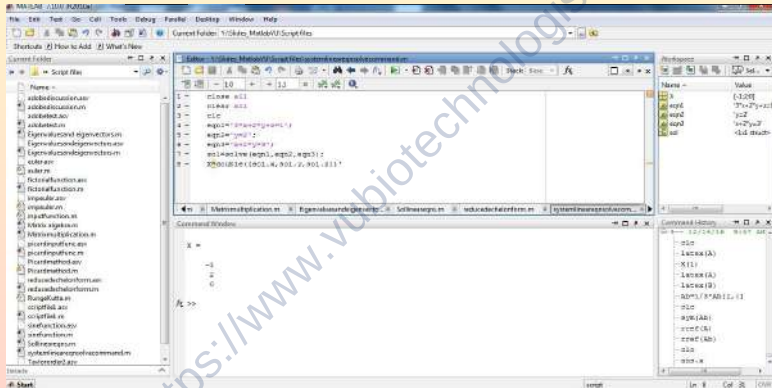


Figure: Display of the MATLAB Editor with code of the solution of linear equations using solve command



- The matrix can also be converted into reduced echelon form using row operations.
- Consider the following example:

Example:

- Consider the system of linear equations:

$$\begin{aligned}3x + 2y + z &= 1, \\ y &= 2, \\ x + 2y &= 3.\end{aligned}\tag{4.1}$$

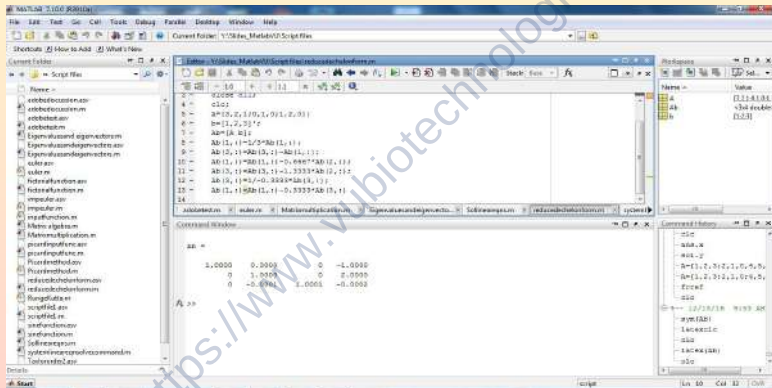
- Enter it into MATLAB as an augmented matrix named Ab .
- Use elementary row operations to reduce it to reduced-echelon form.



Solution:

- The MATLAB recipe for the solution is as under:
- $A = [3, 2, 1; 0, 1, 0; 1, 2, 0];$
 $b = [1, 2, 3]';$
 $Ab = [A \ b];$
 $Ab(1,:) = 1/3 * Ab(1,:);$
 $Ab(3,:) = Ab(3,) - Ab(1,:);$
 $Ab(1,:) = Ab(1,) - 0.6667 * Ab(2,:);$
 $Ab(3,:) = Ab(3,) - 1.3333 * Ab(2,:);$
 $Ab(3,:) = 1 / -0.3333 * Ab(3,:);$
 $Ab(1,:) = Ab(1,) - 0.3333 * Ab(3,);$

Reduced echelon form using row operations





- **rref** command is also used to check that whether a vector can be written as a linear combination of the elements (vectors) of a set or not?
- Consider the following example:

Example:

- Show that $\begin{pmatrix} a \\ b \end{pmatrix}$ is in the span of $\begin{pmatrix} -2 & 1 \\ 1 & 6 \end{pmatrix}$ for any a and b .



Solution:

- First declare that a and b as symbolic.
- Enter the matrix Ab that can help you to show this.
- Use **rref** on the matrix Ab .
- Explicitly give the weights w_1 and w_2 , such that:

$$\begin{pmatrix} a \\ b \end{pmatrix} = w_1 \begin{pmatrix} -2 \\ 1 \end{pmatrix} + w_2 \begin{pmatrix} 1 \\ 6 \end{pmatrix}$$



Solution:

- The MATLAB code for the solution is as under:

```
clear all;  
close all;  
clc;  
syms a b  
A=[-2,1;1,6];  
b=[a;b];  
Ab=[A b];  
rref(Ab)
```

- The possible values for the weights are $\begin{pmatrix} \frac{b}{13} - \frac{6a}{13} \\ \frac{a}{13} + \frac{2b}{13} \end{pmatrix}$



Application of **rref** command

The image shows the MATLAB 7.10.0 (R2010a) environment. The main window displays a script with the following code:

```

1 = clear all;
2 = close all;
3 = clc;
4 = a=[1 0 0; 0 1 0; 0 0 1];
5 = b=[2; 1; 1; 0];
6 = [a b];
7 = rref(a);
8 = [a rref(a)];

```

The Command Window shows the execution of the script, displaying the matrix `a` and the result of the `rref` command:

```

a =
1 0 0
0 1 0
0 0 1

rref(a) =
1 0 0
0 1 0
0 0 1

```

The Workspace window shows the variables `a` and `rref(a)` with their respective values.

Figure: MATLAB code to determine the weights



- Consider the system of equations

$$\begin{aligned}x_1 + 2x_2 - 3x_3 &= -3, \\-4x_1 - 5x_2 + 2x_3 &= -2, \\2x_1 + 3x_2 - x_3 &= 2.\end{aligned}\tag{6.1}$$

- Enter it into MATLAB as an (augmented) matrix named Ab .
- Use elementary row operations to reduce it to reduced echelon form.
- Give the solution of the system. (If the system has no solution say that. Otherwise, write the basic variables (those corresponding to pivot columns) as functions of numbers and free variables, if there are any.)



- Consider the system of equations

$$\begin{aligned}x_1 + 3x_3 + x_5 &= -1, \\4x_1 + x_2 + x_3 + 6x_5 &= 1, \\-3x_1 - 3x_2 - 3x_3 + x_4 - 19x_5 &= 6, \\10x_1 - 4x_2 + 38x_3 + 2x_4 - 12x_5 &= 0.\end{aligned}\tag{6.2}$$

- Enter it into MATLAB as an (augmented) matrix named Ab .
- Use elementary row operations to reduce it to reduced echelon form.
- Give the solution of the system. (If the system has no solution say that. Otherwise, write the basic variables (those corresponding to pivot columns) as functions of numbers and free variables, if there are any.)



- Show that $\begin{pmatrix} h \\ k \end{pmatrix}$ is in the span of $\begin{pmatrix} -3 & 2 \\ 4 & 5 \end{pmatrix}$ for any h and k .



Numerical Methods for Integration

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk



Outline

- 1 Riemann Sum
 - Brief introduction
 - Examples
 - Random selection of the points
- 2 Numerical Methods for Integration Part-I
 - Trapezoidal Rule
 - Simpson's one-Third Rule
 - Simpson's Three-Eight Rule
- 3 Exercise

- 1 Riemann Sum
 - Brief introduction
 - Examples
 - Random selection of the points
- 2 Numerical Methods for Integration Part-I
 - Trapezoidal Rule
 - Simpson's one-Third Rule
 - Simpson's Three-Eight Rule
- 3 Exercise

- 1 Riemann Sum
 - Brief introduction
 - Examples
 - Random selection of the points
- 2 Numerical Methods for Integration Part-I
 - Trapezoidal Rule
 - Simpson's one-Third Rule
 - Simpson's Three-Eight Rule
- 3 Exercise



- The MATLAB built-in-function **quad** is used to approximate definite integrals that could not be evaluated by Fundamental Theorem of Calculus.
- In order to understand the type of calculation carried out by **quad**, we will develop a variety of algorithms for approximating values of definite integrals.
- In this section we will introduce the Right End Point Method, Mid Point Method.
- First, we get the idea of Riemann sums



- Let $f(x)$ be a continuous function on interval $[a, b]$, and suppose this interval is partitioned by the values
 $a = x_0 < x_1 < x_2 < \dots < x_{n-1} < x_n = b$.
- Any sum of the form

$$R = \sum_{k=1}^n f(c_k) \Delta x_k, \quad (1.1)$$

where $\Delta x_k = x_k - x_{k-1}$, and $c_k \in [x_{k-1}, x_k]$ is referred to as a Riemann sum of f . Let

$$\|P\| := \max(\Delta x_1, \Delta x_2, \dots, \Delta x_n) \quad (1.2)$$

denote the norm of the partition, then we say f is Riemann integrable if

$$\lim_{\|P\| \rightarrow 0} \sum_{k=1}^n f(c_k) \Delta x_k \quad (1.3)$$

exists.

Example: Compute the Riemann sum of the function $f(x) = e^{-x^2}$ for $n=4,6,8,10,100$, and 1000 by choosing c_K as the right end point of the interval $[0,2]$.

Solution:

- If we take a regular partition with n intervals, then each interval has length $\Delta x = \frac{b-a}{n}$.
- The k th end point is $x_k = a + k\Delta x$.
- The Riemann sum becomes

$$R = \sum_{k=1}^n f(c_k = a + k\Delta x) \Delta x_k, \quad (1.4)$$

- Particularly, for $n = 4$, it becomes

$$R = \sum_{k=1}^4 f(c_k = 0 + k0.5)0.5, \quad (1.5)$$



MATLAB code

- The MATLAB code for the solution is as under:

- `function value=riemannsum1(f,a,b,n)`

```
clear all;
```

```
clc;
```

```
close all;
```

```
a=0;
```

```
b=2;
```

```
n=100;
```

```
value=0;
```

```
f=inline('exp(-x^2)');
```

```
dx=(b-a)/n;
```

```
for k=1:n
```

```
    c=a+k*dx;
```

```
    value=value+f(c);
```

```
end
```

```
value=dx*value
```

- Riemann sum when c_k is chosen as the right end point of each sub-interval.

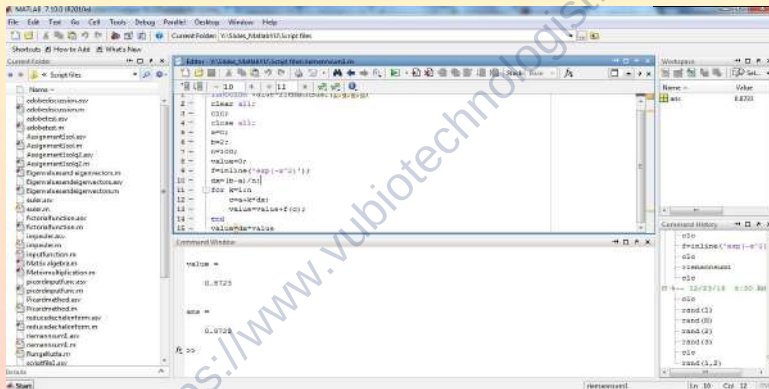


Figure: Riemann sum of the function e^{-x^2} on $[0, 2]$ for $n=100$.



- One interesting aspect of the Riemann sum is that the point c_k need not be chosen in the same place on each interval.
- For example, suppose we partition the interval $[0, 1]$ with $x_0 < x_1 = \frac{2}{3} < x_2 = \frac{4}{3} < x_3 = 2$. If we choose c_1 as the left point of the interval $[0, 2/3]$, c_2 as the right end point of the interval $[2/3, 4/3]$, and c_3 as the left end point of the interval $[4/3, 2]$. Then the possible Riemann sum in this case is:

$$f(0)\frac{2}{3} + f(2/3)\frac{2}{3} + f(4/3)\frac{2}{3}. \quad (1.6)$$



Example: Compute the Riemann sum of the function $f(x) = e^{-x^2}$ for $n = 100$ by choosing c_K is randomly selected on the interval $[0, 2]$.

Solution:

- To choose randomly the points c_K , we know that the points $c = (1 - r)a + rb$ moves from a to b as r goes from 0 to 1.
- Our generic interval is $[a + (k - 1)\Delta x, a + k\Delta x]$ and **rand** denotes a value randomly chosen between 0 and 1.
- Therefore, $c = (1 - \text{rand})(a + (k - 1)\Delta x) + \text{rand}(a + k\Delta x)$ will give the formula that we use to develop the MATLAB code for randomly selection of the points.



Solution:

- The MATLAB code for the solution is as under:

- `function value=riemannsum2(f,a,b,n)`

```
clear all;
```

```
clc;
```

```
close all;
```

```
a=0;
```

```
b=2;
```

```
n=100;
```

```
value=0;
```

```
f=inline('exp(-x^2)');
```

```
dx=(b-a)/n;
```

```
for k=1:n
```

```
c=c+dx*rand+(a+(k-1)*dx);
```

```
value=value+f(c);
```

```
end
```

```
value=dx*value
```

- Riemann sum when the points c_k is selected randomly.

```

1: function value = riemannsum2(f, a, b, n)
2:     rand('seed', 0);
3:     c = randi(100);
4:     c = c * (b-a) / 100;
5:     s = 0;
6:     for k = 1:n
7:         x = a + (b-a) * rand;
8:         value = value + f(x);
9:     end
10:    value = value / n;
11: end
12:
13: % Example usage
14: f = @(x) exp(-x^2);
15: a = 0;
16: b = 2;
17: n = 100;
18: value = riemannsum2(f, a, b, n);
19: disp(value);

```

Command Window:

```

>> riemannsum2
0.0000

```

Figure: Riemann sum of the function e^{-x^2} on $[0,2]$ for $n=100$.



- The important observation for evaluation of the definite integrals using Riemann sum is that if a function is increasing then approximation by evaluation at right end points always overestimates the area, while approximation by evaluation at left end point always underestimates the area.
- Precisely the opposite is true for decreasing functions.
- So, in order to address this Trapezoidal Rule takes an average of these two approximations. That is

$$\begin{aligned} T_n &\approx \frac{\sum_{k=1}^n f(x_k) \Delta x_k + \sum_{k=1}^n f(x_{k-1}) \Delta x_k}{2}, \\ &\approx \frac{b-a}{n} \left(\frac{f(b) + f(a)}{2} + \sum_{k=1}^{n-1} f(x_k) \right), \quad \Delta x_k = \frac{b-a}{n}. \end{aligned} \quad (2.1)$$



Example: Compute the definite integral of $f(x) = e^{-x^2}$ using Trapezoidal Rule considering hundred sub-intervals (that is, $n = 100$) on the interval $[0, 2]$.

Solution:

- Here, $\Delta x_k = \frac{b-a}{n} = 0.02$.
- $\sum_{k=1}^{99} f(x_k) = f(1) + f(2) + \dots + f(99)$.
- Then Trapezoidal approximation for this case becomes:
-

$$T_n \approx \frac{b-a}{n} \left(\frac{f(b) + f(a)}{2} + \sum_{k=1}^{n-1} f(x_k) \right), \quad \Delta x_k = \frac{b-a}{n}, \quad (2.2)$$

$$T_{100} \approx \frac{2-0}{100} \left(\frac{f(2) + f(0)}{2} + f(1) + f(2) + \dots + f(99) \right).$$



- The MATLAB code for the solution is as under:

- `function value=trapezoidalrule(f,a,b,n)`

```
clear all;
```

```
clc;
```

```
close all;
```

```
sum=0;
```

```
a=0;
```

```
b=2;
```

```
n=100;
```

```
f = inline('exp(-x^2)');
```

```
dx = (b - a)/n;
```

```
Averagevalue=(f(b)+f(a))/2;
```

```
for k=1:(n-1)
```

```
c = a + k * dx;
```

```
sum=sum+f(c);
```

```
end
```

```
Answer= (sum+Averagevalue)*dx
```

Trapezoidal approximation of definite integral.

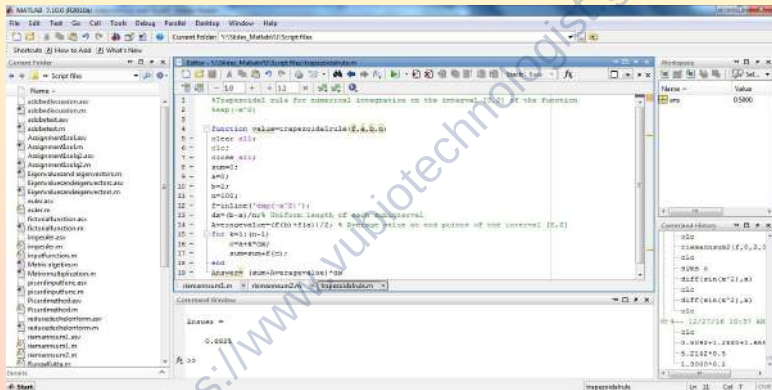


Figure: Evaluation of the definite integral of the function e^{-x^2} using Trapezoidal Rule on $[0,2]$ for $n=100$.



- The idea behind Simpson's Rule is to proceed by approximating $f(x)$ by quadratic polynomials rather than by linear one (lines).
- More generally, any order of polynomial can be used to approximate f , and higher orders give more accurate method.
- The general form of Simpson one-Third Rule is given as under:

$$\int_{x_0}^{x_{n+h}} f(x) dx \approx \frac{\Delta}{3} \left[(y(x_0) + y(x_n)) + 4(y(x_1) + y(x_3) + \dots + y(x_{n-1})) + 2(y(x_2) + y(x_4) + \dots + y(x_{n-2})) \right]. \quad (2.3)$$



Example:

- Compute the definite integral of $f(x) = e^{-x^2}$ on the interval $[0, 2]$ using Simpson's one-Third Rule by dividing the interval $[0, 2]$ into 6 sub-intervals

Note:

- While using Simpson one-Third Rule, the given interval of integration must be divided into an even number of sub-intervals, since we find the area over two sub-intervals at a time.



Solution:

- For this case, $n = 6$ (must be even).
- Calculate the width, $\Delta x = \frac{2-0}{6}$.
- Calculate the values of x_0 to x_6 as
 $x_0 = 0, x_1 = 0 + \Delta x, x_2 = 0 + 2\Delta x, \dots, x_6 = 0 + 6\Delta x = 2$.
- Now find the values of $f(x) = e^{-x^2}$ from $y(x_0)$ to $y(x_6)$ for the corresponding x_0 to x_6 .
- Finally, substitute all the above found values in the Simpson one-Third Rule to calculate the approximate integral value.



MATLAB code for the above example

- The MATLAB recipe for the Simpson's one-Third Rule is as under:

- `function simpson=Simpsononethird(f,a,b,n)`

```
format long
```

```
clear all;
```

```
clc;
```

```
close all;
```

```
sum1=0;
```

```
sum2=0;
```

```
a=0;
```

```
b=2;
```

```
n=6;
```

```
 $f = \text{inline}(' \exp(-x^2)');$ 
```

```
dx=(b-a)/n;
```

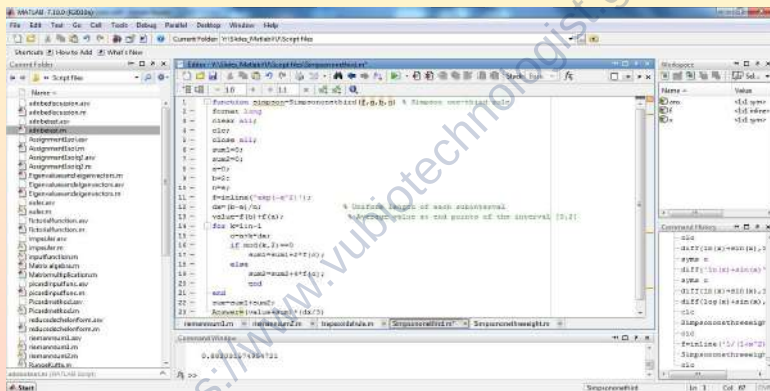
```
value=f(b)+f(a);
```



MATLAB code for the above example

```
for k=1:n-1
c=a+k*dx;
if mod(k,2)==0
sum1=sum1+2*f(c);
else
sum2=sum2+4*f(c);
end
end
sum=sum1+sum2;
Answer=(value+sum)*(dx/3)
```

Simpson's one-Third Rule to approximation the area.



The image shows a MATLAB script titled 'SimpsononeThird.m' in the MATLAB editor. The script defines a function `SimpsononeThird(f, a, b, n)` that calculates the definite integral of a function `f` over the interval `[a, b]` using Simpson's one-third rule with `n` subintervals. The function includes comments in Chinese and the following code:

```

1 function SimpsononeThird(f,a,b,n) % Simpson one-third rule
2 format long
3 clear all
4 clc
5 close all
6 sum=0;
7 sum2=0;
8 n=n/2;
9 h=(b-a)/n;
10 for k=1:n
11     % Uniform length of each subinterval
12     % Approximate value at the middle of the interval [a,b]
13     value=f(b)+f(a);
14     for k=1:n-1
15         c=a+h*k;
16         if mod(k,2)==0
17             sum=sum+4*f(c);
18         else
19             sum2=sum2+f(c);
20         end
21     end
22     sum=sum+sum2;
23     ApproximateValue=(h/3)*(sum+4*f((a+b)/2));

```

The script also includes a command window showing the result of the function call: `SimpsononeThird('exp(-x^2)', 0, 2, 6)` resulting in `0.89321674354722`.

Figure: Evaluation of the definite integral of the function e^{-x^2} using Simpson's one-Third Rule on $[0,2]$ for $n=6$.



- While using Simpson's Three-Eight Rule, the given interval of integration must be divided into sub-intervals whose number n is a multiple of 3.
- The general form of Simpson's Three-Eight Rule is given as under:

$$\int_{x_0}^{x_{n+h}} f(x) dx \approx \frac{3\Delta}{8} \left[(y(x_0) + y(x_n)) + 3(y(x_1) + y(x_2) + \dots + y(x_{n-1})) + 2(y(x_3) + y(x_6) + \dots + y(x_{n-3})) \right]. \quad (2.4)$$



Example:

- Compute the definite integral of $f(x) = e^{-x^2}$ on the interval $[0, 2]$ using Simpson's Three-Eight Rule by dividing the interval $[0, 2]$ into 6 sub-intervals .

Solution:

- For this case, $n = 6$ (must be a multiple of 3).
- Calculate the width, $\Delta x = \frac{2-0}{6}$.
- Calculate the values of x_0 to x_6 as
 $x_0 = 0, x_1 = 0 + \Delta x, x_2 = 0 + 2\Delta x, \dots, x_6 = 0 + 6\Delta x = 2$.
- Now find the values of $f(x) = e^{-x^2}$ from $y(x_0)$ to $y(x_6)$ for the corresponding x_0 to x_6 .
- Finally, substitute all the above found values in the Simpson's Three-Eight Rule to calculate the approximate integral value.

MATLAB code for the above example

- The MATLAB recipe for the Simpson's Three-Eight Rule is as under:
- ```
function simpson38=Simpsononethreeeight(f,a,b,n)
format long
clear all;
clc;
close all;
sum1=0;
sum2=0;
a=0;
b=2;
n=6;
f = inline('exp(-x^2)');
dx=(b-a)/n;
value=f(b)+f(a);
```



## MATLAB code for the above example

```
for k=1:n-1
c=a+k*dx;
if mod(k,3)==0
sum1=sum1+2*f(c);
else
sum2=sum2+3*f(c);
end
end
sum=sum1+sum2;
Answer=(value+sum)*(3*dx/8)
```

## Simpson's Three-Eight Rule to approximation the area.

```

1 function x = SimpsonThreeEightRule(a,b,n) %Simpson's Three-Eight Rule
2 format long
3 clear all;
4 clc;
5 close all;
6 xval=[];
7 sum1=0;
8 A=0;
9 b=2;
10 a=0;
11 h=(b-a)/(n-1); % Uniform length of each subinterval
12 % Function values at the end points of the interval [a,b]
13 for k=1:n-1
14 xval(k)=a+h*k;
15 if mod(k,2)~=0
16 sum1=sum1+2*f(x);
17 else
18 sum2=sum2+3*f(x);
19 end
20 end
21 m10;
22 sum=sum1+sum2;
23 x = (sum*h)/3; % Final result
24
25 SimpsonThreeEightRule.m

```

Command Window: 0.93982299159

Command History:
 

```

> x = SimpsonThreeEightRule(0,2,6)
> x = 0.93982299159

```

Figure: Evaluation of the definite integral of the function  $e^{-x^2}$  using Simpson's Three-Eight Rule on  $[0,2]$  for  $n=6$ .



- For the following integral

$$\int_1^5 \sqrt{1+x^3} dx. \quad (3.1)$$

- Compute the Riemann sum approximation using 100 equally spaced sub-intervals and left end points.
- Compute the Riemann sum approximation using 100 equally spaced sub-intervals and right end points.
- Compute the Riemann sum approximation using 100 equally spaced sub-intervals and mid points.

## Exercise

- Compute the following integral

$$\int_1^5 \sqrt{(1+x^3)} dx \quad (3.2)$$

- Using the Trapezoidal Rule considering 10 equally spaced sub-intervals.
- Using the Trapezoidal Rule considering 100 equally spaced sub-intervals.
- Using the Trapezoidal Rule considering 1000 equally spaced sub-intervals.

Exercise

- Compute the following integral

$$\int_1^5 \sqrt{(1+x^3)} dx \quad (3.3)$$

- Using the Simpson's One-Third Rule considering 6 equally spaced sub-intervals.
- Using the Simpson's One-Third Rule considering 30 equally spaced sub-intervals.
- Using the Simpson's One-Third Rule considering 300 equally spaced sub-intervals.



Exercise

- Compute the following integral

$$\int_1^5 \sqrt{(1+x^3)} dx \quad (3.4)$$

- Using the Simpson's Three-Eight Rule considering 6 equally spaced sub-intervals.
- Using the Simpson's Three-Eight Rule considering 30 equally spaced sub-intervals.
- Using the Simpson's Three-Eight Rule considering 300 equally spaced sub-intervals.



# Orthogonal Polynomials

Department of Mathematics and Statistics  
Virtual University of Pakistan, Lahore, Pakistan.

*imrantalib@vu.edu.pk*



# Outline

## 1 Importance of orthogonal polynomials

- Orthogonal Functions
- Orthonormal Functions
- Orthogonal Polynomials
- Orthonormal Polynomials
- Properties of Orthogonal Polynomials

## 2 Some Important Orthogonal Polynomials

- Orthogonal Shifted Jacobi Polynomials
- Exercise
- Weight function
- Exercise
- Orthogonality Condition
- Exercise

## 3 Function Approximation using Jacobi Polynomials

- MATLAB code to determine the constant factor  $R_T^{(\alpha, \beta)}$ ,  $j = 0, 1, 2, \dots, n, \dots$
- Exercise

- MATLAB code to determine the series coefficients  $C_j$ ,  $j = 0, 1, 2, \dots$



# Outline

## 1 Importance of orthogonal polynomials

- Orthogonal Functions
- Orthonormal Functions
- Orthogonal Polynomials
- Orthonormal Polynomials
- Properties of Orthogonal Polynomials

## 2 Some Important Orthogonal Polynomials

- Orthogonal Shifted Jacobi Polynomials
- Exercise
- Weight function
- Exercise
- Orthogonality Condition
- Exercise

## 3 Function Approximation using Jacobi Polynomials

- MATLAB code to determine the constant factor  $R_T^{(\alpha, \beta)}$ ,  $j = 0, 1, 2, \dots, n, \dots$
- Exercise

- MATLAB code to determine the series coefficients  $C_j$ ,  $j = 0, 1, 2, \dots$



# Outline

## 1 Importance of orthogonal polynomials

- Orthogonal Functions
- Orthonormal Functions
- Orthogonal Polynomials
- Orthonormal Polynomials
- Properties of Orthogonal Polynomials

## 2 Some Important Orthogonal Polynomials

- Orthogonal Shifted Jacobi Polynomials
- Exercise
- Weight function
- Exercise
- Orthogonality Condition
- Exercise

## 3 Function Approximation using Jacobi Polynomials

- MATLAB code to determine the constant factor  $R_{T,j}^{(\alpha,\beta)}$ ,  $j = 0, 1, 2, \dots, n, \dots$
- Exercise
- MATLAB code to determine the series coefficients  $C_i$ ,  $i = 0, 1, 2, \dots$



# Outline

## 1 Importance of orthogonal polynomials

- Orthogonal Functions
- Orthonormal Functions
- Orthogonal Polynomials
- Orthonormal Polynomials
- Properties of Orthogonal Polynomials

## 2 Some Important Orthogonal Polynomials

- Orthogonal Shifted Jacobi Polynomials
- Exercise
- Weight function
- Exercise
- Orthogonality Condition
- Exercise

## 3 Function Approximation using Jacobi Polynomials

- MATLAB code to determine the constant factor  $R_{T,j}^{(\alpha,\beta)}$ ,  $j = 0, 1, 2, \dots, n, \dots$
- Exercise
- MATLAB code to determine the series coefficients  $C_i$ ,  $i = 0, 1, 2, \dots$



- Orthogonal polynomials are very much important in view of the development of the numerical schemes of certain types of differential equations.
- The numerical solution of many fractional and integer order ordinary and partial differential equations is based on the orthogonal polynomials. A number of research articles are available in the existing literature in which the numerical schemes have been developed using orthogonal polynomials.
- These polynomials exhibit the numerical solution in a series form.
- Frequently used orthogonal polynomials are Legendre polynomials, Jacobi polynomials, Bernstein polynomials, and Hermite polynomials.
- There we get the precise idea of these polynomials.



- Two functions  $f_i(x)$  and  $f_j(x)$  are orthogonal on an interval  $[a, b]$ , if

$$\int_a^b f_i(x)f_j(x)dx = 0 \quad (1.1)$$

- Mathematically, we say that the inner product of the functions  $f_i(x)$  and  $f_j(x)$  is zero.





- Two functions  $f_i(x)$  and  $f_j(x)$  are said to be orthonormal, if

$$\int_a^b f_i(x) f_j(x) dx = \delta_{ij}, \quad (1.2)$$

where

$$\begin{aligned} \delta_{ij} &= 1, & i &= j \\ &= 0, & i &\neq j \end{aligned} \quad (1.3)$$



- A set of orthogonal polynomials is an infinite sequence of polynomials,  $p_0(x), p_1(x), p_2(x), \dots, p_n(x), \dots$
- Where  $p_n(x)$  is a polynomial of degree  $n$ .
- Any two polynomials in the set are orthogonal to each other, that is,

$$\int_a^b p_i(x)p_j(x)dx = 0, \quad i \neq j \quad (1.4)$$



- The set of polynomials is orthonormal, if

$$\int_a^b p_i(x)p_j(x)dx = \delta_{ij}. \quad (1.5)$$

**Note:**

- The interval  $[a, b]$  is known as the interval of orthogonality and may be infinite at one or both ends.
- Any infinite sequence of polynomials  $p_n$ , with  $p_n$  having degree  $n$  forms a basis for the infinite-dimensional vector space of all polynomials.



- Any polynomial  $g(x)$  of degree  $n$  can be expressed as a linear combination of orthogonal polynomials:

$$g(x) = \sum_{i=0}^n c_i p_i(x). \quad (1.6)$$

- Where  $c_i$  is constant and can be determined easily.
- Given an orthogonal set of polynomials  $p_0(x), p_1(x), p_2(x), \dots, p_n(x), \dots$ , each polynomial,  $p_k(x)$ , is orthogonal to any polynomial of degree  $< k$ .
- Each polynomial in  $p_0(x), p_1(x), p_2(x), \dots, p_n(x), \dots$  has all  $n$  of its roots real distinct and strictly within the interval of orthogonality. That is not on its ends.



- Jacobi polynomials was first introduced by Carl Gustav Jacob Jacobi (1804–1851).
- These polynomials are solutions of the second order homogeneous differential equations of the form:

$$(1 - x^2)y''(z) + (\beta - \alpha - (\beta + \alpha + 2))y'(z) + n(n + \alpha + \beta + 1)y(z) = 0. \quad (2.1)$$

- The two parametric shifted Jacobi polynomials defined on  $[0, \tau]$  in one dimension is defined as:

$$P_n^{(\alpha, \beta)}(x) = \sum_{k=0}^i \frac{(-1)^{(i-k)} \Gamma(i + \beta + 1) \Gamma(i + k + \alpha + \beta + 1)}{\Gamma(k + \beta + 1) \Gamma(i + \alpha + \beta + 1) \Gamma(i - k + 1) \Gamma(k + 1)} \tau^k x^k, \quad (2.2)$$

- where  $i = 0, 1, 2, \dots$



## Orthogonal Shifted Jacobi Polynomials Contd.

- The two parametric orthogonal shifted Jacobi polynomials defined in (2.2) is also known as the function vector in one dimension.

### Example:

- Construct the function vector  $P_n^{(\alpha, \beta)}(x)$  (Orthogonal Shifted Jacobi Polynomials) defined on  $[0, T]$  of degree 0, 1, 2, and 3 using equation (2.2).

### Solution:

- For  $i = 0$ ,  $k$  is also equal to zero, and we get  $P_0(x) = 1$ .
- For  $i = 1$ ,  $k$  varies from 0 to 1, and we get  $P_1(x) = \frac{2x}{T} - 1$ .
- For  $i = 2$ ,  $k$  varies from 0 to 2, and we get  $P_2(x) = \frac{6x^2}{T^2} - \frac{6x}{T} + 1$ .
- For  $i = 3$ ,  $k$  varies from 0 to 3, and we get  $P_3(x) = \frac{12x}{T} - \frac{30x^2}{T^2} + \frac{20x^3}{T^3} - 1$ .



## Solution Contd.

- In matrix form (vector form), the required polynomials can be written as:

$$\begin{pmatrix} P_0(x) \\ P_1(x) \\ P_2(x) \\ P_3(x) \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{2x}{T} - 1 \\ \frac{6x^2}{T^2} - \frac{6x}{T} + 1 \\ \frac{12x}{T} - \frac{30x^2}{T^2} + \frac{20x^3}{T^3} - 1 \end{pmatrix}$$



## Solution Contd.

- The MATLAB code for the construction of function vector,  $P_n^{(\alpha, \beta)}(x)$  (Orthogonal Shifted Jacobi Polynomials) is as under:
- ```
function P=Jacobfunvec(M,alpha,beta)
clc;
clear all;
close all;
syms x Tk
alpha=0;
beta=0;
M=4;
for i=0:M-1
    a1 = (-1)^(i - k);
    a2 = gamma(i + beta + 1);
    a3 = gamma(i + k + alpha + beta + 1);
```

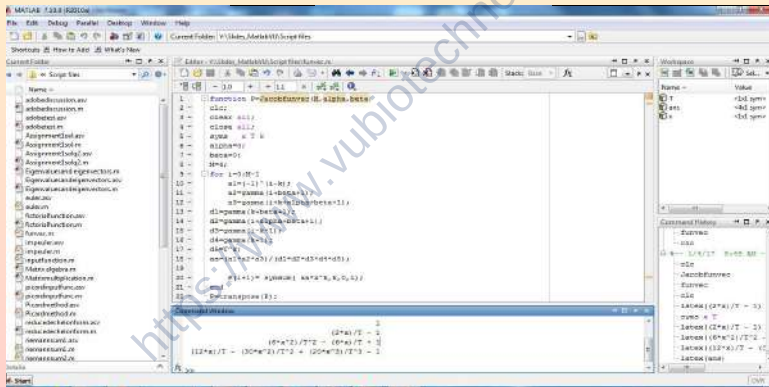



Solution Contd.

```
d1 = gamma(k + beta + 1);  
d2 = gamma(i + alpha + beta + 1);  
d3 = gamma(i - k + 1);  
d4 = gamma(k + 1);  
d5 = Tk;  
aa = (a1 * a2 * a3)/(d1 * d2 * d3 * d4 * d5);  
P(i + 1) = symsum(aa * xk, k, 0, i);  
end  
P = transpose(P);
```

Solution Contd.

- Function vector generated by using orthogonal shifted Jacobi polynomial, $P_n^{(\alpha, \beta)}(x)$.



visit our website to get more PDF

Figure: Orthogonal shifted Jacobi Polynomials generated by taking $j = 0, 1, 2, 3$



- Construct the function vectors (Orthogonal Shifted Jacobi Polynomials, $P_n^{(\alpha, \beta)}(x)$) defined in (2.2) on the interval $[0, 1]$ by taking $\alpha = 1$, $\beta = 2$, and scale level $M = 10$.

Note:

- Scale level M means, the parameter i varies from $0 : 10$. In this case, 10 function vectors of different degrees will be generated.



- The weight function $W_T^{(\alpha,\beta)}(x)$ is defined as under:

$$W_T^{(\alpha,\beta)}(x) = (T-x)^\alpha x^\beta, \quad (2.3)$$

where α and β are the parameters.

- The MATLAB code for defining the weight function is as follow:

- function `W=weightfuncjacob(x,alpha,beta,T)`

```
clc;
```

```
clear all;
```

```
close all;
```

```
syms T x alpha beta
```

```
W = xbeta * (T - x)alpha;
```



- Construct the weight function, $W_T^{(\alpha, \beta)}(x)$ defined in (2.3) on the interval $[0, 1]$ by taking $\alpha = 1, \beta = 2$.
- Construct the weight function, $W_T^{(\alpha, \beta)}(x)$ defined in (2.3) on the interval $[0, 1]$ by taking $\alpha = 0, \beta = 0$.



- The orthogonality condition for the Jacobian Polynomial with respect to weight function is as under:

$$\int_0^T P_{T,i}^{(\alpha,\beta)}(x) P_{T,j}^{(\alpha,\beta)}(x) W_{T,j}^{(\alpha,\beta)}(x) dx = R_{T,j}^{(\alpha,\beta)} \delta_{(i,j)}, \quad (2.4)$$

where

$$R_{T,j}^{(\alpha,\beta)} = \frac{T^{\alpha+\beta+1} \Gamma(j+\alpha+1) \Gamma(j+\beta+1)}{(2j+\alpha+\beta+1) \Gamma(j+1) \Gamma(j+\alpha+\beta+1)} \quad (2.5)$$



Exercise:

- Check the orthogonality of orthogonal shifted Jacobi polynomial, $P_n^{(\alpha, \beta)}(x)$ on $[0, T]$, by choosing $\alpha = 1$, $\beta = 1$ with respect to the weight function $W_T^{(\alpha, \beta)}(x)$.



- The orthogonality relation defined above allows us to approximate $f(x) \in C([0, T])$ in terms of Jacobi series as follows:

$$f(x) = \sum_{i=0}^{\infty} C_i P_i^{(\alpha,\beta)}(x), \quad (3.1)$$

where $C([0, T])$ is a space of continuous functions defined on $[0, T]$.



Function Approximation using Jacobi Polynomials Contd.

- The series coefficients C_i can be easily determined using the orthogonality relationship of Jacobi Polynomials. That is

$$C_i = \frac{1}{R_{T,j}^{(\alpha,\beta)}} \int_0^T f(x) W_T^{(\alpha,\beta)}(x) P_i^{(\alpha,\beta)}(x) dx. \quad (3.2)$$



- The constant factor $R_{T,j}^{(\alpha,\beta)}$ defined in (2.5) is computed in MATLAB as
- ```
function R=testcaseorthogonalitycheck(alpha,beta,T,M)
clc;
clear all;
close all;
syms i alpha beta T
M=4;
alpha=1;
beta=1;
T=1;
for i=0:M-1
a1 = T^(alpha+beta+1);
a2=gamma(i+alpha+1);
a3=gamma(i+beta+1);
```



# MATLAB code to determine the constant factor $R_{T,j}^{(\alpha,\beta)}$ contd.

```
d1=(2*i+alpha+beta+1);
d2=gamma(i+1);
d3=gamma(i+alpha+beta+1);
R(i+1)=(a1*a2*a3)/(d1*d2*d3);
end
R=transpose(R)
```



- Constant factor involved in orthogonality condition of Jacobian Polynomial.

```

10 = for s=0:5-1
11 = A1=1*(alpha+beta+1);
12 = a2=gamma(1+alpha+1);
13 = a3=gamma(1+beta+1);
14 = A1=(2*(1+alpha+beta+1));
15 = a2=gamma(1+1);
16 = a3=gamma(1+alpha+beta+1);
17 = B(1,1)=(A1*a2*a3)/(a2+a3);
18 = end;
19 = return(B);
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2
```



- Compute the constant factor defined in (2.5) for  $j = 0 : 5$  and for  $\alpha = 2$ , and  $\beta = 3$ .
- Compute the constant factor defined in (2.5) for choosing scale level  $M = 7$  and  $\alpha = 3$ , and  $\beta = 1$ .



- The series coefficients defined in (3.2) can be computed using MATLAB using the following code:
- ```
function C=coefvec(f,x,M,alpha,beta,T)
syms x f alpha beta T
alpha=1;
beta=1;
T=1;
M=4;
W=weightfuncjacob(x,alpha,beta,T);
P=Jacobfunvec(x,M,alpha,beta,T);
PW=f*P*W;
for i=0:M-1
R=orthogonal(i,alpha,beta,T);
aa=PW(i+1);
C(i+1)=(1/R)*int(aa,x,0,T);
end
C=transpose(C)
```

- Computation of the series coefficients C_j , $j = 0, 1, 2, 3, 4$ defined in (3.2).

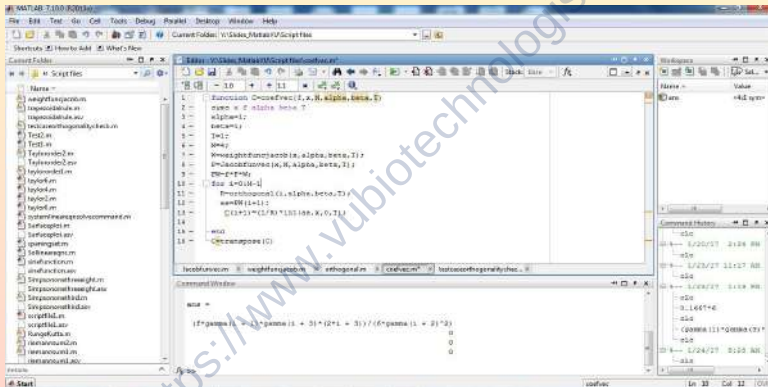


Figure: MATLAB code with generated output to compute the series coefficients C_i , $i = 0, 1, 2, 3, 4$.



- Compute the series coefficients C_i of Jacobi Polynomials for $i = 0 : 10$ and $\alpha = 1, \beta = 2$.
- Compute the series coefficients C_i of Jacobi Polynomials for scale level $M = 6$ and $\alpha = 2, \beta = 1$.



Orthogonal Polynomials II

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk



Outline

- 1 Orthogonal Legendre Polynomials
 - Applications of the Legendre's differential equation
 - Recurrence Relation to generate the Legendre Polynomials
 - Some Legendre Polynomials
 - MATLAB code to compute the Legendre Polynomials
 - Exercise
- 2 Orthogonal Shifted Legendre Polynomials
 - Example:
 - Exercise:
 - Orthogonality Condition
 - MATLAB code to determine the constant factor R_j , $j = 0, 1, 2, \dots, n, \dots$
 - Exercise
- 3 Function Approximation using Legendre Polynomials
 - MATLAB code to determine the series coefficients C_i , $i = 0, 1, 2, \dots$
- 4 Exercise



Outline

1 Orthogonal Legendre Polynomials

- Applications of the Legendre's differential equation
- Recurrence Relation to generate the Legendre Polynomials
- Some Legendre Polynomials
- MATLAB code to compute the Legendre Polynomials
- Exercise

2 Orthogonal Shifted Legendre Polynomials

- Example:
- Exercise:
- Orthogonality Condition
- MATLAB code to determine the constant factor R_j , $j = 0, 1, 2, \dots, n, \dots$
- Exercise

3 Function Approximation using Legendre Polynomials

- MATLAB code to determine the series coefficients C_i , $i = 0, 1, 2, \dots$

4 Exercise



Outline

1 Orthogonal Legendre Polynomials

- Applications of the Legendre's differential equation
- Recurrence Relation to generate the Legendre Polynomials
- Some Legendre Polynomials
- MATLAB code to compute the Legendre Polynomials
- Exercise

2 Orthogonal Shifted Legendre Polynomials

- Example:
- Exercise:
- Orthogonality Condition
- MATLAB code to determine the constant factor R_j , $j = 0, 1, 2, \dots, n, \dots$
- Exercise

3 Function Approximation using Legendre Polynomials

- MATLAB code to determine the series coefficients C_i , $i = 0, 1, 2, \dots$

4 Exercise



Outline

1 Orthogonal Legendre Polynomials

- Applications of the Legendre's differential equation
- Recurrence Relation to generate the Legendre Polynomials
- Some Legendre Polynomials
- MATLAB code to compute the Legendre Polynomials
- Exercise

2 Orthogonal Shifted Legendre Polynomials

- Example:
- Exercise:
- Orthogonality Condition
- MATLAB code to determine the constant factor R_j , $j = 0, 1, 2, \dots, n, \dots$
- Exercise

3 Function Approximation using Legendre Polynomials

- MATLAB code to determine the series coefficients C_i , $i = 0, 1, 2, \dots$

4 Exercise



- These polynomials arise as the solutions of the following differential equation known as Legendre's differential equation:

$$(1 - x^2) \frac{\partial^2 P(x)}{\partial x^2} - 2x \frac{\partial P}{\partial x} + \lambda P(x), \quad \lambda = n(n + 1). \quad (1.1)$$

- These polynomials are orthogonal on the interval $[-1, 1]$.



- Legendre's differential equation is frequently encountered in Physics, particularly when spherical polar coordinates are used and where the problems have cylindrical symmetry.
- Moreover, Legendre polynomials form part of the quantum mechanical solution for rotational motion.



- The Legendre Polynomials defined on $[-1, 1]$ are given by the following recurrence relation:

$$L_{i+1}(z) = \frac{2i+1}{i+1}zL_i(z) - \frac{i}{i+1}L_{i-1}(z), \quad \text{where } L_0(z) = 1, \quad L_1(z) = z \quad (1.2)$$

- $i = 1, 2, 3, \dots$



- Some Legendre Polynomials for different values of i are mentioned below:

$$P_0(z) = 1,$$

$$P_1(z) = z,$$

$$P_2(z) = \frac{1}{2}(3z^2 - 1),$$

$$P_3(z) = \frac{1}{2}(5z^3 - 3z),$$

$$P_4(z) = \frac{1}{8}(35z^4 - 30z^2 + 3),$$

and so on...

(1.3)



- The MATLAB code to compute the Legendre Polynomials defined in (1.3) using the relation (1.2) is given as under:
- ```
function P=Lagendrfucvector(M,T,value1,value2)
syms z
format short
M=2;
value(1)=z;
value(2)=sym((3/2)) * z^2 - sym(1/2);
for i=1:M
value(i+2)=((2 * i + 3)/(i + 2)) * z * value(i + 1) - ((i + 1)/(i + 2)) * value(i);
end
P=simplify(transpose(value));
```



- After execution of the above code, the following outputs are generated:

- $$\begin{pmatrix} P_1(z) \\ P_2(z) \\ P_3(z) \\ P_4(z) \end{pmatrix} = \begin{pmatrix} z \\ \frac{3z^2}{2} - \frac{1}{2} \\ z\left(\frac{5z^2-3}{2}\right) \\ \frac{35z^4}{8} - \frac{15z^2}{4} + \frac{3}{8} \end{pmatrix}$$



- Function vector generated by using orthogonal Legendre polynomial,  $P_n(x)$ .

The image shows a MATLAB script in the Editor window and the Command Window. The script defines a function `LegendreFunctionVector` that takes `N` and `xi` as inputs and returns a matrix of Legendre polynomials. The Command Window shows the execution of the function for `N=3` and `xi=0`, resulting in a matrix of values for the first four Legendre polynomials at  $x=0$ .

```

function P=LegendreFunctionVector(N,xi,value1,value2) %function to generate Legendre polynomials
%
%
format short
N=N;
value1=xi; % depends polynomial for i=1, then is P_1(x)=x
value2=xi+(2/2)+xi*(2/2); % Legendre polynomial for i=2, then is P_2(x)=1/2(3x^2-1)
for i=1:N
 value1=(2*i+1)/(i+2)*xi*value1+(i+1)/(i+2)*value1;
end
end=LegendreFunctionVector(N,xi,value1,value2);

```

```

>>
ans =
 1.0000 0.0000 0.0000 0.0000
 0.0000 0.0000 0.0000 0.0000
 0.0000 0.0000 0.0000 0.0000
 0.0000 0.0000 0.0000 0.0000

```

Figure: Orthogonal Legendre Polynomials generated by taking  $i = 0, 1, 2, 3$



- Determine the Legendre Polynomials using the relation (1.2) for  $i = 1 : 20$  using the above mentioned MATLAB code with slightly modification. That is we have to be determined  $P_3(z), P_4(z), P_5(z), \dots, P_{20}(z)$  using  $P_1(z) = z$ , and  $P_2(z) = \frac{3z^2}{2} - \frac{1}{2}$ .
- Moreover compute the values of the above Legendre Polynomials at  $z = 0, z = 1$ , and  $z = 5$ .



- The transformation  $x = \frac{(z+1)}{2}$  in relation (1.2) transforms the interval  $[-1, 1]$  to  $[0, 1]$ , then the Orthogonal shifted Legendre polynomials are given by

$$P_i(x) = \sum_{k=0}^i (-1)^{(i+k)} \frac{\Gamma(i+k+1)}{\Gamma(i-k+1)(\Gamma(k+1))^2}, \quad i = 0, 1, 2, \dots \quad (2.1)$$

- It can be easily checked that  $P_i(0) = (-1)^i$ , and  $P_i(1) = 1$ .



- Construct the function vector  $P_n(x)$  (Orthogonal Shifted Legendre Polynomials) defined on  $[0, T]$  of degree 0, 1, 2, and 3 using relation (2.1).

### Solution:

- For  $i = 0$ ,  $k$  is also equal to zero, and we get  $P_0(x) = 1$ .
- For  $i = 1$ ,  $k$  varies from 0 to 1, and we get  $P_1(x) = 2x - 1$ .
- For  $i = 2$ ,  $k$  varies from 0 to 2, and we get  $P_2(x) = 6x^2 - 6x + 1$ .
- For  $i = 3$ ,  $k$  varies from 0 to 3, and we get  $P_3(x) = 20x^3 - 30x^2 + 12x - 1$ .



## Solution Contd.

- In matrix form (vector form), the required polynomials can be written as:

$$\begin{pmatrix} P_0(x) \\ P_1(x) \\ P_2(x) \\ P_3(x) \end{pmatrix} = \begin{pmatrix} 1 \\ 2x - 1 \\ 6x^2 - 6x + 1 \\ 20x^3 - 30x^2 + 12x - 1 \end{pmatrix}$$





## Solution Contd.

- The MATLAB code for the construction of function vector,  $P_n(x)$  (Orthogonal Shifted Legendre Polynomials) is as under:

- `function P=Lagdrfunvec(M,T)`

```
syms x T k
```

```
M = 4;
```

```
for i = 0 : M - 1
```

```
 a1 = (-1)^(i+k);
```

```
 a2 = gamma(i + k + 1);
```

```
 d1 = gamma(i - k + 1);
```

```
 d2 = (gamma(k + 1))^2;
```

```
 aa = (a1 * a2)/(d1 * d2);
```

```
 P(i + 1) = symsum(aa * x^k, k, 0, i);
```

```
end
```

```
P = transpose(P);
```



## Solution Contd.

- Function vector generated by using orthogonal shifted Legendre polynomial,  $P_n(x)$ .

The screenshot shows the MATLAB 7.10.0 (R2010b) environment. The script file 'LegendFunctions.m' is open, showing the following code:

```

1 function P = LegendFunctions(n)
2 syms x T h
3 h=4;
4 for i=0:n-1
5 a1=(1-i)*(1+i);
6 a2=gamma(1-i+1);
7 a3=gamma(1-i+1);
8 a4=(gamma(1-i+1))^2;
9 a5=(x+1)/(1+a2);
10 P(i+1)= a5*a4*a3*a2*a1;
11 end
12 P=simplify(P);

```

The Command Window shows the output of the script:

```

>> P =
 1
 3*x - 1
 5*x^2 - 3*x + 1
 7*x^3 - 6*x^2 + 2*x - 1

```

Figure: Orthogonal shifted Legendre Polynomials generated by taking  $i = 0, 1, 2, 3$   
visit our website to get more PDF <https://www.vubiotechnologist.gq>



- Construct the function vector  $P_n(x)$  (Orthogonal Shifted Legendre Polynomials) defined on  $[0, T]$  for  $i = 0 : 10$  using relation (2.1).
- Construct the function vector  $P_n(x)$  (Orthogonal Shifted Legendre Polynomials) defined on  $[0, T]$  for the scale level  $M = 15$  using relation (2.1).



- The orthogonality condition for the Legendre Polynomial is as under:

$$\int_0^1 P_i(x)P_j(x)dx = R_j\delta_{(i,j)}, \quad (2.2)$$

where

$$R_j = \frac{1}{2j+1} \quad (2.3)$$

and

$$\begin{aligned} \delta_{ij} &= 1, & i &= j \\ &= 0, & i &\neq j \end{aligned} \quad (2.4)$$



- The constant factor  $R_j$  defined in (2.5) is computed in MATLAB as

- function R=Lagendrorthogonal(i,M)

```
syms i
```

```
M=4;
```

```
for i=0:M-1
```

```
 a1=1;
```

```
 a2=2*i+1;
```

```
 R(i+1)=a1/a2;
```

```
end
```

```
R
```

- Constant factor involved in orthogonality condition of Legendre Polynomial.

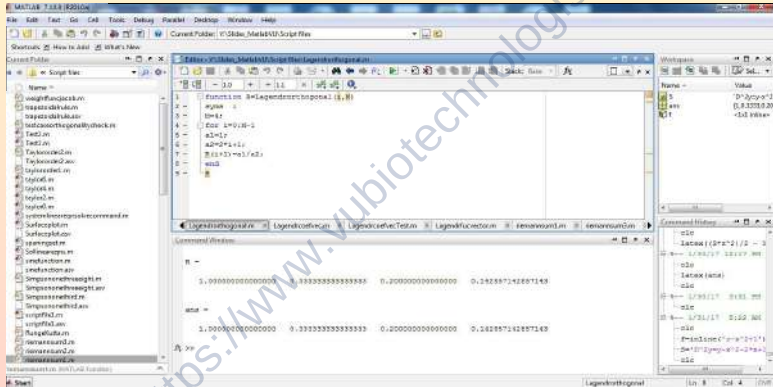


Figure: MATLAB code for generating the constant factor  $R_j$ .



- Compute the constant factor defined in (2.3) for  $j = 0 : 6$ .
- Compute the constant factor defined in (2.3) for choosing scale level  $M = 10$ .



- The orthogonality relation defined in (2.2) allows us to approximate  $f(x) \in C([0, 1])$  in terms of Legendre series as follows:

$$f(x) = \sum_{i=0}^{\infty} C_i P_i(x), \quad (3.1)$$

where  $C([0, 1])$  is a space of continuous functions defined on  $[0, 1]$ .





## Function Approximation using Legendre Polynomials Contd.

- The series coefficients  $C_i$  can be easily determined using the orthogonality relationship of Legendre Polynomials. That is

$$C_i = \frac{1}{R_j} \int_0^1 f(x) P_i(x) dx. \quad (3.2)$$



- The series coefficients defined in (3.2) can be computed in MATLAB using the following code:

- `function C=Lagendrcoefvec(M,T,f,i)`

`syms x T i f`

`M=4;`

`P=Lagdrfunvec(M,T);`

`R=Lagendrorthogonal(i,M);`

`Kw=f*P;`

`for i=0:M-1`

`aa=Kw(i+1);`

`C(i+1)=(1/(R(i+1)))*int(aa,x,0,T);`

`end`

`C=transpose(C);`



- Computation of the series coefficients  $C_i$ ,  $i = 0, 1, 2, 3, 4$  defined in (3.2).

```

function C=LegendreCoefficients(f,T,fx)
%
% Legendre coefficients of f(x) on the interval [T(1),T(2)]
%
% Input:
% f: function handle or string
% T: vector of two elements [a,b]
% fx: function handle or string
%
% Output:
% C: vector of Legendre coefficients
%
% Example:
% C=LegendreCoefficients('x^2',[0,1])
%
% C =
% 0.57735
% 0.57735
% 0.57735
%
% Copyright 2008 by the author. All rights reserved.

% Get the Legendre polynomials
[N,P]=legendre(N,T);

% Compute the coefficients
C=zeros(1,N);
for i=0:N
 C(i+1)=(1/(P(i+1))) * int(f(x),T(1),T(2)));
end
C=orthonormal(C);

```

Command Window:

```

>>
C =
 0.57735
 0.57735
 0.57735
 0.57735
 0.57735

```

Figure: MATLAB code with generated output to compute the series coefficients  $C_i$ ,  $i = 0, 1, 2, 3, 4$ .



- Compute the series coefficients  $C_i$  for  $i = 0 : 10$  using relation (3.2).
- Compute the series coefficients  $C_i$  for scale level  $M = 6$  and using relation (3.2) .



## Exercise

- Determine the Legendre Polynomials using the relation (1.2) for  $i = 1 : 10$ . That is we have to be determined  $P_3(z)$ ,  $P_4(z)$ ,  $P_5(z)$ , ...  $P_{10}(z)$  using  $P_1(z) = z$ , and  $P_2(z) = \frac{3z^2}{2} - \frac{1}{2}$ .
- Moreover compute the values of the above Legendre Polynomials at  $z = 3$ ,  $z = 4$ , and  $z = 6$ .



## Exercise

- Construct the function vector  $P_n(x)$  (Orthogonal Shifted Legendre Polynomials) defined on  $[0, 1]$  for  $i = 0 : 20$  using relation (2.1). Moreover compute the values of  $P_n(x)$  at  $x = 2$ .
- Construct the function vector  $P_n(x)$  (Orthogonal Shifted Legendre Polynomials) defined on  $[0, 1]$  for the scale level  $M = 30$  using relation (2.1). Moreover compute the values of  $P_n(x)$  at  $x = 0$ .



## Exercise

- Compute the constant factor defined in (2.3) for  $j = 0 : 10$  .
- Compute the constant factor defined in (2.3) for choosing scale level  $M = 40$ .